

MONOPOLY.LK

A Sri Lanka themed Monopoly simulation in C

Viva Preparation Guide - every file, function and rule explained

With full source-code appendix

විභාගයට සූදානම් වීමේ සම්පූර්ණ මාර්ගෝපදේශය

Table of Contents

1	About This Guide.....	4
2	Project Overview.....	5
	2.1 What is MONOPOLY.LK?.....	5
	2.2 Technology used.....	5
	2.3 The three design rules of this codebase.....	5
	2.4 The GameState game[1] idiom.....	5
	2.5 How the modules divide the work.....	5
3	How to Build and Run.....	7
4	The Code Tree (File Structure).....	8
5	Architecture and Execution Flow.....	9
	5.1 Program start.....	9
	5.2 A single turn (playTurn).....	9
	5.3 What does a 'round' mean here?.....	9
	5.4 Round-end bookkeeping (playGame, one round).....	9
	5.5 How the modules talk to each other.....	9
6	The Data Model (types.h).....	11
7	Module Deep Dives.....	13
	7.1 main.c - the entry point.....	13
	7.2 board.c - building the 40-square board.....	13
	7.3 players.c - the 4 AI strategies.....	14
	7.4 game.c - the game engine.....	15
	7.5 finance.c - money, rent, buying, buildings, taxes, bankruptcy.....	15
	7.6 bank.c - loans, collateral and foreclosure.....	16
	7.7 auction.c - the auction engine.....	17
	7.8 events.c - cards, economic events and regulations.....	17
	7.9 economy.c - inflation, modifiers, ageing, maintenance.....	18
	7.10 insurance.c - policies, disasters, repairs.....	19
	7.11 market.c - market review, regional cards, report.....	19
	7.12 functions.h - the project's 'API'.....	20
8	Game Rules - Quick Reference.....	21
	8.1 Money.....	21
	8.2 Board.....	21
	8.3 Turn order.....	21
	8.4 Rent (Table 6).....	21
	8.5 Buying.....	21
	8.6 Building (Rule 9).....	21
	8.7 Mortgage.....	21
	8.8 Bank loans (Rule-LK 2-7).....	21
	8.9 Jail (Rule 13).....	21
	8.10 Tax.....	21
	8.11 Insurance.....	22
	8.12 Disasters.....	22
	8.13 Ageing & maintenance.....	22
	8.14 Market review (Rule-LK 30-34).....	22
	8.15 Cards & events.....	22
	8.16 Anti-Speculation Act (Rule-LK 8).....	22
	8.17 Bankruptcy.....	22
	8.18 Winning.....	22
9	Key Algorithms Explained.....	23
	9.1 Even development (Rule 9).....	23

9.2	The modifier multiplication (Rule-LK 34).....	23
9.3	Compounding loan interest.....	23
9.4	The real definition of a round.....	23
9.5	Weighted disaster selection.....	23
9.6	Recursive turn-order resolution.....	23
9.7	Inflation model.....	23
10	Verification and Testing.....	24
10.1	How verify_analysis.py works.....	24
10.2	The 6 confirmed bugs that were found and fixed.....	24
10.3	What the analyzer's REVIEW items really mean.....	24
11	Likely Viva Questions.....	25
12	Complete Function Index.....	27
12.1	main.c.....	27
12.2	board.c.....	27
12.3	players.c.....	27
12.4	finance.c.....	27
12.5	bank.c.....	27
12.6	auction.c.....	27
12.7	events.c.....	28
12.8	economy.c.....	28
12.9	insurance.c.....	28
12.10	market.c.....	28
12.11	game.c.....	28
13	Appendix - Complete Source Code.....	29
A.1	types.h - the data model (362 lines).....	29
A.2	functions.h - the project API (185 lines).....	33
A.3	main.c - entry point (18 lines).....	36
A.4	board.c - board construction (206 lines).....	36
A.5	players.c - the 4 AI strategies (406 lines).....	39
A.6	game.c - the game engine (641 lines).....	44
A.7	finance.c - the economic core (1077 lines).....	52
A.8	bank.c - loans and foreclosure (395 lines).....	66
A.9	auction.c - the auction engine (179 lines).....	72
A.10	events.c - cards and regulations (403 lines).....	74
A.11	economy.c - inflation and modifiers (504 lines).....	79
A.12	insurance.c - policies and disasters (393 lines).....	86
A.13	market.c - market review and regional cards (431 lines).....	91
A.14	verify_analysis.py - the verification analyzer (1459 lines).....	97
A.15	Rent.csv - property price/rent data (24 lines).....	115

1 About This Guide

This guide explains every part of the MONOPOLY.LK project so you can answer viva questions confidently. It was written directly from the source code, so every description, function name and code snippet matches the actual project.

- Section 2 - what the project is, what technology it uses, and why it was designed that way.
- Section 3 - how to build and run the program.
- Section 4 - the complete code tree (file structure).
- Section 5 - how the program flows: startup, one turn, one round, and how modules connect.
- Section 6 - the data model: every constant, enum and struct in types.h.
- Section 7 - a deep dive into each code file, function by function.
- Section 8 - every game rule of MONOPOLY.LK as a quick reference.
- Section 9 - the tricky algorithms explained step by step.
- Section 10 - verification and the 6 bugs that were found and fixed.
- Section 11 - likely viva questions with model answers.
- Section 12 - a full index of every function in the project.
- Section 13 - the complete source code of every file (appendix).

මම මාර්ගෝපදේශය ඒරැණ විස්තර කිරීමට අවශ්‍ය සියලු විස්තර අඩංගු කරයි - කේත අංශය තුළ නියත සංභල ඉඟි ඔබට සංකල්ප මතක තබා ගැනීමට උපකාරී වේ.

2 Project Overview

2.1 What is MONOPOLY.LK?

MONOPOLY.LK is a completely text-based computer simulation of a Sri Lanka themed Monopoly board game. There are no humans - the four seats are filled by four computer-controlled players, each with a different investment personality (an AI strategy). The program runs an entire game from start to finish and prints every action as text, like a narrated game log.

Beyond the classic Monopoly rules, the assignment adds many economic extensions that are modelled in code: bank loans with interest and collateral, mortgages and redemption, insurance policies, natural disasters, inflation, property market booms and declines, regional development cards, national event cards, government regulations, property ageing and depreciation, building maintenance and even an anti-speculation act.

මෙය ස්ථික් වියර්ත්ම ක්‍රීඩාවකි (ස්ථික් කොප්පු), පාලනය සම්පූර්ණ යනේම AI ක්‍රීඩකයින් 4 දෙනෙකු අතර සිදු වේ.

2.2 Technology used

- **Language**

C, written in an old-school, disciplined style (declarations at the top of every block, no fancy shortcuts).

- **Libraries**

Only the C standard library: `stdio.h`, `stdlib.h`, `string.h`, `time.h`. There is no third-party code, no graphics, no database.

- **Compiler**

gcc 16.1.0 (MSYS2). The whole program builds with one command: `gcc *.c -o monopoly`

- **Verification**

A Python 3 script (`verify_analysis.py`) that replays the printed game log and checks every money transaction for bugs.

- **Platform**

A Windows console application (`monopoly.exe`).

2.3 The three design rules of this codebase

Looking at the git history you will see three rules the author followed on purpose. In a viva, explaining these shows you understand why the code looks the way it does:

1. No pointer parameters - every function receives the game state and reads and writes it through array syntax. That is why nearly every function looks like `foo(GameState game[], ...)`.
2. No ternary operators - no `x = a ? b : c` ;. Everything is written with plain `if/else` so the logic is easy to read and check.
3. No global variables - all state lives inside one `GameState` struct created in `main()` and passed down through the whole program.

2.4 The GameState game[1] idiom

The most important design decision is the shared state object:

```
/* main.c - the only place a GameState is ever created */
GameState game[1] = {0}; /* a one-element array, all fields zeroed */

startGame(game);          /* passed as array, read/written as game[0].xxx */
```

`GameState` contains everything the game needs to remember: the 40-square board, the 4 players, and the economy. Because pointers were not allowed, it is declared as a one-element array. In C, passing an array to a function and writing `game[0].field` silently changes the original object - the array parameter is (behind the scenes) a pointer, but the source code never has to say so. This gives every function read/write access to all state without a single explicit pointer.

සෑම function එක `GameState game[]` ලබාගෙන `game[0]` හරහා සම්පූර්ණ තත්ත්වය (state) කියවන/ලියන කරයි.

2.5 How the modules divide the work

File	Side-effect / responsibilities	More details in
main.c	Entry point; creates GameState; calls startGame().	7.1
types.h	All constants, enums and structs (the data model).	6
functions.h	Prototypes of every function in the project.	7.12
board.c	Builds the 40-square Sri Lanka board.	7.2
players.c	The 4 AI strategies (all the decisions).	7.3
game.c	Turn loop, rounds, dice, jail, winner.	7.4
finance.c	Money, rent, buying, mortgage, building, bankruptcy.	7.5
bank.c	Loans, collateral, foreclosure, Bank visits.	7.6
auction.c	The auction engine for unclaimed/forced sales.	7.7
events.c	Event cards, economic events, regulations.	7.8
economy.c	Inflation, modifiers, ageing, maintenance, LKR formatting.	7.9
insurance.c	Insurance policies, disasters, repairs.	7.10
market.c	Market boom/decline, regional cards, condition display.	7.11
verify_analysis.py	Python verification harness (test tool, not the game).	10

3 How to Build and Run

Everything is plain C, so any gcc works. From the project folder:

```
gcc *.c -o monopoly      # compile all .c files into monopoly.exe
monopoly.exe             # run the game (prints the whole game to screen)
monopoly.exe > run.txt   # capture the game log for verification
```

[Note] `gcc \*.c -o monopoly` produces a clean build with zero warnings or errors. Each .c file #includes types.h and functions.h, and main.c + game.c are the only places that need the whole picture.

To run the automatic verifier on a saved log:

```
python verify_analysis.py run.txt # replays the log and reports [BUG]/[REVIEW] items
```

කිරීමට කිරීමට කිරීමට මගේම අවශ්‍ය නැත.

4 The Code Tree (File Structure)

Here is the whole project laid out as a tree, with the size of each file and a one-line description. Memorising this order (types -> headers -> game flow -> economy files) helps you describe the architecture in a viva.

```
MONOPOLY.LK/
|
|-- types.h           (361 lines)  ALL constants, enums, structs - the data model
|-- functions.h       (184 lines)  prototypes of every function, grouped by module
|
|-- main.c            (17 lines)   entry point - creates GameState, calls startGame
|-- game.c            (640 lines)  main loop, turns, dice, jail, rounds, winner
|-- board.c           (205 lines)  the 40-square Sri Lanka board
|-- players.c         (405 lines)  the 4 AI strategies / decision functions
|-- finance.c         (1076 lines) money, rent, buying, mortgage, buildings, taxes
|-- bank.c            (394 lines)  loans, collateral, foreclosure, Bank visits
|-- auction.c         (178 lines)  the auction engine
|-- events.c          (402 lines)  event cards, economic events, regulations
|-- economy.c         (503 lines)  inflation, modifiers, ageing, maintenance, LKR format
|-- insurance.c       (392 lines)  insurance policies, disasters, repairs
|-- market.c         (430 lines)  market boom/decline, regional cards, conditions
|
|-- verify_analysis.py (1458 lines) Python log-replay verification analyzer
|-- Rent.csv          (23 lines)   property price/rent table (data export)
|
|-- output/
|   |-- verification_report.txt    reported bugs + the 6 confirmed fixes
|
|-- run.txt / run2.txt / run3.txt / run4.txt / test.txt  game logs (verification)
|-- monopoly.exe      the compiled Windows binary
|-- monopoly_final.zip the submission archive
|
|-- generate_viva_pdf.py  THIS script - produces the study guide PDF
```

Roughly half the code is 'game logic' (finance.c, economy.c, bank.c, insurance.c, market.c, auction.c, events.c) and the other half is 'control flow and decisions' (game.c, board.c, players.c, main.c).

පුද්ගලයෙකු ජයග්‍රහණය කරන්නේ game.c ඇතුළු ඒර්ධාන ප්‍රංඛණේලිය හරහාය.

5 Architecture and Execution Flow

5.1 Program start

```
main.c main()
| creates GameState game[1] = {0}
v
game.c startGame(game)
| srand(time(NULL)) -> random seed so each game differs
| initializeBoard(game) (board.c) -> fills board[0..39]
| initializePlayers(game) (players.c) -> 4 AI players, 30,000 LKR each
| initEconomy(game) (economy.c) -> interest rate, tax rate, modifiers[0]
| initMarket(game) (market.c) -> group cooldowns cleared
| determineTurnOrder(game) (game.c) -> recursive dice roll-off for who starts
| playGame(game, order) -> the whole game
| displayFinalResults(game) -> winner + final standings
```

5.2 A single turn (playTurn)

This is the heart of the game. One AI player's whole turn, in order:

```
playTurn(game, playerIndex)
1. tryAutoRepair / performMaintenance / renovateStructuralDamage
   -> fix any damaged, worn or neglected buildings first
2. handleJail -> if in jail: doubles frees you / pay bail / forced after 3 turns
   (if jail handling ran, the turn ENDS - no movement)
3. rollDice() -> two d6 summed (2..12)
4. movePlayer -> pos = (pos + dice) % 40 ; if it wrapped, collect LKR 2000 (GO)
5. resolve landing by square type:
   PROPERTY/RAILWAY/UTILITY -> payRent then buyProperty (else -> auction)
   EVENT -> executeEvent (national event card)
   TAX -> payTax (15% of net worth, market-scaled)
   COMMUNITY_FUND -> payCommunityFundTax (10% of property value)
   GO_TO_JAIL -> teleport to square 10, marked in jail
   BANK -> handleBankVisit (loans)
   INSURANCE -> handleInsuranceVisit
   GO / JAIL(visiting) / FREE_PARKING -> nothing
6. constructBuildings -> keep building while affordable (even development)
7. sellDecliningProperties -> Opportunistic Trader dumps declining properties
8. handleMortgageDecisions -> redeem a mortgage, then maybe raise a new one
```

5.3 What does a 'round' mean here?

This is an unusual and important detail. In a normal board game a round is 'everyone takes one turn'. Here a round only ends when EVERY solvent player has passed GO at least once. Because players move different distances each turn, finishing the round usually takes several turns per player. playGame keeps cycling the turn order until allPassed becomes true.

```
while round <= 500:
    take next player's turn (skip bankrupt players)
    if that player passed GO then flag them
    if EVERY non-bankrupt player has passed GO this round:
        -> round-end bookkeeping (5.4) and start a new round
```

5.4 Round-end bookkeeping (playGame, one round)

```
processLoans(game) (bank.c) interest added to principal; foreclose at 0 rounds
processInsuranceExpiry(game) (insurance.c) policies count down, expire at 0
ageProperties(game) (economy.c) age +1; past 50 -> depreciation +5% per 5 rounds
ageBuildings(game) (economy.c) condition -2; neglect > 20 -> structural damage
decrementEventTimers(game) (events.c) construction suspension / closed property
decrementModifiers(game) (economy.c) all timed effects tick down and are removed
enforceAntiSpeculation(game) (finance.c) force-sell undeveloped surplus if triggered
```

```
every 10 rounds: triggerDisaster + applyInflation + reviewPropertyMarket
every 15 rounds: triggerEconomicEvent + drawRegionalCard
every 20 rounds: triggerGovernmentRegulation
```

```
displayRoundSummary(game) + displayMarketConditions(game)
```

5.5 How the modules talk to each other

```
+-----+
v               |
```

```

main.c -> startGame -> playGame -> playTurn +-> playTurn(next player)...
      |
      landing / decisions call down into:
finance.c payRent->calculateRent -> modifierMultiplier(economy.c)
      buyProperty->shouldBuyProperty(players.c) / runAuction(auction.c)
economy.c currentMarketValue <- used by finance, bank, insurance, auction
players.c every should*/wants* decision funnels through strategy switch
bank.c    obtainLoan -> totalEligibleCollateral -> modifierMultiplier
events.c  executeEvent -> addModifier(economy.c) / triggerDisaster(insurance.c)
market.c  reviewPropertyMarket -> addSourcedModifier(economy.c)
insurance.c triggerDisaster -> payCompensation / mark damaged
      |
every function reads/writes the SAME GameState passed as game[]

```

මොනොපොලි ක්‍රීඩාවේ ජීවන චක්‍රය: අවුරුද්දේ පුරවුවේ ගමන් -> කුලී/මිලදී ගැනීම -> ගොඩනැගිල්ල - සෑම වටයකම මෙම රටීයයි.

6 The Data Model (types.h)

types.h defines the whole vocabulary of the game. It has four kinds of things: `#define` constants (numbers), `typedef` enums (named lists), `typedef` structs (shapes of data), and the GameState bundle that joins everything. Because every source file `#includes` types.h, the numbers all have one single source of truth.

6.1 Game constants

Constant	Value	Meaning
MAX_PLAYERS	4	exactly four AI players
BOARD_SIZE	40	squares on the board
MAX_ROUNDS	500	hard cap; game ends here
START_MONEY	30000	each player's starting cash (LKR)
GO_MONEY	2000	collected when passing GO
JAIL_BAIL	300	bail fee
JAIL_SQUARE / JAIL_MAX_TURNS	10 / 3	jail location / forced-bail after 3 turns
INCOME_TAX_RATE	15	base income tax % (drifts with market)
COMMUNITY_FUND_TAX_RATE	10	base community fund tax %
MAX_HOUSES / MAX_HOTELS	4 / 1	buildings allowed per property
LOAN_DURATION_ROUNDS	20	borrowed money must be repaid in ... rounds
LOAN_INTEREST_RATE	8	starting interest %
LOAN_COLLATERAL_PERCENT	75	max loan = 75% of collateral (Rule-LK 2)
LOAN_EXTEND_ROUNDS	10	extending a loan adds 10 rounds
MORTGAGE_REPAY_PERCENT	110	redeeming a mortgage costs 110%
LOAN_INTEREST_MIN/MAX/STEP	1/25/2	interest limits + step size
TAX_RATE_MAX	25	income tax never exceeds 25%
RAILWAY_RENT_1..4	250/500/1000/2000	rent by number of railways owned
UTILITY_RENT_ONE/TWO	4x / 10x	utility rent = dice multiplier
AUCTION_OPENING_DIVISOR / BID_INCREMENT	2 / 250	auction start = value/2, bid +250
MARKET_REVIEW_EVERY / COOLDOWN	10 / 30	market boom/decline cadence
PROPERTY_AGE_LIMIT / DEPRECIATION_STEP / MAX	50 / 5 / 30	ageing rules
RENOVATION_COST_PERCENT / RENT_BOOST	10 / 5	renovation cost / +5% rent
NEGLECT_ROUNDS	20	no maintenance this long = structural
STRUCTURAL_VALUE/RENT_LOSS, REPAIR_MULT	15 / 25 / 150	structural damage effects
REPAIR_COST_PERCENT	30	disaster repair = 30% of value
PREMIUM_BASIC/COMPREHENSIVE/BI	5/10/15	insurance premium % of value
INSURANCE_DURATION / RENEW_AT / WARNING	20 / 10 / 3	policy lifecycle
DISASTER_WEIGHT / BOOST	20 / 30	weighted disaster picker
ANTI_SPEC max undeveloped / trigger	3 / 5	Anti-Speculation Act rules
EVENT/ECON/REG/REGIONAL card counts	20/8/8/12	deck sizes
ECONOMY/EVENT/REGULATION CYCLE	10/15/20	rounds between periodic events
MIN_CASH_SAFETY	1000	common strategy cash reserve

6.2 Enumerations (named lists)

Enum	Values	Role
------	--------	------

SquareType	GO, PROPERTY, RAILWAY, UTILITY, TAX, COMMUNITY_FUND, EVENT, BANK, INSURANCE, JAIL, FREE_PARKING, GO_TO_JAIL	what each of the 40 squares does
PropertyGroup	BROWN ... DARK_BLUE, NO_GROUP	the 8 colour groups (+sentinel)
PlayerType	AGGRESSIVE_INVESTOR, CONSERVATIVE_BANKER, RISK_TAKER, OPPORTUNISTIC_TRADER	the 4 AI personalities
InsuranceType	NO, BASIC, COMPREHENSIVE, BUSINESS_INTERRUPTION	policy types
DisasterType	FIRE, FLOOD, RIOT, BUILDING_COLLAPSE, ELECTRICAL_FAILURE	disasters
ModifierType	MOD_VALUE_GLOBAL ... 18 kinds	every kind of temporary effect
ModifierSource	SRC_GENERAL, SRC_BOOM, SRC_DECLINE, SRC_REGIONAL	where an effect came from (for the report)

6.3 Structs - the shape of the data

The structs are nested: GameState contains Economy, Player[4], Square[40]; each Square contains a Property; each Player contains a Loan.

```

GameState
+-- Square board[40]           the 40 squares
+-- Player players[4]         the 4 AI players
`-- Economy economy           interests, tax, modifiers, event timers

Square
+-- int    index              0..39
+-- SquareType type          what kind of square
+-- char   name[50]
+-- Property property        the buyable data (used for ALL squares)

Property (one per square)
+-- name / group / purchasePrice / baseRent
+-- mortgageValue / houseCost / hotelCost
+-- owner (-1 = nobody) houses hotel
+-- mortgaged loanLocked (pledged as collateral)
+-- insurance (type) insuranceRoundsLeft
+-- damaged repairCostOwed lostIncomeRoundsLeft
+-- age depreciation          (value ageing)
+-- condition roundsSinceMaintenance structurallyDamaged
+-- preDamagePurchasePrice preDamageBaseRent maintenanceCostMultiplierPercent

Player
+-- name strategy position cash
+-- inJail jailTurns bankrupt
+-- propertiesOwned railwaysOwned utilitiesOwned
+-- Loan loan (active / amount / interestRate / remainingRounds)
+-- sufferedLoss (Risk Taker insures only after a loss)
+-- antiSpecRounds (consecutive rounds above 3 undeveloped props)

Economy
+-- inflationRate loanInterestRate incomeTaxRate
+-- constructionSuspendedRoundsLeft closedPropertyIndex/RoundsLeft
+-- antiSpeculationActive currentCardIndex
+-- groupCooldownUntilRound[8] lastBoomGroup lastDeclineGroup
+-- ActiveModifier modifiers[48] + modifierCount

```

Almost every invariant of the game is stored somewhere in these fields: owner == -1 means unowned, loanLocked means 'cannot be sold or mortgaged', damaged means 'earns no rent', condition drives how much rent a developed property collects, and depreciation drives the real market value.

[Note] MAX_MODIFIERS = 48 simply caps the number of simultaneous temporary effects; addModifier silently ignores new ones if the list is full (a safety guard).

සෑම දෑයක් එක struct එකක තුළ ගබඩා: GameState = පුවරුව + ක්‍රීඩකයෝ + ආර්ථික නිකුරු.

7 Module Deep Dives

This is the main body of the guide. Each subsection covers one source file: what it is for, every function it defines, and the important logic explained step by step.

සෑම file එක: අරමුණ, function ලැයිස්තුව, සහ හදලා ඇති තර්කය.

7.1 main.c - the entry point

Absurdly simple on purpose. main() creates the one and only GameState object in the whole program, prints a banner, and hands control to startGame(). Nothing else happens here; all real setup lives in game.c.

main.c (entire file, 17 lines)

```
int main(void)
{
    /* the only place a GameState is ever created */
    GameState game[1] = {0};

    printf("====...\n");
    printf("      MONOPOLY-LK SIMULATION\n");
    printf("====...\n\n");

    startGame(game);
    return 0;
}
```

Why {0}? It zero-initialises every field. Without it, uninitialised data would contain garbage, and fields the board builders forget to set (e.g. owner) would look wrong. You will see board.c explicitly sets owner = -1 everywhere - that fix is the reason special squares are not 'owned' by player 0.

7.2 board.c - building the 40-square board

board.c turns a flat array of Square structs into the complete game board. There are four 'setter' functions that fill in common values, and one big initialiser that calls them 40 times.

Function	Purpose
setProperty(...)	fills a colour-group property: price, base rent, mortgage, house/hotel costs, and resets all ~20 runtime fields (owner=-1, condition=100, houses=0, ...)
setRailway(...)	a railway: price + mortgage = price/2; everything else zeroed (no buildings, no insurance)
setUtility(...)	the two utilities: same shape as a railway
setSpecialSquare(...)	non-buyable squares (GO, Jail, Tax, ...); critically sets owner = -1 so they never look owned
initializeBoard(...)	calls the setters 40 times with the Sri Lanka street data

The board layout (index, name, group, price/rent). This is the standard Monopoly ring layout, indexed 0..39 going clockwise from GO.

0	GO	20	Free Parking
1	Pettah (Brown 1500/100)	21	Kandy City (Red 5500/450)
2	Community Fund (tax 10%)	22	National Event Card
3	Maradana (Brown 1800/120)	23	Peradeniya (Red 5800/480)
4	Income Tax (tax 15%)	24	Katugastota (Red 6000/500)
5	Colombo Fort Railway	25	Galle Railway Station
6	Bambalapitiya (LB 2500/180)	26	Galle Fort (Yellow 6500/600)
7	National Event Card	27	Unawatuna (Yellow 6800/620)
8	Wellawatte (LB 2700/200)	28	Water Supply Board (utility)
9	Mount Lavinia (LB 3000/220)	29	Hikkaduwa (Yellow 7000/650)
10	Jail / Just Visiting	30	Go To Jail
11	Nugegoda (Pink 3500/260)	31	Jaffna Town (Green 8000/750)
12	Electricity Board (util)	32	Nallur (Green 8300/780)
13	Maharagama (Pink 3800/280)	33	Ceylinco Insurance
14	Kottawa (Pink 4000/300)	34	Trincomalee (Green 8500/800)
15	Kandy Railway Station	35	Jaffna Railway Station
16	Negombo (Orange 4500/350)	36	National Event Card
17	Sri Lanka Insurance	37	Nuwara Eliya (DkBlue 10000/1000)
18	Katunayake (Orange 4700/370)	38	Bank of Ceylon
19	Ja-Ela (Orange 5000/400)	39	Galle Face (DkBlue 12000/1200)

That makes 22 colour properties, 4 railways, 2 utilities, and 12 special squares (GO, Community Fund, Income Tax, 3 events, Jail, 2 insurance, Free Parking, Go To Jail, Bank) = 40 in total.

පුළුල්ව තනනන් 40 කි; තාන පළෙඹේ ලකුණ (index) මගින් ගමන් ගණන් කරයි.

7.3 players.c - the 4 AI strategies

This file is pure decision-making. It contains no money movement - it only answers questions like 'should I buy this?', 'do I want a loan?', 'how much will I bid?'. Every decision function switches on the player's PlayerType, so changing a strategy just means editing one case.

Function	Question it answers
initializePlayers	sets names, strategies, position 0, 30,000 cash, zero counters
shouldBuyProperty	should I buy the square I just landed on?
shouldConstruct	should I spend on this house/hotel?
wantsLoan	do I want to borrow now?
wantsToRepayLoan	do I want to repay the loan?
wantsIncreaseLoan	borrow more against new collateral?
wantsExtendLoan	push the deadline back?
wantsRefinance	move to the current (lower) interest rate?
desiredInsurance	which policy (or none) do I want?
shouldRenovateAgeDepreciation	pay to fix age depreciation?
shouldMaintain	pay maintenance on a worn building?
willingToBid	bid this amount at auction?
shouldMortgage	mortgage a property to raise cash?
shouldRedeemMortgage	buy a mortgaged property back?
shouldPayBail	pay the 300-lkr bail instead of waiting?

The strategy decision matrix (memory aid)

Decision	Aggressive Investor	Conservative Banker	Risk Taker	Opportunistic Trader
Buy property	always if it completes a monopoly; else keep ≥ 1000	no props during recession; keep cash $\geq 2x$ price	any cash $\geq$ price	never into a declining group; price $\leq 40\%$ cash
Build (house/hotel)	cash - cost ≥ 1000	no hotels while a loan is active; keep $\geq 50\%$ cash	cash $\geq$ cost (any)	subsidy $\rightarrow$ keep 30%; inflation $\rightarrow$ keep 60%; else 40%
Want a loan	always	only near bankruptcy (cash $<$	always	cash < 5000
Repay loan	cash $\geq 2x$ loan	cash $\geq$ loan	rarely (cash $\geq 5x$ loan)	cash $\geq 1.5x$ loan
Extend loan	never (prefers borrowing more)	≤ 5 rounds left and broke	≤ 10 rounds left	≤ 5 rounds left
Refinance	shared rule for all: only when current economy rate is lower			
Insurance	hotel $\rightarrow$ Comprehensive else Basic	always Comprehensive	none until a loss; then BI if hotel	Comprehensive only if price $\geq$ 6000
Bid at auction	up to 120% of market value	below 100% of market value	always (if funded)	up to 80% of market value
Mortgage	cash < 1000	cash < 3000	cash < 500	cash < 1200
Pay bail	cash ≥ 5000	cash ≥ 1500	never	cash ≥ 5000
Maintain building	keep 80% cash after cost	keep 80% cash after cost	only when condition < 25	keep 80% cash after cost
Renovate age depreciation	when depreciation $\geq 20\%$	$\geq 10\%$	$\geq 20\%$	$\geq 15\%$

[Note] The 'cash safety' numbers used here (1000 / 3000 / 500 / 5000 ...) are all named constants in types.h (MIN_CASH_SAFETY, COMFORTABLE_CASH, ...).

7.4 game.c - the game engine

game.c is the conductor. It performs the start-of-game setup, owns the turn/round loops, and decides who wins.

Function	Purpose
rollDice	two d6 summed -> 2..12 (movement and utility rent use the sum; jail uses the individual)
movePlayer	advances position by dice, wraps at 40; returns 1 (and pays LKR 2000) if GO was passed
handleJail	implements Rule 13: doubles release, bail release, or forced bail after 3 turns; returns 1 = turn ends without moving
playTurn	one whole turn: repair -> jail -> roll+move -> landing dispatch -> build -> sell -> mortgage
resolveGroup / determineTurnOrder	roll each player's dice; sort highest first; any tie re-rolls recursively until a strict order exists
countHouses / countHotels	totals a player's houses/hotels for the summary
displayRoundSummary	per-player cash, net worth, properties, railways, utilities, houses, hotels, loan, status
countSolventPlayers	how many players are not bankrupt
playGame	the main loop - cycles turns until every solvent player has passed GO, then does round-end bookkeeping; caps at 500 rounds
determineWinner	solvent player with the highest net worth; edge case: if everyone is bankrupt, still pick the highest net worth
displayFinalResults	winner block + final standings for all players
startGame	seed random - setup - determine turn order - playGame - results

The recursive turn-order resolution is a nice talking point in a viva - it replaces a 'while loop until no ties' with clean recursion:

```

resolveGroup(groupPlayers, n, output, startIndex)
  roll for every player in group
  sort the group by roll, highest first
  walk the sorted list in runs:
    run of 1 -> place that player directly in output
    run of 2+ -> call resolveGroup on JUST those tied players
  return how many players were placed

```

7.5 finance.c - money, rent, buying, buildings, taxes, bankruptcy

The biggest file (1076 lines) and the economic core. Almost every rule about moving money lives here.

Group	Functions	What it does
Ownership helpers	stripOwnership, resetLoan, adjustOwnedCount, groupOf	keeps owner -1 / counters / loan fields consistent
Money	receiveMoney, payMoney	receive no-ops for bankrupts; payMoney triggers Risk Taker auto-sales and then bankruptcy+liquidation
Taxes	payTax, payCommunityFundTax	Income Tax = rate x NET WORTH (clamped >= 0); Community Fund = rate x property value
Mortgage	findPropertyToMortgage, findMortgagedProperty, mortgageProperty, redeemMortgage,	mortgage gives mortgageValue x market modifier; redeem costs 110%
Rent	calculateRent, calculateRailwayRent,	rent tables x modifiers x condition %
Groups/buildings	groupSize, ownsMonopoly, developGroup, constructBuildings	even development (Rule 9), max 4 houses -> hotel
Value/net worth	calculatePropertyValue, calculateBuildingValue, calculateNetWorth	net worth = cash + property + buildings - loan (Rule 15)
Anti-speculation	countUndevelopedProperties, sellUndevelopedPropertyToAuction,	force-auction surplus undeveloped property
Strategy sales	sellLowValueProperty, sellDecliningProperties	Risk Taker survival sale; Trader dumps declining assets
Buying	buyProperty, wouldCompleteMonopoly	market price, strategy decides, else auction (Rule 5)
Rent payment	payRent	checks who owns, skips if mortgaged/closed/damaged/BI, transfers rent

How rent is calculated (calculateRent)

```
if hotel      -> rent = baseRent * 10
else by houses: 0 -> *1, 1 -> *2, 2 -> *3, 3 -> *5, 4 -> *7

rent = rent * group-rent-modifier / 100      (MOD_GROUP_RENT)
rent = rent * global-rent-modifier / 100     (MOD_RENT_GLOBAL)
if hotel: rent = rent * hotel-rent-mod / 100 (MOD_HOTEL_RENT)
if developed: rent = rent * conditionPercent / 100 (Table 3)
```

Condition percent (rentConditionPercent in economy.c): condition >= 90 -> 100%, >= 75 -> 90%, >= 50 -> 75%, >= 25 -> 50%, below 25 -> 0% (closed).

Railway rent is 250/500/1000/2000 depending on how many railways the same owner holds (x2 rent modifiers). Utility rent is 4x the dice if one utility is owned, 10x the dice if both are owned (x2 utility/global rent modifiers).

Buying -> if the buyer refuses, an auction runs (Rule 5)

```
buyProperty(game, playerIndex)
  price = currentMarketValue(pos)
  if PROPERTY: price *= purchase-price-modifier (boom +15%)
  wantsToBuy = shouldBuyProperty(...)
  if cash < price: wantsToBuy = 0
  if !wantsToBuy -> runAuction(pos, seller = -1) // Rule 5
  else -> payMoney, set owner, increment owned counter
```

payMoney and bankruptcy (the safety wrapper)

This is one function every payment goes through, so bankruptcy handling is centralised. If the Risk Taker goes below zero, they first try to sell their cheapest undeveloped property (Section 3.4). If cash is still negative, the player is declared bankrupt, all assets are liquidated and auctioned, and cash is clamped to zero.

```
while (cash < 0 && !bankrupt && strategy == RISK_TAKER):
  before = countUndevelopedProperties(...)
  sellLowValueProperty(...) // cheapest undeveloped prop
  if countUndevelopedProperties() >= before: break // nowhere left to sell

if cash < 0 and not bankrupt:
  bankrupt = 1
  liquidateBankruptAssets(player) // demolish, unown, auction each square
  cash = 0 // remaining debt is written off
```

Even development (Rule 9) - developGroup

A group can only be built on if the player owns the monopoly. developGroup always builds on the property with the fewest houses, so development stays even. A member that is mortgaged, damaged or loan-locked cannot be built on, and (bug fix) a blocked member with fewer than 4 houses also blocks the whole group from raising a hotel.

```
developGroup(game, playerIndex, group)
  if !ownsMonopoly -> return 0
  pick target = owned property with the fewest houses (minHouses)
  skip mortgaged / damaged / loan-locked members
  a skipped member with < 4 houses means: no hotel for the group
  never build a 5th house on a target already at 4 houses
  if all four houses everywhere -> upgrade target to a HOTEL (houses = 0)
  else -> add one HOUSE to the target
```

constructBuildings() repeats this in a do/while loop (max 20 safety steps) so a player can build SEVERAL houses in one turn - the loop stops when nothing more can be built (Strategy says no, cash runs out, or fully built).

Net worth (Rule 15)

```
netWorth = cash
  + currentMarketValue of every owned property (railway/util included)
  + (houses * houseCost) or hotelCost for hotels
  - outstanding loan amount
```

note: mortgaged properties contribute ZERO to net worth

කුලිය (rent) ගණන් කරන්නේ පළමුවේ එකින් එක: base x houses ගුණකාරය x modifiers x තත්ත්වය%.

7.6 bank.c - loans, collateral and foreclosure

The Bank of Ceylon square (index 38) offers loans. The loan system is a mini-economy of its own: you pledge owned property as collateral, the bank lends you 75% of its value, interest is added to the balance every round, and if you

default the collateral is seized.

Function	Purpose
groupBasePrice	the basis value of a colour group (the first property's price) used ONLY for loan math
totalEligibleCollateral	sum of group base prices (or rail/util mortgage values) that are owned, unmortgaged, unpledged, x market modifier
calculateMaxLoan	75% of eligible collateral (Rule-LK 2)
obtainLoan	one loan at a time; locks every eligible property as collateral; pays out; 20 rounds at the current interest
repayLoan	partial or full repayment; on full repayment all collateral is released
handleBankVisit	one transaction per Bank landing: repay full / increase / extend / refinance, or obtain a new loan
increaseLoan	borrow 75% of newly-acquired collateral and add it to the loan
extendLoan	+10 rounds to the repayment deadline
refinanceLoan	switch the loan to the economy's current (lower) rate
demolishBuildingsOn	zero out houses/hotel (used on seizure and bankruptcy)
foreclose	default: pledged properties are demolished, unowned and auctioned; player bankrupt if nothing
processLoans	every round: interest = amount x rate% added to principal; count down; at 0 -> foreclose

The interest that is added each round is compounding interest: it is added to the principal, so next round's interest is higher. That makes long unpaid loans grow very fast.

```
processLoans(game)
  for each player with an active loan:
    interest = loan.amount * loan.interestRate / 100
    loan.amount += interest // COMPOUNDED
    loan.remainingRounds--
    if remainingRounds <= 0: foreclose(player)
```

ණයක් = ඇප (collateral) මත පදනම් වූ lenders 75%; ප්‍රොසිය සෑම round එකින් එක මුදලට එකතු වේ (එක්රවන්නී ප්‍රොසිය).

7.7 auction.c - the auction engine

When nobody buys a square directly, or a forced sale happens, runAuction sells it to the highest bidder among all solvent players.

Function	Purpose
getAskingValue	current market value x auction-price modifier (declining groups start -25%)
runAuction	opening = value/2; bids must rise by 250; a player who passes is out forever; seller never bids; winner pays; seller (if any) receives the money

```
runAuction(propIndex, sellerIndex)
  openingBid = askingValue / 2
  active = all solvent players except the forced seller (seller excluded - bug fix)
  while activeCount > 1 (max 200 safety rounds):
    each active player:
      candidate = currentBid + 250
      willingToBid()? -> bids, become highBidder
      else -> out of the auction for good
  special cases: single remaining bidder gets one final chance;
  if no one bid, the property stays with the Bank
  winner pays currentBid; sellerIndex >= 0 receives it; ownership transferred
```

[Note] Why exclude the seller? Without that bug fix, a forced seller could bid on their own property, win it, and 'pay' themselves - a net-zero transfer that defeats the Anti-Speculation Act. Paying the bid to yourself is exactly neutral, so the forced sale would never actually change ownership.

7.8 events.c - cards, economic events and regulations

Three kinds of 'shocks' to the economy come from this file. Almost every effect is implemented by adding a timed modifier to the economy - it never permanently changes a base price.

Function	Purpose
clampLoanInterest / adjustLoanInterest	keep the economy interest rate in [1, 25] and move it by a step
executeEvent	20 national event cards, drawn from a cycling deck (0..19); effects include rent boosts, disasters, interest changes, tax amnesty, grants...
triggerEconomicEvent	1 of 8 random economy-wide events every 15 rounds
triggerGovernmentRegulation	1 of 8 random regulations every 20 rounds (tax rise, luxury tax, subsidies, Anti-Speculation Act...)
decrementEventTimers	count down non-modifier timers (construction suspension, closed property)

The 20 national cards (Appendix A of the assignment) map to cases 0-19:

0 Tourism Hype	hotel rent x2	(5 rounds)
1 Fuel Shortage	railway rent x2	(5 rounds)
2 Heavy Floods	random property damaged (disaster)	
3 Political Rally	one owned property closed	for 2 rounds
4 Stock Market Rise	all values +10%	(15 rounds)
5 Economic Downturn	all values -15%	(15 rounds)
6 Housing Subsidy	construction -30%	(15 rounds)
7/8 Interest Cut/Rise	interest +/- 2 points	
9 Tax Amnesty	everyone +2000	
10 Power Failure	utility rent /2	(3 rounds)
11 Foreign Funding	Orange group +15%	(15 rounds)
12 Port Expansion	railway values +20%	(15 rounds)
13 Festival Season	hotel rent +50%	(5 rounds)
14 Labour Strike	construction suspended	(2 rounds)
15 Insurance Discount	premiums -20%	(10 rounds)
16 Property Revaluation	random group +15%	(15 rounds)
17 Currency Deprecia.	construction +10%	(15 rounds)
18 Government Grant	5000 to a random SOLVENT player (bug fix: not bankrupt)	
19 National Disaster	random developed property damaged (disaster)	

The deck is a simple counter: currentCardIndex rises 0 -> 19 then wraps to 0. That is equivalent to 'draw the top card, put it at the bottom of the deck' without needing a real deck data structure.

The 8 economic events and 8 government regulations are chosen randomly with rand() % 8. Regulation case 7 activates the Anti-Speculation Act, and regulation case 3 charges a 25% luxury tax on every hotel (printed per player/property - bug fix).

7.9 economy.c - inflation, modifiers, ageing, maintenance

The modifier system is the cleverest part of the project. Instead of permanently changing prices when events happen, every effect is stored as an ActiveModifier with a fixed % and a countdown. All matching modifiers multiply together, so two +25% bonuses are not +50% but roughly +56% (that is Rule-LK 34 - they are cumulative).

Function	Purpose
initEconomy	zero modifiers, base interest/tax, card index 0
applyRate	Old x (1 + rate/100), never below 1
addModifier / addSourcedModifier	append a timed effect (source = general/boom/decline/regional)
modifierMultiplier	multiply every matching modifier's % together; 100 = no effect
isModifierActive	is at least one matching modifier running? (flag effects)
decrementModifiers	tick each countdown down; remove (compact the array) expired ones
formatLKR	30000 -> "30,000" (thousands separators)
applyInflation	every 10 rounds a random rate from {-3,0,2,5,8,12}% is permanently applied to price/rent/mortgage/house/hotel costs of every buyable square
currentMarketValue	the single truth for 'real price': purchasePrice x (1 - depreciation) x all value modifiers
currentTaxRatePercent	base tax rate x global value modifier
ageProperties	age +1 per round; past 50, +5% depreciation per 5 rounds, capped at 30%
tryRenovateAgeDepreciation	landing on your own aged property: pay 10% -> age/depreciation reset, baseRent +5%
rentConditionPercent	Table 3 mapping condition -> rent %

ageBuildings	condition -2 per round; 20+ rounds of neglect -> structural damage (-15% value, -25% rent, x150% repair)
performMaintenance	restore condition to 100: hotel -> 8% of hotel cost, house -> 5% of house cost
renovateStructuralDamage	pay 150% of replacement value to undo structural damage and restore pre-damage price/rent

```

currentMarketValue(pos)
    price = purchasePrice
    price -= price * depreciation / 100          // age depreciation
    mult = MOD_VALUE_GLOBAL (all values)
    mult *= MOD_GROUP_VALUE (this colour group) if property
    mult *= MOD_RAIL_VALUE if railway
    mult *= MOD_INDEX_VALUE (this exact square)
    return price * mult / 100

```

Because buyPrice, insurance premiums/repairs, renovation, net worth and auction values ALL call currentMarketValue, every part of the game always sees the same 'real price' - no numbers can disagree.

මෙය stage එකයි: සිදුවීමේ මූලික මිල වනෙස් නොකරයි - දියුණු / පහත කිරීම modifiers වලින් කල් පමණක්.

7.10 insurance.c - policies, disasters, repairs

Every 10 rounds (or via event cards 2/19) one random developed property may be hit by a disaster. Insurance softens the blow.

Function	Purpose
propertyValue / repairCost	value = currentMarketValue; repair cost = 30% of value
isCovered	Basic covers only Fire + Flood; Comprehensive and Business Interruption cover everything
compensationPercent	Basic 80%, Comprehensive/BI 100%
pickDisaster	weighted random: 5 types at weight 20 each; Heavy Monsoon boosts flood by +30, Political Unrest boosts riot by +30
purchaseInsurance	premium = value x (5/10/15)% x insurance modifier; 20 rounds
findPropertyToInsure / findPropertyToRenew	which property to buy a first policy / renew near expiry (<= 10 rounds)
handleInsuranceVisit	renew first, otherwise buy a new policy per the strategy
tryAutoRepair	start of turn: if the owner can afford repairCostOwed, repair it (prints the cost - bug fix) and rent resumes
payCompensation	pays the owner; Business Interruption also (a) gets 5 rounds of lost income, (b) claims are x150% during political unrest
triggerDisaster	pick a random developed property, pick a disaster, compute repair cost; insured -> compensation; uninsured -> owner marked sufferedLoss; remainder owed = damaged and earns no rent
processInsuranceExpiry	count down policies; warn at 3 rounds; expire at 0 -> no insurance

```

triggerDisaster(game)
    collect all developed properties (houses > 0 or hotel)
    if none exist -> nothing happens
    chosen = a random developed property
    disaster = pickDisaster (weighted by flood/riot risks)
    cost = 30% of current market value
    if covered:
        compensation = cost x 80%/100%
        BI policies also set lostIncomeRoundsLeft = 5
        repairCostOwed = cost - compensation (usually 0 for full cover)
    else:
        owner.sufferedLoss = 1 (Risk Taker now wants insurance!)
        repairCostOwed = cost
    if repairCostOwed > 0: damaged = 1 // no rent until repaired

```

7.11 market.c - market review, regional cards, report

The property market behaves like a real economy: colour groups boom and decline on a cycle, and regional development cards give location-specific bonuses.

Function	Purpose
initMarket	clear group cooldowns and last boom/decline groups
groupName	enum -> text (Brown, Light Blue, ...)
pickEligibleGroup	random group not on cooldown and not one of the excluded ones
reviewPropertyMarket	every 10 rounds: boom group (+20% value, +25% rent, +15% mortgage/purchase, +10% construction) and decline group (-15% value, -20% rent, -10% mortgage, auction -25%); each for 10 rounds; 30-round cooldown
drawRegionalCard	1 of 12 Sri Lanka themed regional cards, 15 rounds each
displayMarketConditions	Rule-LK 36 report: boom/decline/regional/other active effects + inflation + interest + tax rate

All these effects are added as sourced modifiers (SRC_BOOM / SRC_DECLINE / SRC_REGIONAL), which lets the end-of-round report group them into the correct sections.

7.12 functions.h - the project's 'API'

functions.h is a header file that only declares functions - it contains no logic. It has two protective lines (`#ifndef ... #define ... #endif`) so it can be included many times without error. It is organised by module with comment banners (board.c, players.c, finance.c, ...), which makes it a great map of the whole project: read it and you have seen every function that exists.

In a viva you can use functions.h as your index: point at the section comment and explain what that module is responsible for.

8 Game Rules - Quick Reference

8.1 Money

- Everyone starts with LKR 30,000.
- Passing (or landing on) GO gives LKR 2,000.
- All amounts print in LKR with thousands separators (formatLKR).

8.2 Board

- 40 squares, indices 0..39 (board.c). Rent/price values are set once and only permanently change via inflation.
- 22 colour properties in 8 groups, 4 railways, 2 utilities, 12 special squares.

8.3 Turn order

- Decided by recursive dice roll-off - ties re-roll until a strict order exists.

8.4 Rent (Table 6)

- 0 houses: x1, 1: x2, 2: x3, 3: x5, 4: x7, hotel: x10 of base rent.
- Railways: 250/500/1000/2000 depending on how many the owner holds (Table 7).
- Utilities: 4x dice (one owned) or 10x dice (both owned) (Table 8).
- Every rent is then scaled by active rent modifiers and, for developed properties, by condition % (Table 3).
- No rent if the square is mortgaged, politically closed, disaster-damaged, or in BI lost-income.

8.5 Buying

- Price = currentMarketValue (x purchase-price modifier for colour groups).
- If the landing player refuses (or cannot pay), an auction runs (Rule 5).

8.6 Building (Rule 9)

- Only with a full monopoly of the colour group.
- Build on the property with the fewest houses (even development).
- Max 4 houses per property; the 4th house becomes a hotel.
- Blocked member (mortgaged/damaged/loaned) with < 4 houses blocks hotels for the group.
- Construction can be suspended by events or made cheaper/expensive by modifiers.

8.7 Mortgage

- Only undeveloped, un-pledged properties can be mortgaged.
- Payout = mortgageValue x market modifier; redemption costs 110%.
- Mortgaged properties earn no rent and count zero in net worth.

8.8 Bank loans (Rule-LK 2-7)

• One loan at a time; 75% of eligible collateral; interest per round added to the balance (compounding); 20-round term.

- Collateral = colour groups by their base price, rail/utills by mortgage value.
- Extend +10 rounds, refinance to current rate, increase against new collateral.
- Default -> foreclosure: pledged property seized and auctioned.

8.9 Jail (Rule 13)

- Go To Jail teleports to square 10 and marks the player.
- Doubles releases (no move that turn); bail 300; forced bail after 3 turns.

8.10 Tax

- Income Tax = market-scaled rate (base 15%, cap 25%) x net worth, clamped ≥ 0 .
- Community Fund = 10% x property value.

- Luxury Property Tax (regulation) = 25% of each hotel's value, printed per owner.

8.11 Insurance

- Basic 5% (fire/flood, 80%), Comprehensive 10% (all, 100%), Business Interruption 15% (all, 100%, + protects 5 rounds of income).
- Premium = current value x % x insurance modifiers; 20 rounds.
- Heavy Monsoon boosts floods; Political Unrest boosts riots and BI claims.

8.12 Disasters

- Every 10 rounds one random developed property is hit.
- Repair cost = 30% of value. Uninsured owners carry repairCostOwed and earn nothing until repaired.
- Insured owners receive compensation; loss sets the Risk Taker's sufferedLoss flag.

8.13 Ageing & maintenance

- Properties age +1 per round; past 50, +5% depreciation per 5 rounds (max 30%).
- Renovation (10% of value) on your own square resets age and boosts base rent +5%.
- Developed buildings lose 2 condition per round; maintenance returns it to 100.
- 20+ rounds of neglect = structural damage (value -15%, rent -25%, repair x150%).

8.14 Market review (Rule-LK 30-34)

- Every 10 rounds: one group booms, a different one declines, for 10 rounds; 30-round cooldown.
- All market effects are temporary modifiers - never permanent price changes.

8.15 Cards & events

- 20 national cards (cycling deck) when landing on EVENT squares.
- 1 of 8 economic events every 15 rounds; 1 of 12 regional cards every 15 rounds;
- 1 of 8 government regulations every 20 rounds.

8.16 Anti-Speculation Act (Rule-LK 8)

- Max 3 undeveloped properties. After 5 consecutive rounds over the limit, the excess is force-auctioned and the seller cannot bid.

8.17 Bankruptcy

- Risk Taker first tries to sell its cheapest undeveloped property.
- Otherwise: buildings demolished, every property auctioned, cash zeroed, player removed.
- Bankrupt players take no further part (receiveMoney no-ops).

8.18 Winning

- Last solvent player wins; if several remain after 500 rounds, highest net worth wins.

9 Key Algorithms Explained

9.1 Even development (Rule 9)

Goal: never pile houses on one property while a group-mate stays bare. `developGroup` scans the group, skips blocked members, and picks the property with the fewest houses. It returns 1 when something was built so that `constructBuildings` can immediately call it again - which is how a player raises several houses in a single turn.

9.2 The modifier multiplication (Rule-LK 34)

Every temporary effect is a percent stored in `Economy.modifiers`. `modifierMultiplier` multiplies all matching percents together. A group whose value had +20% (boom) and +15% (event) is worth $x1.20 \times 1.15 = x1.38$, not $x1.35$ - that is 'cumulative', the rule's intention. 100 means 'no change', so the maths is always $\%/100$.

9.3 Compounding loan interest

Each round the interest is computed from the CURRENT balance and added to it. The balance therefore grows exponentially. Refinancing, repaying and extending are all tools to fight that growth.

9.4 The real definition of a round

A round ends when every solvent player has passed GO at least once (tracked in `passedGoThisRound[]`). This is why `playGame` cycles turns and checks `allPassed` after every single turn, not after every 4 turns.

9.5 Weighted disaster selection

5 disasters each start at weight 20. Extra flood/riot risk (from the Heavy Monsoon or Political Unrest events) adds +30 to that type. `pickDisaster` builds a total, rolls `rand()` % total, and walks the weights - a standard weighted-random draw.

9.6 Recursive turn-order resolution

`resolveGroup` bubblesorts a group by dice roll and walks the sorted list. Runs of equal rolls are handed back to `resolveGroup` recursively. Each recursion reduces the tied group, so the recursion always terminates and produces a strict, fair order.

9.7 Inflation model

Every 10 rounds one of {-3, 0, 2, 5, 8, 12}% is chosen. `applyRate` permanently changes `purchasePrice`, `mortgageValue`, `baseRent`, `houseCost` and `hotelCost` of every square (plus the loan interest rate). This is the ONLY place base values can permanently change - everything else is a temporary modifier on top.

10 Verification and Testing

The project is verified by generate-a-log-then-replay-it. The game prints every transaction to stdout; that log is saved; verify_analysis.py parses it line by line and checks it against a Python mirror of the rules.

10.1 How verify_analysis.py works

- It mirrors the board constants (prices, rents, groups, players, inflation set) at the top of the file.
- An Analyzer object keeps live state: for every square - owner, mortgaged, houses/hotel, insurance, price, age, damaged - and for every player - position, cash, bankrupt, loan.
- Regex handlers match each log line type (rounds, dice+moves, GO payments, purchases, rent, construction, mortgages, loans, bankruptcy, auctions, insurance, taxes, disasters, market reviews, game-over arithmetic, final standings).
- It asserts the same invariants a human grader would check: dice in [2,12], (from+dice)%40 == to, rent payer != owner, no double loans, auction bidder pays, net worth = cash + property - loans, bankrupt players have 0 cash.
- Each mismatch is printed as [BUG] (invariant broken) or [REVIEW] (needs human judgement).

```
python verify_analysis.py run.txt      # -> 0 confirmed bugs
python verify_analysis.py run2.txt     # -> 0 confirmed bugs
python verify_analysis.py test.txt     # <- the pre-fix log: shows the bugs
```

10.2 The 6 confirmed bugs that were found and fixed

#	Bug	Where	Root cause / fix
1	Government Grant could go to a bankrupt player	events.c (case 18)	random player chosen without a solvent check; fix: scan forward to the first non-bankrupt player
2	Anti-Speculation forced seller could re-buy their own property	auction.c runAuction	seller could bid and 'pay themselves' (net-zero); fix: exclude the seller from bidding
3	Luxury Property Tax charged without printing	events.c	tax paid with no printf; fix: print owner, property and amount before paying
4	Disaster repair cost charged silently	insurance.c tryAutoRepair	repairs paid but never logged -> apparent cash gaps; fix: print 'Repair Cost : LKR N.'
5	Income Tax could pay a player (negative net worth)	finance.c payTax	13% x negative net worth = negative tax = cash gain; fix: clamp the amount to zero
6	Rule 9 even-development violated when a group member was blocked	finance.c developGroup	blocked member never cleared allFourHouses; fix: clear it, and guard against building a 5th house

10.3 What the analyzer's REVIEW items really mean

A few REVIEW lines are known model approximations, not bugs: the analyzer's value model does not perfectly simulate inflation/ageing timing, auction winners' receiving rent is printed after the payer's bankruptcy line (log ordering), and small residual net-worth diffs appear. The report's conclusion: 0 confirmed bugs on three fresh post-fix games.

පරීක්ෂණ කිරීම: කිරීමාව පිටත කරන log එක නැවත කියවා ගැනුම් සත්‍යය පරීක්ෂා කරයි.

11 Likely Viva Questions

11.1 Why is GameState declared as game[1] instead of just game?

So we never have to write pointers. An array parameter decays to a pointer under the hood, so every function receives read/write access to the original state while the source only uses plain array syntax (game[0].field). It also makes 'pass the whole game' as trivial as passing one argument.

11.2 How do the four strategies differ?

players.c is one big switch on PlayerType per decision. The Aggressive Investor completes monopolies aggressively, the Conservative Banker avoids risk and hotels while in debt, the Risk Taker spends everything and only insures after a loss, and the Opportunistic Trader buys cheap, sells into declines and flips assets.

11.3 What is a 'round' in your game?

A round completes only when every solvent player has passed GO at least once, tracked in playedGoThisRound[]. This usually takes several turns per player. All round-end bookkeeping (loans, insurance expiry, ageing, modifiers, regulations) fires at that point.

11.4 How do you avoid permanently corrupting prices when events happen?

Every effect is a timed ActiveModifier in Economy.modifiers[]. currentMarketValue multiplies all matching modifiers on the fly. When the timer hits zero, decrementModifiers removes it and values return to normal. The only permanent change is inflation via applyRate.

11.5 How is rent calculated?

Base rent x a table multiplier by houses/hotel (x1,x2,x3,x5,x7,x10), then x each active rent modifier (group, global, hotel), then x the condition percent from Table 3. Railways and utilities have their own tables based on how many the owner has.

11.6 What happens on bankruptcy?

The Risk Taker tries to sell its cheapest undeveloped property first. Otherwise liquidateBankruptAssets demolishes buildings, strips ownership and auctions every owned square. Cash is clamped to zero and the player receives no further money (receiveMoney no-ops).

11.7 How does the loan system work?

You pledge owned, unmortgaged, unlocked property as collateral worth 75% of its value. Interest is compounded into the balance every round. At zero rounds left, foreclose seizes and auctions the pledged property, and the player may become bankrupt.

11.8 Why is there a Python file in a C project?

verify_analysis.py is a test tool, not part of the game. It replays a captured game log and checks every money movement, ownership change and dice roll against the rules. It found the six bugs listed in Section 10, all of which were fixed.

11.9 What was the hardest logic to get right?

Rule 9 even development: a blocked group member must not only be skipped, it must also prevent the group from building a hotel, and build targets must never exceed 4 houses. The verification tool caught the violation and the fix is in developGroup.

11.10 What is the anti-speculation act?

A regulation that limits a player to 3 undeveloped properties. After 5 rounds above the limit, the surplus is force-auctioned - and the seller is banned from bidding so they cannot buy it back at net-zero cost.

11.11 Which taxes exist?

Income Tax: market-scaled rate x net worth, clamped at both 25% max and 0% minimum. Community Development Fund: 10% of property value. Luxury Property Tax: 25% of each hotel's value, a government regulation.

11.12 How is the winner decided?

The last solvent player wins. If the game reaches round 500 with several players alive, determine Winner picks the highest net worth (cash + property + buildings - loans).

12 Complete Function Index

Every function in the project, grouped by file. Useful for scanning 'which file has what' in one place.

12.1 main.c

main		

12.2 board.c

setProperty	setRailway	setUtility
setSpecialSquare	initializeBoard	

12.3 players.c

initializePlayers	shouldBuyProperty	shouldConstruct
wantsLoan	wantsToRepayLoan	wantsIncreaseLoan
wantsExtendLoan	wantsRefinance	desiredInsurance
shouldRenovateAgeDepreciation	shouldMaintain	willingToBid
shouldMortgage	shouldRedeemMortgage	shouldPayBail

12.4 finance.c

stripOwnership	resetLoan	adjustOwnedCount
groupOf	receiveMoney	payMoney
liquidateBankruptAssets	payTax	payCommunityFundTax
findPropertyToMortgage	findMortgagedProperty	mortgageProperty
redeemMortgage	handleMortgageDecisions	calculateRent
calculateRailwayRent	calculateUtilityRent	groupSize
ownsMonopoly	developGroup	constructBuildings
calculatePropertyValue	calculateBuildingValue	calculateNetWorth
countUndevelopedProperties	wouldCompleteMonopoly	sellLowValueProperty
sellDecliningProperties	sellUndevelopedPropertyToAuction	forceSellExcess
enforceAntiSpeculation	buyProperty	payRent

12.5 bank.c

groupBasePrice	totalEligibleCollateral	calculateMaxLoan
obtainLoan	repayLoan	handleBankVisit
increaseLoan	extendLoan	refinanceLoan
demolishBuildingsOn	foreclose	processLoans

12.6 auction.c

getAskingValue	runAuction	

12.7 events.c

clampLoanInterest	adjustLoanInterest	executeEvent
triggerEconomicEvent	triggerGovernmentRegulation	decrementEventTimers

12.8 economy.c

applyRate	initEconomy	addModifier
addSourcedModifier	modifierMultiplier	isModifierActive
decrementModifiers	formatLKR	applyInflation
currentMarketValue	currentTaxRatePercent	ageProperties
tryRenovateAgeDepreciation	rentConditionPercent	ageBuildings
performMaintenance	renovateStructuralDamage	

12.9 insurance.c

propertyValue	repairCost	pickDisaster
isCovered	compensationPercent	purchaseInsurance
findPropertyToInsure	findPropertyToRenew	handleInsuranceVisit
tryAutoRepair	disasterName	payCompensation
triggerDisaster	processInsuranceExpiry	

12.10 market.c

initMarket	groupName	pickEligibleGroup
reviewPropertyMarket	drawRegionalCard	printSectionHeader
printBoomDecline	describeRegionalCondition	printRegionalSection
describeOtherCondition	printOtherConditions	displayMarketConditions

12.11 game.c

rollDice	movePlayer	handleJail
playTurn	resolveGroup	determineTurnOrder
countHouses	countHotels	displayRoundSummary
countSolventPlayers	playGame	determineWinner
displayFinalResults	startGame	

13 Appendix - Complete Source Code

All source files in full, as they exist in the project at the time this guide was generated. This section uses syntax-tinted code: green comments, red strings, blue numbers, navy keywords and grey preprocessor lines.

A.1 types.h - the data model (362 lines)

```
#ifndef TYPES_H
#define TYPES_H

#define MAX_PLAYERS      4
#define BOARD_SIZE      40
#define MAX_NAME_LENGTH  50
#define MAX_ROUNDS      500
#define START_MONEY     30000

#define GO_MONEY        2000
#define JAIL_BAIL       300

#define INCOME_TAX_RATE  15 /* base rate at game start, drifts with market */
#define COMMUNITY_FUND_TAX_RATE 10 /* base rate, drifts with market */

#define MAX_HOUSES      4
#define MAX_HOTELS      1

#define LOAN_DURATION_ROUNDS 20
#define LOAN_INTEREST_RATE  8 /* starting rate, drifts with inflation */
#define LOAN_COLLATERAL_PERCENT 75 /* Rule-LK 2: max loan is 75% of collateral */
#define LOAN_EXTEND_ROUNDS  10 /* Rule-LK 5: extension adds 10 rounds */
#define MORTGAGE_REPAY_PERCENT 110 /* redeeming a mortgage costs 110% of its value */

/* Loan interest rate limits */
#define LOAN_INTEREST_MIN  1 /* interest can never go below this */
#define LOAN_INTEREST_MAX  25 /* ...or above this */
#define LOAN_INTEREST_STEP 2 /* cards/regulations move it by 2 points */
#define TAX_RATE_MAX       25
#define HOTEL_LUXURY_TAX_PERCENT 25 /* luxury tax: 25% of a hotel's value */

/* Strategy decision thresholds, in LKR (rule-book strategy rules) */
#define MIN_CASH_SAFETY    1000 /* reserve before spending / borrowing */
#define MIN_CASH_FLOOR    500 /* risk taker's floor before mortgaging */
#define TRADER_CASH_FLOOR 1200 /* opportunistic trader's mortgage floor */
#define COMFORTABLE_CASH  3000 /* conservative banker's comfort line */
#define BAIL_CASH_SPARE    1500 /* conservative banker's bail threshold */
#define RICH_CASH_LINE     5000 /* rich-player threshold */
#define EXPENSIVE_PROP_PRICE 6000 /* price above which a trader insures */

/* Rent tables (Tables 7, 8): one railway/utility owned -> this rent */
#define RAILWAY_RENT_1     250
#define RAILWAY_RENT_2     500
#define RAILWAY_RENT_3    1000
#define RAILWAY_RENT_4    2000
#define UTILITY_RENT_ONE   4 /* rent = 4 x dice */
#define UTILITY_RENT_TWO  10 /* rent = 10 x dice when both owned */

/* Auction rules */
#define AUCTION_OPENING_DIVISOR 2 /* opening bid = asking value / 2 */
#define AUCTION_BID_INCREMENT  250 /* every bid must raise the price by this */
#define AUCTION_MAX_ROUNDS     200 /* safety limit so bidding always ends */

/* Property market review (Rule-LK 30) */
#define MARKET_REVIEW_EVERY  10 /* review happens every 10 rounds */
#define MARKET_COOLDOWN_ROUNDS 30 /* a group can't re-trigger for 30 rounds */

/* Property ageing and neglect (Rule-LK 15, 16, 25-27) */
#define PROPERTY_AGE_LIMIT  50 /* value starts dropping past this age */
#define DEPRECIATION_STEP   5 /* every 5 rounds past the limit */
#define MAX_DEPRECIATION    30
#define RENOVATION_COST_PERCENT 10 /* renovation costs 10% of current value */
#define RENOVATION_RENT_BOOST 5 /* renovated base rent rises 5% */
#define NEGLECT_ROUNDS      20 /* no maintenance this long = structural damage */
#define STRUCTURAL_VALUE_LOSS 15 /* damaged building loses 15% of its value */
#define STRUCTURAL_RENT_LOSS 25 /* ...and 25% of its rent */
#define DAMAGE_REPAIR_MULTIPLIER 150 /* fixing structural damage costs 150% */
```

```

/* Insurance rules */
#define REPAIR_COST_PERCENT      30 /* repairs cost 30% of a property's value */
#define PREMIUM_BASIC_PERCENT   5
#define PREMIUM_COMPREHENSIVE_PERCENT 10
#define PREMIUM_INTERRUPTION_PERCENT 15
#define INSURANCE_DURATION_ROUNDS 20
#define INSURANCE_RENEW_AT      10 /* renew when 10 or fewer rounds remain */
#define INSURANCE_EXPIRY_WARNING 3 /* warn 3 rounds before a policy expires */
#define DISASTER_WEIGHT         20 /* every disaster is equally likely at base */
#define DISASTER_BOOST          30 /* weather/unrest makes one type +30 likely */
#define BI_LOST_INCOME_ROUNDS   5 /* Business Interruption: no rent for 5 rounds */

/* Jail (Rule 13) */
#define JAIL_SQUARE              10
#define JAIL_MAX_TURNS           3

/* Event / card deck sizes */
#define EVENT_CARD_COUNT        20
#define ECONOMIC_EVENT_COUNT     8
#define REGULATION_COUNT        8
#define REGIONAL_CARD_COUNT     12

/* Round cadences for periodic events */
#define ECONOMY_CYCLE            10 /* disasters, inflation, market review */
#define EVENT_CYCLE              15 /* economic events and regional cards */
#define REGULATION_CYCLE        20 /* government regulations */

/* Anti-Speculation Act (Rule-LK 8) */
#define MAX_UNDEVELOPED_PROPS    3 /* at most this many undeveloped properties */
#define ANTI_SPEC_TRIGGER_ROUNDS 5 /* enforced after 5 consecutive rounds */

/* Board Square Types */
typedef enum
{
    GO,
    PROPERTY,
    RAILWAY,
    UTILITY,
    TAX,
    COMMUNITY_FUND,
    EVENT,
    BANK,
    INSURANCE,
    JAIL,
    FREE_PARKING,
    GO_TO_JAIL
} SquareType;

/* Property Colour Groups */
typedef enum
{
    BROWN,
    LIGHT_BLUE,
    PINK,
    ORANGE,
    RED,
    YELLOW,
    GREEN,
    DARK_BLUE,
    NO_GROUP
} PropertyGroup;

/* Player Strategies */
typedef enum
{
    AGGRESSIVE_INVESTOR,
    CONSERVATIVE_BANKER,
    RISK_TAKER,
    OPPORTUNISTIC_TRADER
} PlayerType;

```

```

/* Insurance Types */
typedef enum
{
    NO_INSURANCE,
    BASIC_INSURANCE,
    COMPREHENSIVE_INSURANCE,
    BUSINESS_INTERRUPTION
} InsuranceType;

/* Disaster Types (Rule-LK 10) */
typedef enum
{
    FIRE,
    FLOOD,
    RIOT,
    BUILDING_COLLAPSE,
    ELECTRICAL_FAILURE
} DisasterType;

typedef struct
{
    int active;
    int amount;
    int interestRate;
    int remainingRounds;
} Loan;

typedef struct
{
    char name[MAX_NAME_LENGTH];

    PropertyGroup group;

    int purchasePrice;
    int mortgageValue;
    int baseRent;

    int houseCost;
    int hotelCost;

    int owner;          /* -1 means unowned */

    int houses;
    int hotel;

    int mortgaged;
    int loanLocked;     /* pledged as loan collateral */

    InsuranceType insurance;
    int insuranceRoundsLeft;

    int damaged;        /* disaster damage - no rent until repaired */
    int repairCostOwed;
    int lostIncomeRoundsLeft; /* Business Interruption: no rent for 5 rounds */

    int age;
    int depreciation;

    int condition;      /* building condition, Rule-LK 25-27 */
    int roundsSinceMaintenance;
    int structurallyDamaged;
    int preDamagePurchasePrice; /* remembered so renovation can restore it */
    int preDamageBaseRent;
    int maintenanceCostMultiplierPercent;
} Property;

typedef struct
{
    int index;

```

```

    SquareType type;

    char name[MAX_NAME_LENGTH];

    Property property;
} Square;

typedef struct
{
    char name[MAX_NAME_LENGTH];

    PlayerType strategy;

    int position;

    int cash;

    int inJail;

    int jailTurns;

    int bankrupt;

    int propertiesOwned;

    int railwaysOwned;

    int utilitiesOwned;

    Loan loan;

    int sufferedLoss; /* Risk Taker only insures after a loss */

    int antiSpecRounds; /* rounds in a row above the 3-undeveloped limit */
} Player;

/* Kinds of temporary modifiers that can be active at once. Each
event, card, regulation, market review and regional card adds a
timed entry to Economy.modifiers[]. All matching entries are
multiplied together (Rule-LK 34) so effects are cumulative and
never permanently corrupt the base property values/costs. */
typedef enum
{
    MOD_VALUE_GLOBAL, /* all property values */
    MOD_GROUP_VALUE, /* one colour group's property values */
    MOD_INDEX_VALUE, /* a single square's value (any type) */
    MOD_RAIL_VALUE, /* all railway station values */
    MOD_PURCHASE_PRICE, /* direct purchase prices (boom +15%) */
    MOD_RENT_GLOBAL, /* all property/rail/utility rent */
    MOD_GROUP_RENT, /* one colour group's rent */
    MOD_HOTEL_RENT, /* hotel rent */
    MOD_RAIL_RENT, /* railway rent */
    MOD_UTIL_RENT, /* utility rent */
    MOD_CONSTRUCTION, /* house/hotel construction costs */
    MOD_INSURANCE, /* insurance premiums */
    MOD_MARKET_MORTGAGE, /* mortgage values (boom +15% / decline -10%) */
    MOD_AUCTION_PRICE, /* auction starting prices (decline -25%) */
    MOD_FLOOD_RISK, /* Heavy Monsoon - flood more likely */
    MOD_RIOT_RISK, /* Political Unrest - riot more likely */
    MOD_BI_CLAIMS, /* Political Unrest - business interruption claims */
    MOD RECESSION /* Economic Recession - strategies may react */
} ModifierType;

/* Where a modifier came from - used by Rule-LK 36 to group the
currently active conditions into Market Boom / Market Decline /
Regional Development sections. */
typedef enum
{
    SRC_GENERAL, /* events, national cards, government regulations */
    SRC_BOOM, /* Property Market Review - boom group */
    SRC_DECLINE, /* Property Market Review - decline group */

```



```

    SRC_REGIONAL    /* Regional Development Card */

} ModifierSource;

typedef struct
{
    ModifierType type;
    int group;      /* group for GROUP modifiers, else -1 */
    int index;      /* square index for INDEX modifiers, else -1 */
    int percent;    /* 100 means no change */
    int roundsLeft;
    ModifierSource source;

} ActiveModifier;

#define MAX_MODIFIERS 48

/* Current state of the economy - inflation, interest rate, and every
   temporary rent/cost bonus or penalty from events and the market. */
typedef struct
{
    int inflationRate;
    int loanInterestRate;

    int constructionSuspendedRoundsLeft;

    int closedPropertyIndex;
    int closedPropertyRoundsLeft;

    int incomeTaxRate;

    int antiSpeculationActive;

    int currentCardIndex; /* position in the National Event Card deck */

    int groupCooldownUntilRound[NO_GROUP];

    int lastBoomGroup;
    int lastDeclineGroup;

    ActiveModifier modifiers[MAX_MODIFIERS];
    int modifierCount;

} Economy;

/* Everything the game needs to remember, bundled into one struct so
   it can be passed to functions as a plain array parameter instead
   of using global variables or pointers. */
typedef struct
{
    Square board[BOARD_SIZE];
    Player players[MAX_PLAYERS];
    Economy economy;

} GameState;

#endif

```

A.2 functions.h - the project API (185 lines)

```

#ifndef FUNCTIONS_H
#define FUNCTIONS_H

#include "types.h"

/* board.c */

void initializeBoard(GameState game[]);

void setProperty(GameState game[],
                 int index,
                 char name[],
                 PropertyGroup group,
                 int purchasePrice,

```

```

        int baseRent,
        int mortgageValue,
        int houseCost,
        int hotelCost);

void setRailway(GameState game[], int index, char name[], int purchasePrice);
void setUtility(GameState game[], int index, char name[], int purchasePrice);

void setSpecialSquare(GameState game[],
        int index,
        SquareType type,
        char name[]);

/* players.c */

void initializePlayers(GameState game[]);

int shouldBuyProperty(GameState game[], int playerIndex);
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel);
int wantsLoan(GameState game[], int playerIndex);
int wantsToRepayLoan(GameState game[], int playerIndex);
int wantsIncreaseLoan(GameState game[], int playerIndex);
int wantsExtendLoan(GameState game[], int playerIndex);
int wantsRefinance(GameState game[], int playerIndex);
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex);
int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent);
int shouldMaintain(GameState game[], int playerIndex, int condition, int cost);
int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid);
int shouldMortgage(GameState game[], int playerIndex);
int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost);
int shouldPayBail(GameState game[], int playerIndex);

/* finance.c */

void stripOwnership(GameState game[], int index);
void resetLoan(GameState game[], int playerIndex);
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta);
PropertyGroup groupOf(GameState game[], int index);

void buyProperty(GameState game[], int playerIndex);
void payRent(GameState game[], int playerIndex, int diceValue);
void payTax(GameState game[], int playerIndex);
void payCommunityFundTax(GameState game[], int playerIndex);
void payMoney(GameState game[], int playerIndex, int amount);
void liquidateBankruptAssets(GameState game[], int playerIndex);
void receiveMoney(GameState game[], int playerIndex, int amount);

int calculatePropertyValue(GameState game[], int playerIndex);
int calculateBuildingValue(GameState game[], int playerIndex);
int calculateNetWorth(GameState game[], int playerIndex);

int findPropertyToMortgage(GameState game[], int playerIndex);
int findMortgagedProperty(GameState game[], int playerIndex);
void mortgageProperty(GameState game[], int playerIndex);
void redeemMortgage(GameState game[], int playerIndex);
void handleMortgageDecisions(GameState game[], int playerIndex);

int calculateRent(GameState game[], int position);
int calculateRailwayRent(GameState game[], int playerIndex);
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue);
int countUndevelopedProperties(GameState game[], int playerIndex);

int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos);
void sellLowValueProperty(GameState game[], int playerIndex);
void sellDecliningProperties(GameState game[], int playerIndex);
void enforceAntiSpeculation(GameState game[]);
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex);

int groupSize(GameState game[], PropertyGroup group);
int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group);
int developGroup(GameState game[], int playerIndex, PropertyGroup group);
void constructBuildings(GameState game[], int playerIndex);

/* game.c */

void startGame(GameState game[]);
void playGame(GameState game[], int turnOrder[]);

```

```

int playTurn(GameState game[], int playerIndex);
int movePlayer(GameState game[], int playerIndex, int dice);
int rollDice(void);
int handleJail(GameState game[], int playerIndex);

int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
                 int output[], int startIndex);

void determineTurnOrder(GameState game[], int turnOrder[]);
void displayRoundSummary(GameState game[], int round);
int countSolventPlayers(GameState game[]);
int countHouses(GameState game[], int playerIndex);
int countHotels(GameState game[], int playerIndex);
int determineWinner(GameState game[]);
void displayFinalResults(GameState game[]);

/* events.c */

void clampLoanInterest(GameState game[]);
void adjustLoanInterest(GameState game[], int deltaPercent);
void executeEvent(GameState game[], int playerIndex);
void triggerEconomicEvent(GameState game[]);
void triggerGovernmentRegulation(GameState game[]);
void decrementEventTimers(GameState game[]);

/* market.c */

void initMarket(GameState game[]);
void groupName(PropertyGroup group, char buffer[]);
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2);
void reviewPropertyMarket(GameState game[], int currentRound);
void drawRegionalCard(GameState game[]);
void displayMarketConditions(GameState game[]);

/* bank.c */

int groupBasePrice(GameState game[], PropertyGroup group);
int totalEligibleCollateral(GameState game[], int playerIndex);
int calculateMaxLoan(GameState game[], int playerIndex);
void obtainLoan(GameState game[], int playerIndex);
void repayLoan(GameState game[], int playerIndex, int amount);
void handleBankVisit(GameState game[], int playerIndex);
void increaseLoan(GameState game[], int playerIndex);
void extendLoan(GameState game[], int playerIndex);
void refinanceLoan(GameState game[], int playerIndex);
void demolishBuildingsOn(GameState game[], int index);
void foreclose(GameState game[], int playerIndex);
void processLoans(GameState game[]);

/* insurance.c */

int propertyValue(GameState game[], int propIndex);
int repairCost(GameState game[], int propIndex);
DisasterType pickDisaster(GameState game[]);
int isCovered(InsuranceType policy, DisasterType disaster);
int compensationPercent(InsuranceType policy);
void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type);
int findPropertyToInsure(GameState game[], int playerIndex);
int findPropertyToRenew(GameState game[], int playerIndex);
void handleInsuranceVisit(GameState game[], int playerIndex);
void tryAutoRepair(GameState game[], int playerIndex);
void triggerDisaster(GameState game[]);
void processInsuranceExpiry(GameState game[]);

/* economy.c */

int applyRate(int oldValue, int ratePercent);
void initEconomy(GameState game[]);
void applyInflation(GameState game[]);
void addModifier(GameState game[], ModifierType type, int group, int index,
                int percent, int roundsLeft);
void addSourcedModifier(GameState game[], ModifierType type, int group,
                       int index, int percent, int roundsLeft,
                       ModifierSource source);
int modifierMultiplier(GameState game[], ModifierType type, int group, int index);
int isModifierActive(GameState game[], ModifierType type, int group, int index);
void decrementModifiers(GameState game[]);

```

```

void formatLKR(int amount, char out[]);
int currentMarketValue(GameState game[], int propIndex);
int currentTaxRatePercent(GameState game[], int baseRate);
void ageProperties(GameState game[]);
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex);
int rentConditionPercent(int condition);
void ageBuildings(GameState game[]);
void performMaintenance(GameState game[], int playerIndex);
void renovateStructuralDamage(GameState game[], int playerIndex);

/* auction.c */

int getAskingValue(GameState game[], int propIndex);
void runAuction(GameState game[], int propIndex, int sellerIndex);

#endif

```

A.3 main.c - entry point (18 lines)

```

#include <stdio.h>
#include "types.h"
#include "functions.h"

int main(void)
{
    /* the only place a GameState is ever created */
    GameState game[1] = {0};

    printf("=====\n");
    printf("      MONOPOLY-LK SIMULATION\n");
    printf("=====\n\n");

    startGame(game);

    return 0;
}

```

A.4 board.c - board construction (206 lines)

```

#include <stdio.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void setProperty(GameState game[],
                 int index,
                 char name[],
                 PropertyGroup group,
                 int purchasePrice,
                 int baseRent,
                 int mortgageValue,
                 int houseCost,
                 int hotelCost)
{
    game[0].board[index].index = index;
    game[0].board[index].type = PROPERTY;

    strcpy(game[0].board[index].name, name);
    strcpy(game[0].board[index].property.name, name);

    game[0].board[index].property.group = group;
    game[0].board[index].property.purchasePrice = purchasePrice;
    game[0].board[index].property.baseRent = baseRent;
    game[0].board[index].property.mortgageValue = mortgageValue;
    game[0].board[index].property.houseCost = houseCost;
    game[0].board[index].property.hotelCost = hotelCost;

    /* nobody owns it yet */
    game[0].board[index].property.owner = -1;

    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;

    game[0].board[index].property.insurance = NO_INSURANCE;
    game[0].board[index].property.insuranceRoundsLeft = 0;
}

```

```

game[0].board[index].property.damaged = 0;
game[0].board[index].property.repairCostOwed = 0;
game[0].board[index].property.lostIncomeRoundsLeft = 0;

game[0].board[index].property.age = 0;
game[0].board[index].property.depreciation = 0;
game[0].board[index].property.condition = 100;

game[0].board[index].property.roundsSinceMaintenance = 0;
game[0].board[index].property.structurallyDamaged = 0;
game[0].board[index].property.preDamagePurchasePrice = 0;
game[0].board[index].property.preDamageBaseRent = 0;
game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
}

void setRailway(GameState game[], int index, char name[], int purchasePrice)
{
    game[0].board[index].index = index;
    game[0].board[index].type = RAILWAY;

    strcpy(game[0].board[index].name, name);
    strcpy(game[0].board[index].property.name, name);

    game[0].board[index].property.purchasePrice = purchasePrice;
    game[0].board[index].property.mortgageValue = purchasePrice / 2;
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;

    /* railways can't have houses/hotels/insurance, but we still zero
       these out since GameState is local now and won't do it for us */
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
    game[0].board[index].property.insuranceRoundsLeft = 0;
    game[0].board[index].property.damaged = 0;
    game[0].board[index].property.repairCostOwed = 0;
    game[0].board[index].property.lostIncomeRoundsLeft = 0;
    game[0].board[index].property.age = 0;
    game[0].board[index].property.depreciation = 0;
    game[0].board[index].property.condition = 100;
    game[0].board[index].property.roundsSinceMaintenance = 0;
    game[0].board[index].property.structurallyDamaged = 0;
    game[0].board[index].property.preDamagePurchasePrice = 0;
    game[0].board[index].property.preDamageBaseRent = 0;
    game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
    game[0].board[index].property.baseRent = 0;
    game[0].board[index].property.houseCost = 0;
    game[0].board[index].property.hotelCost = 0;
}

void setUtility(GameState game[], int index, char name[], int purchasePrice)
{
    game[0].board[index].index = index;
    game[0].board[index].type = UTILITY;

    strcpy(game[0].board[index].name, name);
    strcpy(game[0].board[index].property.name, name);

    game[0].board[index].property.purchasePrice = purchasePrice;
    game[0].board[index].property.mortgageValue = purchasePrice / 2;
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;

    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
    game[0].board[index].property.insuranceRoundsLeft = 0;
    game[0].board[index].property.damaged = 0;
    game[0].board[index].property.repairCostOwed = 0;
    game[0].board[index].property.lostIncomeRoundsLeft = 0;
    game[0].board[index].property.age = 0;
    game[0].board[index].property.depreciation = 0;
    game[0].board[index].property.condition = 100;
    game[0].board[index].property.roundsSinceMaintenance = 0;

```

```

game[0].board[index].property.structurallyDamaged = 0;
game[0].board[index].property.preDamagePurchasePrice = 0;
game[0].board[index].property.preDamageBaseRent = 0;
game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
game[0].board[index].property.baseRent = 0;
game[0].board[index].property.houseCost = 0;
game[0].board[index].property.hotelCost = 0;
}

void setSpecialSquare(GameState game[],
                    int index,
                    SquareType type,
                    char name[])
{
    game[0].board[index].index = index;
    game[0].board[index].type = type;

    strcpy(game[0].board[index].name, name);

    /* Owner must be -1 (unowned), not the default 0, otherwise this
       square would look like it belongs to Player 0. */
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
}

void initializeBoard(GameState game[])
{
    setSpecialSquare(game, 0, GO, "GO");
    setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund");
    setSpecialSquare(game, 4, TAX, "Income Tax");
    setSpecialSquare(game, 7, EVENT, "National Event Card");
    setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting");
    setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance");
    setSpecialSquare(game, 20, FREE_PARKING, "Free Parking");
    setSpecialSquare(game, 22, EVENT, "National Event Card");
    setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail");
    setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance");
    setSpecialSquare(game, 36, EVENT, "National Event Card");
    setSpecialSquare(game, 38, BANK, "Bank of Ceylon");

    setRailway(game, 5, "Colombo Fort Railway Station", 1500);
    setRailway(game, 15, "Kandy Railway Station", 1500);
    setRailway(game, 25, "Galle Railway Station", 1500);
    setRailway(game, 35, "Jaffna Railway Station", 1500);

    setUtility(game, 12, "Ceylon Electricity Board", 1500);
    setUtility(game, 28, "National Water Supply and Drainage Board", 1500);

    /* Brown */
    setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000);
    setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000);

    /* Light Blue */
    setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000);
    setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000);
    setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000);

    /* Pink */
    setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000);
    setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000);
    setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000);

    /* Orange */
    setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000);
    setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000);
    setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000);

    /* Red */
    setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000);
    setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000);
    setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000);
}

```

```

/* Yellow */
setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000);
setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000);
setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000);

/* Green */
setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000);
setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000);
setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000);

/* Dark Blue */
setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000);
setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000);
}

```

A.5 players.c - the 4 AI strategies (406 lines)

```

#include <stdio.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initializePlayers(GameState game[])
{
    int i;

    strcpy(game[0].players[0].name, "Aggressive Investor");
    game[0].players[0].strategy = AGGRESSIVE_INVESTOR;

    strcpy(game[0].players[1].name, "Conservative Banker");
    game[0].players[1].strategy = CONSERVATIVE_BANKER;

    strcpy(game[0].players[2].name, "Risk Taker");
    game[0].players[2].strategy = RISK_TAKER;

    strcpy(game[0].players[3].name, "Opportunistic Trader");
    game[0].players[3].strategy = OPPORTUNISTIC_TRADER;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        game[0].players[i].position = 0;
        game[0].players[i].cash = START_MONEY;

        game[0].players[i].inJail = 0;
        game[0].players[i].jailTurns = 0;

        game[0].players[i].bankrupt = 0;

        game[0].players[i].propertiesOwned = 0;
        game[0].players[i].railwaysOwned = 0;
        game[0].players[i].utilitiesOwned = 0;

        resetLoan(game, i);

        game[0].players[i].sufferedLoss = 0;
        game[0].players[i].antiSpecRounds = 0;
    }
}

/* Decides whether a player wants to buy the property they just
   landed on. Each strategy has its own rule, from Section 3. */
int shouldBuyProperty(GameState game[], int playerIndex)
{
    int price;
    int position;

    position = game[0].players[playerIndex].position;
    price = game[0].board[position].property.purchasePrice;

    /* match the price buyProperty() actually charges (Rule-LK 31/34) */
    if(game[0].board[position].type == PROPERTY)
    {
        price = (currentMarketValue(game, position) *
                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                     game[0].board[position].property.group, -1)) / 100;
    }

    switch(game[0].players[playerIndex].strategy)

```

```

{
    case AGGRESSIVE_INVESTOR:
        /* buys to complete a monopoly group even if it dips below
           the usual 1000 reserve (Section 3.1) */
        if(wouldCompleteMonopoly(game, playerIndex, position))
            return 1;
        return (game[0].players[playerIndex].cash - price >= 1000);

    case CONSERVATIVE_BANKER:
        /* avoids property purchases entirely during a recession,
           but will still buy rail/utility income (Section 3.2) */
        if(game[0].board[position].type == PROPERTY &&
            isModifierActive(game, MOD_RECESSION, -1, -1))
            return 0;
        return (game[0].players[playerIndex].cash - price >=
            game[0].players[playerIndex].cash / 2);

    case RISK_TAKER:
        return (game[0].players[playerIndex].cash >= price);

    case OPPORTUNISTIC_TRADER:
        /* never buys into a group that is currently declining */
        if(game[0].board[position].type == PROPERTY &&
            modifierMultiplier(game, MOD_GROUP_VALUE,
                game[0].board[position].property.group, -1) < 100)
            return 0;
        /* only buys cheaper properties */
        return (price <= game[0].players[playerIndex].cash * 40 / 100);

    default:
        return 0;
}
}

/* Decides whether a player is willing to spend on a house/hotel. */
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash - cost >= MIN_CASH_SAFETY);

        case CONSERVATIVE_BANKER:
            /* never builds hotels while a loan is active - houses are
               still fine (Section 3.2) */
            if(game[0].players[playerIndex].loan.active && isHotel)
                return 0;
            return (cash - cost >= cash / 2);

        case RISK_TAKER:
            return (cash >= cost);

        case OPPORTUNISTIC_TRADER:
            /* builds eagerly while a construction subsidy is active */
            if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)
                return (cash - cost >= cash * 30 / 100);

            /* delays building while inflation pushes costs up */
            if(game[0].economy.inflationRate > 0)
                return (cash - cost >= cash * 60 / 100);

            return (cash - cost >= cash * 40 / 100);

        default:
            return 0;
    }
}

int wantsLoan(GameState game[], int playerIndex)
{

```



```

int cash;

cash = game[0].players[playerIndex].cash;

switch(game[0].players[playerIndex].strategy)
{
    case AGGRESSIVE_INVESTOR:
        return 1;

    case CONSERVATIVE_BANKER:
        /* only borrows if bankruptcy is close */
        return (cash < MIN_CASH_SAFETY);

    case RISK_TAKER:
        return 1;

    case OPPORTUNISTIC_TRADER:
        return (cash < RICH_CASH_LINE);

    default:
        return 0;
}
}

int wantsToRepayLoan(GameState game[], int playerIndex)
{
    int cash;
    int loanAmount;

    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash >= 2 * loanAmount);

        case CONSERVATIVE_BANKER:
            return (cash >= loanAmount);

        case RISK_TAKER:
            /* rarely repays - prefers to keep using the cash */
            return (cash >= 5 * loanAmount);

        case OPPORTUNISTIC_TRADER:
            return (cash >= (loanAmount * 3) / 2);

        default:
            return 0;
    }
}

/* Rule-LK 5: whether a player wants to borrow more against new
collateral during a Bank visit. */
int wantsIncreaseLoan(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return 1;

        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (cash < RICH_CASH_LINE);

        default:
            return 0;
    }
}

```

```

}

/* Rule-LK 5: whether a player wants to extend the loan term. */
int wantsExtendLoan(GameState game[], int playerIndex)
{
    int remainingRounds;
    int cash;
    int loanAmount;

    remainingRounds = game[0].players[playerIndex].loan.remainingRounds;
    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            /* prefers to borrow more rather than extend */
            return 0;

        case CONSERVATIVE_BANKER:
            return (remainingRounds <= 5 && cash < loanAmount);

        case RISK_TAKER:
            return (remainingRounds <= 10);

        case OPPORTUNISTIC_TRADER:
            return (remainingRounds <= 5);

        default:
            return 0;
    }
}

/* Rule-LK 5: whether a player wants to refinance at the current rate
(only makes sense when the current economy rate is lower). */
int wantsRefinance(GameState game[], int playerIndex)
{
    return (game[0].economy.loanInterestRate <
            game[0].players[playerIndex].loan.interestRate);
}

/* Which insurance policy (if any) a player wants for a property. */
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
{
    int hasHotel;
    int purchasePrice;

    hasHotel = game[0].board[propIndex].property.hotel;
    purchasePrice = game[0].board[propIndex].property.purchasePrice;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            if(hasHotel)
                return COMPREHENSIVE_INSURANCE;
            return BASIC_INSURANCE;

        case CONSERVATIVE_BANKER:
            return COMPREHENSIVE_INSURANCE;

        case RISK_TAKER:
            /* only insures after already suffering a loss */
            if(!game[0].players[playerIndex].sufferedLoss)
                return NO_INSURANCE;
            if(hasHotel)
                return BUSINESS_INTERRUPTIOIN;
            return BASIC_INSURANCE;

        case OPPORTUNISTIC_TRADER:
            if(purchasePrice >= EXPENSIVE_PROP_PRICE)
                return COMPREHENSIVE_INSURANCE;
            return NO_INSURANCE;

        default:
            return NO_INSURANCE;
    }
}

```

```

/* Whether a player will pay to renovate away age depreciation. */
int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
{
    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (depreciationPercent >= 10);

        case OPPORTUNISTIC_TRADER:
            return (depreciationPercent >= 15);

        default:
            /* not specified for these two strategies - reasonable default */
            return (depreciationPercent >= 20);
    }
}

int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    if(game[0].players[playerIndex].strategy == RISK_TAKER)
    {
        /* ignores depreciation until repair becomes unavoidable */
        return (condition < 25 && cash >= cost);
    }

    return (cash - cost >= cash * 20 / 100);
}

int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
{
    int marketValue;

    if(game[0].players[playerIndex].cash < candidateBid)
        return 0;

    marketValue = currentMarketValue(game, propIndex);

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (candidateBid <= (marketValue * 120) / 100);

        case CONSERVATIVE_BANKER:
            return (candidateBid < marketValue);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (candidateBid <= (marketValue * 80) / 100);

        default:
            return 0;
    }
}

int shouldMortgage(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return (cash < MIN_CASH_FLOOR);

        case AGGRESSIVE_INVESTOR:
            return (cash < MIN_CASH_SAFETY);
    }
}

```

```

        case OPPORTUNISTIC_TRADER:
            return (cash < TRADER_CASH_FLOOR);

        default:
            return 0;
    }
}

int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
{
    return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);
}

/* Whether a player pays bail after failing to roll doubles.
   The assignment doesn't specify which strategy prefers which
   option, so this is a reasonable assumption per strategy. */
int shouldPayBail(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash >= BAIL_CASH_SPARE);

        case RISK_TAKER:
            return 0;

        case AGGRESSIVE_INVESTOR:
        case OPPORTUNISTIC_TRADER:
            return (cash >= RICH_CASH_LINE);

        default:
            return 0;
    }
}

```

A.6 game.c - the game engine (641 lines)

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include "types.h"
#include "functions.h"

int rollDice(void)
{
    int die1 = rand() % 6 + 1;
    int die2 = rand() % 6 + 1;

    return die1 + die2;
}

/* Moves a player, returns 1 if they passed or landed on GO. */
int movePlayer(GameState game[], int playerIndex, int dice)
{
    int oldPosition;
    int passedGo;
    char moneyBuffer[32];

    oldPosition = game[0].players[playerIndex].position;
    game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;

    printf("%s moves from Square %d to Square %d.\n",
           game[0].players[playerIndex].name,
           oldPosition,
           game[0].players[playerIndex].position);

    passedGo = 0;

    if(oldPosition + dice >= BOARD_SIZE)
    {
        passedGo = 1;

        receiveMoney(game, playerIndex, GO_MONEY);
    }
}

```

```

        printf("%s passed GO.\n", game[0].players[playerIndex].name);

        formatLKR(GO_MONEY, moneyBuffer);

        printf("Collected LKR %s\n", moneyBuffer);

        printf("Current Balance : LKR %d\n",
            game[0].players[playerIndex].cash);
    }

    return passedGo;
}

/* Handles a player's turn while they're in jail.
Return codes:
    0 = wasn't in jail - take a normal turn
    1 = was in jail (released or not) - turn ends, no move this turn */
int handleJail(GameState game[], int playerIndex)
{
    int die1, die2;

    if(!game[0].players[playerIndex].inJail)
        return 0;

    die1 = rand() % 6 + 1;
    die2 = rand() % 6 + 1;

    printf("%s is in Jail. Rolled %d and %d.\n",
        game[0].players[playerIndex].name, die1, die2);

    if(die1 == die2)
    {
        printf("Doubles! %s is released from Jail.\n",
            game[0].players[playerIndex].name);

        /* Rule 13: rolling doubles releases the player, but they do
            not move this turn - they remain on the Jail square and
            move on their next turn. */
        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;

        return 1;
    }

    /* Rule 13: paying bail is a strategy choice made after a failed
        doubles roll. The player is released but does not move this
        turn. */
    if(shouldPayBail(game, playerIndex) &&
        game[0].players[playerIndex].cash >= JAIL_BAIL)
    {
        printf("%s pays bail of LKR %d and is released from Jail.\n",
            game[0].players[playerIndex].name, JAIL_BAIL);

        payMoney(game, playerIndex, JAIL_BAIL);

        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;

        return 1;
    }

    game[0].players[playerIndex].jailTurns++;

    if(game[0].players[playerIndex].jailTurns >= JAIL_MAX_TURNS)
    {
        printf("%s has been in Jail for %d turns - must pay bail of "
            "LKR %d and is released.\n",
            game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);

        payMoney(game, playerIndex, JAIL_BAIL);

        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;

        return 1;
    }
}

```

```

}

printf("%s remains in Jail (%d/%d turns).\n",
       game[0].players[playerIndex].name,
       game[0].players[playerIndex].jailTurns,
       JAIL_MAX_TURNS);

return 1;
}

/* One player's whole turn, matching Rule 3. Returns 1 if GO was
passed at any point during the turn. */
int playTurn(GameState game[], int playerIndex)
{
    int dice;
    int pos;
    int passedGo;
    int jailResult;
    SquareType squareType;

    if(game[0].players[playerIndex].bankrupt)
        return 0;

    printf("\n-----\n");
    printf("%s's Turn\n", game[0].players[playerIndex].name);

    /* Step 1: fix any damaged, worn, or neglected buildings first */
    tryAutoRepair(game, playerIndex);
    performMaintenance(game, playerIndex);
    renovateStructuralDamage(game, playerIndex);

    passedGo = 0;

    jailResult = handleJail(game, playerIndex);

    if(jailResult == 1)
        return 0; /* was in jail - released or not, no move this turn */

    /* jailResult == 0: wasn't in jail - take a completely normal turn. */
    dice = rollDice();
    printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);

    if(movePlayer(game, playerIndex, dice))
        passedGo = 1;

    /* Step 4: resolve landing action */
    pos = game[0].players[playerIndex].position;
    squareType = game[0].board[pos].type;

    if(squareType == PROPERTY || squareType == RAILWAY || squareType == UTILITY)
    {
        payRent(game, playerIndex, dice);
        buyProperty(game, playerIndex);
    }
    else if(squareType == EVENT)
    {
        executeEvent(game, playerIndex);
    }
    else if(squareType == TAX)
    {
        payTax(game, playerIndex);
    }
    else if(squareType == COMMUNITY_FUND)
    {
        payCommunityFundTax(game, playerIndex);
    }
    else if(squareType == GO_TO_JAIL)
    {
        printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);
        game[0].players[playerIndex].position = JAIL_SQUARE;
        game[0].players[playerIndex].inJail = 1;
        game[0].players[playerIndex].jailTurns = 0;
    }
    else if(squareType == BANK)
    {
        handleBankVisit(game, playerIndex);
    }
}

```

```

else if(squareType == INSURANCE)
{
    handleInsuranceVisit(game, playerIndex);
}
/* GO, JAIL (just visiting), FREE_PARKING need no action */

/* Step 6: build if eligible */
constructBuildings(game, playerIndex);

/* Section 3.4: the Opportunistic Trader first dumps any property
that economic events have marked for a decline. */
sellDecliningProperties(game, playerIndex);

/* Step 7: mortgage or redeem if the player's situation calls for it */
handleMortgageDecisions(game, playerIndex);

return passedGo;
}

/* Resolves one group of (possibly tied) players into a strict order.
Writes the result into output[] starting at startIndex, and
returns how many players it wrote. Any players still tied after
rolling call this function again on just themselves. */
int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
int output[], int startIndex)
{
    int rolls[MAX_PLAYERS];
    int i, j, temp;
    int runStart, runLength;
    int tiedPlayers[MAX_PLAYERS];
    int written;
    int k;

    written = 0;

    for(i = 0; i < groupSize; i++)
    {
        rolls[i] = rollDice();
        printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);
    }

    /* sort this group by roll, highest first */
    for(i = 0; i < groupSize - 1; i++)
    {
        for(j = 0; j < groupSize - 1 - i; j++)
        {
            if(rolls[j] < rolls[j + 1])
            {
                temp = rolls[j];
                rolls[j] = rolls[j + 1];
                rolls[j + 1] = temp;

                temp = groupPlayers[j];
                groupPlayers[j] = groupPlayers[j + 1];
                groupPlayers[j + 1] = temp;
            }
        }
    }

    /* walk the sorted list - a run of equal rolls is a tie */
    i = 0;

    while(i < groupSize)
    {
        runStart = i;
        runLength = 1;

        while(runStart + runLength < groupSize &&
            rolls[runStart + runLength] == rolls[runStart])
        {
            runLength++;
        }

        if(runLength == 1)
        {
            output[startIndex + written] = groupPlayers[runStart];

```

```

        written++;
    }
    else
    {
        printf("Tie detected between %d players at %d. Rolling again.\n",
            runLength, rolls[runStart]);

        for(k = 0; k < runLength; k++)
            tiedPlayers[k] = groupPlayers[runStart + k];

        written += resolveGroup(game, tiedPlayers, runLength,
            output, startIndex + written);
    }

    i = runStart + runLength;
}

return written;
}

void determineTurnOrder(GameState game[], int turnOrder[])
{
    int allPlayers[MAX_PLAYERS];
    int i;

    for(i = 0; i < MAX_PLAYERS; i++)
        allPlayers[i] = i;

    printf("\nDetermining the First Player\n");

    resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);

    printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);

    printf("\nTurn order:\n");
    for(i = 0; i < MAX_PLAYERS; i++)
        printf("%s\n", game[0].players[turnOrder[i]].name);
}

int countHouses(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            !game[0].board[i].property.hotel)
        {
            count += game[0].board[i].property.houses;
        }
    }

    return count;
}

int countHotels(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            game[0].board[i].property.hotel)
        {
            count++;
        }
    }

    return count;
}

```



```

}

void displayRoundSummary(GameState game[], int round)
{
    int i;

    printf("\n=====\\n");
    printf("Round %d Summary\\n", round);
    printf("=====\\n");

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        char cashBuf[32];
        char netBuf[32];
        char loanBuf[32];

        formatLKR(game[0].players[i].cash, cashBuf);
        formatLKR(calculateNetWorth(game, i), netBuf);

        printf("%s\\n", game[0].players[i].name);
        printf("Cash : LKR %s\\n", cashBuf);
        printf("Net Worth : LKR %s\\n", netBuf);
        printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
        printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
        printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
        printf("Houses : %d\\n", countHouses(game, i));
        printf("Hotels : %d\\n", countHotels(game, i));

        if(game[0].players[i].loan.active)
        {
            formatLKR(game[0].players[i].loan.amount, loanBuf);
            printf("Outstanding Loan : LKR %s\\n", loanBuf);
        }
        else
        {
            printf("Outstanding Loan : None\\n");
        }

        if(game[0].players[i].bankrupt)
            printf("Status : BANKRUPT\\n");

        printf("-----\\n");
    }
}

int countSolventPlayers(GameState game[])
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!game[0].players[i].bankrupt)
            count++;
    }

    return count;
}

/* A round is not "each player took one turn" - it only finishes
once every player still in the game has passed GO at least once.
Since players move different distances each turn, this usually
takes several turns per player, not one - so this loop keeps
cycling through turnOrder until everyone has crossed GO. */
void playGame(GameState game[], int turnOrder[])
{
    int round;
    int turnPointer;
    int playerIndex;
    int passedGoThisRound[MAX_PLAYERS];
    int i;
    int allPassed;
    int passedGo;

    round = 1;

```

```

turnPointer = 0;

for(i = 0; i < MAX_PLAYERS; i++)
    passedGoThisRound[i] = 0;

printf("\n=====\\n");
printf("ROUND %d\\n", round);
printf("=====\\n");

while(round <= MAX_ROUNDS)
{
    playerIndex = turnOrder[turnPointer];

    if(!game[0].players[playerIndex].bankrupt)
    {
        passedGo = playTurn(game, playerIndex);

        if(passedGo)
            passedGoThisRound[playerIndex] = 1;
    }

    turnPointer = (turnPointer + 1) % MAX_PLAYERS;

    /* has every player still playing passed GO this round? */
    allPassed = 1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        if(!passedGoThisRound[i])
        {
            allPassed = 0;
            break;
        }
    }

    if(!allPassed)
        continue;

    processLoans(game);
    processInsuranceExpiry(game);
    ageProperties(game);
    ageBuildings(game);
    decrementEventTimers(game);
    decrementModifiers(game);
    enforceAntiSpeculation(game);

    if(round % ECONOMY_CYCLE == 0)
    {
        triggerDisaster(game);
        applyInflation(game);
        reviewPropertyMarket(game, round);
    }

    if(round % EVENT_CYCLE == 0)
    {
        triggerEconomicEvent(game);
        drawRegionalCard(game);
    }

    if(round % REGULATION_CYCLE == 0)
    {
        triggerGovernmentRegulation(game);
    }

    displayRoundSummary(game, round);
    displayMarketConditions(game);

    if(countSolventPlayers(game) <= 1)
    {
        printf("\\nOnly one solvent player remains. Ending game.\\n");
        break;
    }

    round++;
}

```

```

    if(round <= MAX_ROUNDS)
    {
        printf("\n=====\\n");
        printf("ROUND %d\\n", round);
        printf("=====\\n");
    }

    for(i = 0; i < MAX_PLAYERS; i++)
        passedGoThisRound[i] = 0;
}

/* Whoever is still solvent wins. If several are still solvent at
round 500, the highest net worth wins (Rule 15). */
int determineWinner(GameState game[])
{
    int i;
    int winner;
    int bestNetWorth;
    int netWorth;

    winner = -1;
    bestNetWorth = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        netWorth = calculateNetWorth(game, i);

        if(winner == -1 || netWorth > bestNetWorth)
        {
            winner = i;
            bestNetWorth = netWorth;
        }
    }

    if(winner == -1)
    {
        /* edge case: everyone bankrupt at once - compare everyone anyway */
        for(i = 0; i < MAX_PLAYERS; i++)
        {
            netWorth = calculateNetWorth(game, i);

            if(winner == -1 || netWorth > bestNetWorth)
            {
                winner = i;
                bestNetWorth = netWorth;
            }
        }
    }

    return winner;
}

void displayFinalResults(GameState game[])
{
    int winner;
    int i;
    char cashBuf[32];
    char propBuf[32];
    char loanBuf[32];
    char netBuf[32];

    winner = determineWinner(game);

    printf("\n=====\\n");
    printf("GAME OVER\\n");
    printf("=====\\n");

    printf("Winner\\n%s\\n", game[0].players[winner].name);

    formatLKR(game[0].players[winner].cash, cashBuf);
    printf("Total Cash\\nLKR %s\\n", cashBuf);

    formatLKR(calculatePropertyValue(game, winner) +

```

```

        calculateBuildingValue(game, winner), propBuf);
printf("Total Property Value\nLKR %s\n", propBuf);

if(game[0].players[winner].loan.active)
{
    formatLKR(game[0].players[winner].loan.amount, loanBuf);
    printf("Outstanding Loans\nLKR %s\n", loanBuf);
}
else
{
    printf("Outstanding Loans\nNone\n");
}

formatLKR(calculateNetWorth(game, winner), netBuf);
printf("Net Worth\nLKR %s\n", netBuf);
printf("=====\n");

printf("\nFinal Standings (all players)\n");
printf("=====\n");

for(i = 0; i < MAX_PLAYERS; i++)
{
    printf("\n%s\n", game[0].players[i].name);
    printf("Cash : LKR %d\n", game[0].players[i].cash);
    printf("Net Worth : LKR %d\n", calculateNetWorth(game, i));
    printf("Properties : %d\n", game[0].players[i].propertiesOwned);
    printf("Railways : %d\n", game[0].players[i].railwaysOwned);
    printf("Utilities : %d\n", game[0].players[i].utilitiesOwned);
    if(game[0].players[i].bankrupt)
    {
        printf("Status : Bankrupt\n");
    }
    else
    {
        printf("Status : Active\n");
    }
}
}

void startGame(GameState game[])
{
    int turnOrder[MAX_PLAYERS];
    char moneyBuffer[32];

    srand((unsigned)time(NULL));

    initializeBoard(game);
    initializePlayers(game);
    initEconomy(game);
    initMarket(game);

    printf("Player 1 : %s\n", game[0].players[0].name);
    printf("Player 2 : %s\n", game[0].players[1].name);
    printf("Player 3 : %s\n", game[0].players[2].name);
    printf("Player 4 : %s\n", game[0].players[3].name);

    formatLKR(START_MONEY, moneyBuffer);
    printf("\nEach player begins with LKR %s.\n", moneyBuffer);

    determineTurnOrder(game, turnOrder);

    playGame(game, turnOrder);

    displayFinalResults(game);
}

```

A.7 finance.c - the economic core (1077 lines)

```

#include <stdio.h>
#include "types.h"
#include "functions.h"

/* Sets a square back to "unowned": no owner, no mortgage, no loan
pledge and no insurance. Used when selling, foreclosing or
liquidating a property. */
void stripOwnership(GameState game[], int index)
{
    game[0].board[index].property.owner = -1;
}

```

```

    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
}

/* Clears a player's loan completely (amount, rate and rounds). */
void resetLoan(GameState game[], int playerIndex)
{
    game[0].players[playerIndex].loan.active = 0;
    game[0].players[playerIndex].loan.amount = 0;
    game[0].players[playerIndex].loan.interestRate = 0;
    game[0].players[playerIndex].loan.remainingRounds = 0;
}

/* Adds delta to the owned-count that matches the square's type
   (property, railway or utility). Pass 1 when buying, -1 when
   selling or losing a square. */
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
{
    if(game[0].board[index].type == PROPERTY)
        game[0].players[playerIndex].propertiesOwned += delta;
    else if(game[0].board[index].type == RAILWAY)
        game[0].players[playerIndex].railwaysOwned += delta;
    else if(game[0].board[index].type == UTILITY)
        game[0].players[playerIndex].utilitiesOwned += delta;
}

/* The colour group of a square, or -1 if it isn't a property. */
PropertyGroup groupOf(GameState game[], int index)
{
    if(game[0].board[index].type == PROPERTY)
        return game[0].board[index].property.group;

    return (PropertyGroup)-1;
}

void receiveMoney(GameState game[], int playerIndex, int amount)
{
    /* bankrupt players take no further part in the game (Section 1.3) */
    if(game[0].players[playerIndex].bankrupt)
        return;

    game[0].players[playerIndex].cash += amount;
}

void payMoney(GameState game[], int playerIndex, int amount)
{
    game[0].players[playerIndex].cash -= amount;
}

/* Section 3.4: a Risk Taker facing bankruptcy first sells their
   cheapest undeveloped property to the Bank to raise cash.
   Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
   soon as a sale fails to reduce the stock of undeveloped
   properties - otherwise this loop would never end. */
while(game[0].players[playerIndex].cash < 0 &&
      !game[0].players[playerIndex].bankrupt &&
      game[0].players[playerIndex].strategy == RISK_TAKER)
{
    int before = countUndevelopedProperties(game, playerIndex);

    sellLowValueProperty(game, playerIndex);

    if(countUndevelopedProperties(game, playerIndex) >= before)
        break;
}

if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)
{
    game[0].players[playerIndex].bankrupt = 1;

    printf("\n*** BANKRUPTCY ***\n");
    printf("%s has been declared bankrupt.\n",
           game[0].players[playerIndex].name);

    liquidateBankruptAssets(game, playerIndex);
}

```

```

/* cash never stays negative - assets were liquidated, so any
   remaining debt is written off (Section 1.3) */
game[0].players[playerIndex].cash = 0;
}
}

/* Rule 14: on bankruptcy, buildings are demolished and every
   property/railway/utility owned is auctioned off (Rule-LK 19). */
void liquidateBankruptAssets(GameState game[], int playerIndex)
{
    int i;

    printf("%s's remaining assets are being liquidated.\n",
        game[0].players[playerIndex].name);

    resetLoan(game, playerIndex);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        demolishBuildingsOn(game, i);

        stripOwnership(game, i);
        game[0].board[i].property.damaged = 0;
        game[0].board[i].property.repairCostOwed = 0;

        adjustOwnedCount(game, playerIndex, i, -1);

        runAuction(game, i, -1);
    }
}

/* Income Tax (Rule-LK): 15% of the player's net worth at the base
   rate. Market conditions scale the rate (currentTaxRatePercent). */
void payTax(GameState game[], int playerIndex)
{
    int rate;
    int netWorth;
    int amount;

    rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
    netWorth = calculateNetWorth(game, playerIndex);
    amount = (netWorth * rate) / 100;

    /* a bankrupt player's net worth can be negative - a tax can never
       pay the player money, so clamp it at zero */
    if(amount < 0)
        amount = 0;

    printf("\n%s landed on Income Tax.\n",
        game[0].players[playerIndex].name);

    printf("Tax Rate : %d%%\n", rate);
    printf("Net Worth : LKR %d\n", netWorth);
    printf("%s paid tax : LKR %d\n",
        game[0].players[playerIndex].name,
        amount);

    payMoney(game, playerIndex, amount);
}

/* Community Development Fund (Rule-LK): 10% of the player's total
   assets at current market rate. Only property (land) counts -
   buildings are not included. The 10% rate is scaled by market
   conditions just like the Income Tax rate. */
void payCommunityFundTax(GameState game[], int playerIndex)
{
    int rate;
    int assets;
    int amount;

    rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);
    assets = calculatePropertyValue(game, playerIndex);

```

```

amount = (assets * rate) / 100;

printf("\n%s landed on Community Development Fund.\n",
       game[0].players[playerIndex].name);

printf("Tax Rate : %d%%\n", rate);
printf("Property Assets (market rate) : LKR %d\n", assets);
printf("%s paid community development tax : LKR %d\n",
       game[0].players[playerIndex].name,
       amount);

payMoney(game, playerIndex, amount);
}

int findPropertyToMortgage(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        /* must be undeveloped before it can be mortgaged */
        if(game[0].board[i].type == PROPERTY &&
           (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
        {
            continue;
        }

        return i;
    }

    return -1;
}

int findMortgagedProperty(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner == playerIndex &&
           game[0].board[i].property.mortgaged)
        {
            return i;
        }
    }

    return -1;
}

void mortgageProperty(GameState game[], int playerIndex)
{
    int propIndex;
    int payout;
    PropertyGroup group;

    if(!shouldMortgage(game, playerIndex))
        return;

    propIndex = findPropertyToMortgage(game, playerIndex);

    if(propIndex == -1)
        return;

    /* Rule-LK 34: a booming group's mortgage values rise by 15%, a
       declining group's fall by 10%. Railways and utilities are
       unaffected. */
    group = groupOf(game, propIndex);

```

```

    payout = (game[0].board[propIndex].property.mortgageValue *
        modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;

    game[0].board[propIndex].property.mortgaged = 1;

    receiveMoney(game, playerIndex, payout);

    printf("\n%s mortgaged %s for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name,
        payout);
}

void redeemMortgage(GameState game[], int playerIndex)
{
    int propIndex;
    int redeemCost;

    propIndex = findMortgagedProperty(game, playerIndex);

    if(propIndex == -1)
        return;

    /* pay back the mortgage value plus 10% interest */
    redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;

    if(!shouldRedeemMortgage(game, playerIndex, redeemCost))
        return;

    if(game[0].players[playerIndex].cash < redeemCost)
        return;

    payMoney(game, playerIndex, redeemCost);

    game[0].board[propIndex].property.mortgaged = 0;

    printf("\n%s redeemed the mortgage on %s for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name,
        redeemCost);
}

void handleMortgageDecisions(GameState game[], int playerIndex)
{
    redeemMortgage(game, playerIndex);
    mortgageProperty(game, playerIndex);
}

/* Rent for a normal property, based on houses/hotel (Table 6),
   scaled by any active bonuses and by building condition (Table 3). */
int calculateRent(GameState game[], int position)
{
    int rent;
    int conditionPercent;
    int houses;
    int hotel;
    int baseRent;
    PropertyGroup group;
    int condition;

    houses = game[0].board[position].property.houses;
    hotel = game[0].board[position].property.hotel;
    baseRent = game[0].board[position].property.baseRent;
    group = game[0].board[position].property.group;
    condition = game[0].board[position].property.condition;

    if(hotel)
    {
        rent = baseRent * 10;
    }
    else
    {
        if(houses == 0)
            rent = baseRent * 1;
        else if(houses == 1)
            rent = baseRent * 2;
        else if(houses == 2)

```



```

        rent = baseRent * 3;
    else if(houses == 3)
        rent = baseRent * 5;
    else
        rent = baseRent * 7;
}

/* Rule-LK 34: all matching modifiers multiply together. */
rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

if(hotel)
    rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;

if(houses > 0 || hotel)
{
    conditionPercent = rentConditionPercent(condition);
    rent = (rent * conditionPercent) / 100;
}

return rent;
}

/* Rent for a railway, based on how many the same owner has (Table 7). */
int calculateRailwayRent(GameState game[], int playerIndex)
{
    int i;
    int count;
    int baseRent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == RAILWAY &&
            game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 1)
        baseRent = RAILWAY_RENT_1;
    else if(count == 2)
        baseRent = RAILWAY_RENT_2;
    else if(count == 3)
        baseRent = RAILWAY_RENT_3;
    else if(count == 4)
        baseRent = RAILWAY_RENT_4;
    else
        baseRent = 0;

    baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;
    baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return baseRent;
}

/* Rent for a utility, based on the dice roll just made (Table 8). */
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
{
    int i;
    int count;
    int rent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == UTILITY &&
            game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 2)

```

```

        rent = UTILITY_RENT_TWO * diceValue;
    else if(count == 1)
        rent = UTILITY_RENT_ONE * diceValue;
    else
        rent = 0;

    rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
    rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return rent;
}

int groupSize(GameState game[], PropertyGroup group)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
            count++;
    }

    return count;
}

int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int owned;

    owned = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            if(game[0].board[i].property.owner == playerIndex)
                owned++;
        }
    }

    return (owned == groupSize(game, group));
}

/* Builds one house (or upgrades to a hotel) somewhere in this
colour group. Always adds to whichever property currently has the
fewest houses, so development stays even (Rule 9). Returns 1 if
something was actually built, 0 if not - constructBuildings()
uses this to keep building repeatedly in the same turn. */
int developGroup(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int minHouses;
    int targetIndex;
    int allFourHouses;

    if(!ownsMonopoly(game, playerIndex, group))
        return 0;

    minHouses = MAX_HOUSES + 1;
    targetIndex = -1;
    allFourHouses = 1;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            /* can't build on a mortgaged, damaged or loan-locked
            property - those earn no rent and/or are pledged */
            if(game[0].board[i].property.mortgaged ||
                game[0].board[i].property.damaged ||
                game[0].board[i].property.loanLocked)
            {

```

```

/* Rule 9: a blocked member that isn't fully developed
   means the group can't support a hotel yet */
    if(!game[0].board[i].property.hotel &&
        game[0].board[i].property.houses < MAX_HOUSES)
    {
        allFourHouses = 0;
    }

    continue;
}

if(game[0].board[i].property.hotel)
    continue;

if(game[0].board[i].property.houses < MAX_HOUSES)
    allFourHouses = 0;

if(game[0].board[i].property.houses < minHouses)
{
    minHouses = game[0].board[i].property.houses;
    targetIndex = i;
}
}

if(targetIndex == -1)
    return 0;    /* every property already has a hotel */

/* Rule 9: never pile a 5th house onto one property just because a
   group member is blocked from building */
if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)
    return 0;

if(allFourHouses)
{
    int hotelCost;
    char costBuf[32];

/* Rule-LK 13/31/34: base hotel cost is permanent; temporary
   events only add modifiers on top. A market boom's +10%
   applies only to the booming group, while event/regulation
   modifiers (group -1) are global. */
    hotelCost = (game[0].board[targetIndex].property.hotelCost *
        modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

    if(!shouldConstruct(game, playerIndex, hotelCost, 1))
        return 0;

    if(game[0].players[playerIndex].cash < hotelCost)
        return 0;

    payMoney(game, playerIndex, hotelCost);

    game[0].board[targetIndex].property.houses = 0;
    game[0].board[targetIndex].property.hotel = 1;

    printf("\n%s upgraded %s to a Hotel.\n",
        game[0].players[playerIndex].name,
        game[0].board[targetIndex].name);

    formatLKR(hotelCost, costBuf);
    printf("Construction Cost : LKR %s.\n", costBuf);

    return 1;
}

{
    int houseCost;
    char costBuf[32];

    houseCost = (game[0].board[targetIndex].property.houseCost *
        modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

    if(!shouldConstruct(game, playerIndex, houseCost, 0))

```

```

        return 0;

    if(game[0].players[playerIndex].cash < houseCost)
        return 0;

    payMoney(game, playerIndex, houseCost);

    game[0].board[targetIndex].property.houses++;

    printf("\n%s constructed one house on %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[targetIndex].name);

    formatLKR(houseCost, costBuf);
    printf("Construction Cost : LKR %s.\n", costBuf);

    return 1;
}
}

/* Keeps building on each colour group, one step at a time, until
either nothing more can be built there this turn (unaffordable,
the strategy doesn't want to, or the group is fully built up) -
this is what lets a strategy build "the maximum possible number
of houses immediately" in a single turn instead of one per turn. */
void constructBuildings(GameState game[], int playerIndex)
{
    PropertyGroup group;
    int builtSomething;
    int safetyLimit;

    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        return;

    for(group = BROWN; group < NO_GROUP; group++)
    {
        safetyLimit = 0;

        do
        {
            builtSomething = developGroup(game, playerIndex, group);
            safetyLimit++;
        }
        while(builtSomething && safetyLimit < 20);
    }
}

int calculatePropertyValue(GameState game[], int playerIndex)
{
    int i;
    int total;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        /* mortgaged properties contribute zero to net worth */
        if(game[0].board[i].property.mortgaged)
            continue;

        /* railways and utilities also use currentMarketValue so their
        value modifiers (rail/port/water effects) apply too */
        total += currentMarketValue(game, i);
    }

    return total;
}

int calculateBuildingValue(GameState game[], int playerIndex)
{
    int i;
    int total;

```

```

total = 0;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type != PROPERTY)
        continue;

    if(game[0].board[i].property.owner != playerIndex)
        continue;

    if(game[0].board[i].property.hotel)
        total += game[0].board[i].property.hotelCost;
    else
        total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
}

return total;
}

/* Rule 15 net worth formula: cash + property value + building value,
   minus any outstanding loan amount. */
int calculateNetWorth(GameState game[], int playerIndex)
{
    int cash;
    int propertyValue;
    int buildingValue;
    int outstandingLoans;

    cash = game[0].players[playerIndex].cash;
    propertyValue = calculatePropertyValue(game, playerIndex);
    buildingValue = calculateBuildingValue(game, playerIndex);

    outstandingLoans = 0;

    if(game[0].players[playerIndex].loan.active)
        outstandingLoans = game[0].players[playerIndex].loan.amount;

    return cash + propertyValue + buildingValue - outstandingLoans;
}

int countUndevelopedProperties(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
           game[0].board[i].property.owner == playerIndex &&
           game[0].board[i].property.houses == 0 &&
           !game[0].board[i].property.hotel)
        {
            count++;
        }
    }

    return count;
}

/* True if buying the property at pos would complete the whole colour
   group for the player (Section 3.1). */
int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
{
    int i;
    PropertyGroup group;

    if(game[0].board[pos].type != PROPERTY)
        return 0;

    if(game[0].board[pos].property.owner != -1)
        return 0;

    group = game[0].board[pos].property.group;

    for(i = 0; i < BOARD_SIZE; i++)

```

```

{
    if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
    {
        if(i != pos && game[0].board[i].property.owner != playerIndex)
            return 0;
    }
}

return 1;
}

/* Sells the player's lowest-value undeveloped property back to the
   Bank at market value (Section 3.4 bankruptcy tactic). */
void sellLowValueProperty(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be sold */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }

    if(best == -1)
        return;

    receiveMoney(game, playerIndex, bestValue);

    stripOwnership(game, best);

    adjustOwnedCount(game, playerIndex, best, -1);

    printf("\n%s sold %s to the Bank for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[best].name,
        bestValue);
}

/* Section 3.4: the Opportunistic Trader sells properties expected to
   lose value following economic events - any undeveloped property in
   a group that is currently declining, under a global downturn, or
   carrying its own negative value modifier. Loan-locked collateral
   (Rule-LK 3) is never sold. */
void sellDecliningProperties(GameState game[], int playerIndex)
{
    int i;
    int globalMult;

    if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)
        return;

    if(game[0].players[playerIndex].bankrupt)
        return;

```

```

globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

for(i = 0; i < BOARD_SIZE; i++)
{
    int groupMult;
    int indexMult;
    int value;

    if(game[0].board[i].type != PROPERTY)
        continue;

    if(game[0].board[i].property.owner != playerIndex)
        continue;

    if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
        continue;

    if(game[0].board[i].property.loanLocked)
        continue;

    groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
                                    game[0].board[i].property.group, -1);
    indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);

    if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)
        continue;

    value = currentMarketValue(game, i);

    receiveMoney(game, playerIndex, value);

    stripOwnership(game, i);

    adjustOwnedCount(game, playerIndex, i, -1);

    printf("\n%s sold %s to the Bank for LKR %d "
           "(expected to lose value).\n",
           game[0].players[playerIndex].name,
           game[0].board[i].name,
           value);
}
}

/* Releases the player's lowest-value undeveloped property and puts
it up for auction (Anti-Speculation Act enforcement). */
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be auctioned */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }
}

```

```

}

if(best == -1)
    return;

printf("\nAnti-Speculation Act : %s must sell %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[best].name);

stripOwnership(game, best);

adjustOwnedCount(game, playerIndex, best, -1);

runAuction(game, best, playerIndex);
}

/* Sells excess undeveloped properties until the player is back at
the allowed limit (or nothing more can be sold). */
void forceSellExcess(GameState game[], int playerIndex)
{
    int excess;

    excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;

    while(excess > 0)
    {
        int before = countUndevelopedProperties(game, playerIndex);

        sellUndevelopedPropertyToAuction(game, playerIndex);

/* nothing sellable left (e.g. every excess property is
loan-locked) - stop to avoid a non-terminating loop */
        if(countUndevelopedProperties(game, playerIndex) >= before)
            break;

        excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
    }
}

/* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
at most three undeveloped properties. After 5 consecutive rounds
above the limit the excess is force-auctioned. */
void enforceAntiSpeculation(GameState game[])
{
    int i;
    int undeveloped;

    if(!game[0].economy.antiSpeculationActive)
    {
        for(i = 0; i < MAX_PLAYERS; i++)
            game[0].players[i].antiSpecRounds = 0;

        return;
    }

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
        {
            game[0].players[i].antiSpecRounds = 0;
            continue;
        }

        undeveloped = countUndevelopedProperties(game, i);

        if(undeveloped > MAX_UNDEVELOPED_PROPS)
        {
            game[0].players[i].antiSpecRounds++;

            if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)
            {
                forceSellExcess(game, i);
                game[0].players[i].antiSpecRounds = 0;
            }
        }
        else

```



```

        {
            game[0].players[i].antiSpecRounds = 0;
        }
    }
}

void buyProperty(GameState game[], int playerIndex)
{
    int pos;
    int price;
    int wantsToBuy;
    char priceBuf[32];
    char balanceBuf[32];

    pos = game[0].players[playerIndex].position;

    if(game[0].board[pos].type != PROPERTY &&
        game[0].board[pos].type != RAILWAY &&
        game[0].board[pos].type != UTILITY)
    {
        return;
    }

    if(game[0].board[pos].property.owner != -1)
        return; /* already owned */

    price = currentMarketValue(game, pos);

    if(game[0].board[pos].type == PROPERTY)
    {
        /* Rule-LK 31/34: a booming group's direct purchase prices
           rise by 15% (group-scoped, not global). Railways and
           utilities are priced at their current market value, so
           their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
           affect their purchase price too. */
        price = (price *
                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                    game[0].board[pos].property.group, -1)) / 100;
    }

    wantsToBuy = shouldBuyProperty(game, playerIndex);

    if(game[0].players[playerIndex].cash < price)
        wantsToBuy = 0;

    /* Rule 5: if they don't buy directly, it goes to auction */
    if(!wantsToBuy)
    {
        runAuction(game, pos, -1);
        return;
    }

    payMoney(game, playerIndex, price);

    game[0].board[pos].property.owner = playerIndex;

    if(game[0].board[pos].type == PROPERTY)
        game[0].players[playerIndex].propertiesOwned++;
    else if(game[0].board[pos].type == RAILWAY)
        game[0].players[playerIndex].railwaysOwned++;
    else if(game[0].board[pos].type == UTILITY)
        game[0].players[playerIndex].utilitiesOwned++;

    formatLKR(price, priceBuf);
    formatLKR(game[0].players[playerIndex].cash, balanceBuf);

    printf("\n%s purchased %s for LKR %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[pos].name,
        priceBuf);

    printf("Remaining Balance : LKR %s.\n",
        balanceBuf);
}

void payRent(GameState game[], int playerIndex, int diceValue)

```

```

{
    int pos;
    int owner;
    int rent;
    char rentBuf[32];

    pos = game[0].players[playerIndex].position;

    if(game[0].board[pos].type != PROPERTY &&
        game[0].board[pos].type != RAILWAY &&
        game[0].board[pos].type != UTILITY)
    {
        return;
    }

    owner = game[0].board[pos].property.owner;

    if(owner == playerIndex)
    {
        /* landed on your own square - no rent, maybe renovate */
        if(game[0].board[pos].type == PROPERTY)
            tryRenovateAgeDepreciation(game, playerIndex, pos);
        return;
    }

    if(owner == -1)
        return;

    if(game[0].board[pos].property.mortgaged)
        return;

    if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)
    {
        printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",
            game[0].players[playerIndex].name, game[0].board[pos].name);
        return;
    }

    if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)
    {
        printf("\n%s landed on %s, but it is damaged and collects no rent.\n",
            game[0].players[playerIndex].name, game[0].board[pos].name);
        return;
    }

    if(game[0].board[pos].type == PROPERTY &&
        game[0].board[pos].property.lostIncomeRoundsLeft > 0)
    {
        printf("\n%s landed on %s, but it earns no rent right now "
            "(Business Interruption).\n",
            game[0].players[playerIndex].name, game[0].board[pos].name);
        return;
    }

    if(game[0].board[pos].type == PROPERTY)
        rent = calculateRent(game, pos);
    else if(game[0].board[pos].type == RAILWAY)
        rent = calculateRailwayRent(game, owner);
    else
        rent = calculateUtilityRent(game, owner, diceValue);

    payMoney(game, playerIndex, rent);
    receiveMoney(game, owner, rent);

    formatLKR(rent, rentBuf);

    printf("\n%s landed on %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[pos].name);

    printf("Rent Paid : LKR %s.\n", rentBuf);

    printf("Owner : %s.\n", game[0].players[owner].name);
}

```

A.8 bank.c - loans and foreclosure (395 lines)

```
#include <stdio.h>
```

```

#include "types.h"
#include "functions.h"

/* Rule-LK: each colour group has a base price that is used ONLY for
loan calculations (individual purchase prices are used for
buying). It matches the first property in the group on the board,
so this reads it straight from the board instead of storing a
second copy. Railways and utilities use their own mortgage value. */
int groupBasePrice(GameState game[], PropertyGroup group)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.group == group)
        {
            return game[0].board[i].property.purchasePrice;
        }
    }

    return 0;
}

int totalEligibleCollateral(GameState game[], int playerIndex)
{
    int i;
    int total;
    int group;
    int mult;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
            game[0].board[i].type != RAILWAY &&
            game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        /* Rule-LK 34: a booming group's collateral is worth 15% more,
a declining group's 10% less. Colour property collateral
is based on the group base price (not the individual
purchase price - that is only used for buying). */
        group = groupOf(game, i);

        mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);

        if(game[0].board[i].type == PROPERTY)
            total += (groupBasePrice(game, group) * mult) / 100;
        else
            total += (game[0].board[i].property.mortgageValue * mult) / 100;
    }

    return total;
}

/* Rule-LK 2: max loan is 75% of eligible collateral. */
int calculateMaxLoan(GameState game[], int playerIndex)
{
    return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;
}

void obtainLoan(GameState game[], int playerIndex)
{

```

```

int i;
int maxLoan;
char loanBuf[32];

if(game[0].players[playerIndex].loan.active)
    return; /* only one active loan allowed at a time */

maxLoan = calculateMaxLoan(game, playerIndex);

if(maxLoan <= 0)
    return;

game[0].players[playerIndex].loan.active = 1;
game[0].players[playerIndex].loan.amount = maxLoan;
game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;

receiveMoney(game, playerIndex, maxLoan);

printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);

formatLKR(maxLoan, loanBuf);
printf("Loan Amount : LKR %s.\n", loanBuf);
printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);

printf("Collateral :\n");

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type != PROPERTY &&
        game[0].board[i].type != RAILWAY &&
        game[0].board[i].type != UTILITY)
    {
        continue;
    }

    if(game[0].board[i].property.owner == playerIndex &&
        !game[0].board[i].property.mortgaged &&
        !game[0].board[i].property.loanLocked)
    {
        game[0].board[i].property.loanLocked = 1;
        printf("%s\n", game[0].board[i].name);
    }
}
}

void repayLoan(GameState game[], int playerIndex, int amount)
{
    int i;

    if(!game[0].players[playerIndex].loan.active)
        return;

    if(amount > game[0].players[playerIndex].loan.amount)
        amount = game[0].players[playerIndex].loan.amount;

    if(game[0].players[playerIndex].cash < amount)
        amount = game[0].players[playerIndex].cash;

    payMoney(game, playerIndex, amount);
    game[0].players[playerIndex].loan.amount -= amount;

    printf("\n%s repaid LKR %d towards their loan.\n",
        game[0].players[playerIndex].name, amount);

    printf("Outstanding Balance : LKR %d\n",
        game[0].players[playerIndex].loan.amount);

    if(game[0].players[playerIndex].loan.amount <= 0)
    {
        resetLoan(game, playerIndex);

        for(i = 0; i < BOARD_SIZE; i++)
        {
            if(game[0].board[i].property.owner == playerIndex)
                game[0].board[i].property.loanLocked = 0;
        }
    }
}

```

```

    }

    printf("%s has fully repaid the loan. Collateral released.\n",
           game[0].players[playerIndex].name);
}
}

/* Rule-LK 5: landing on the Bank offers repaying part, repaying in
   full, or (if no loan yet) obtaining one. With an existing loan a
   player can also extend the term, refinance at the current rate, or
   borrow more against newly acquired collateral. Only one
   transaction happens per visit. */
void handleBankVisit(GameState game[], int playerIndex)
{
    printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);

    if(game[0].players[playerIndex].loan.active)
    {
        if(wantsToRepayLoan(game, playerIndex))
        {
            int repayAmount;

            /* every strategy's wantsToRepayLoan() only triggers once
               cash can clear the whole loan, so always repay in full */
            repayAmount = game[0].players[playerIndex].loan.amount;

            repayLoan(game, playerIndex, repayAmount);
            return;
        }

        if(wantsIncreaseLoan(game, playerIndex))
        {
            increaseLoan(game, playerIndex);
            return;
        }

        if(wantsExtendLoan(game, playerIndex))
        {
            extendLoan(game, playerIndex);
            return;
        }

        if(wantsRefinance(game, playerIndex))
        {
            refinanceLoan(game, playerIndex);
            return;
        }

        return;
    }

    if(wantsLoan(game, playerIndex))
    {
        obtainLoan(game, playerIndex);
    }
}

/* Rule-LK 5: a player with an active loan may borrow more against
   collateral they've acquired since the loan was taken out. */
void increaseLoan(GameState game[], int playerIndex)
{
    int i;
    int availableCollateral;
    int additional;

    if(!game[0].players[playerIndex].loan.active)
        return;

    availableCollateral = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {

```

```

        continue;
    }

    if(game[0].board[i].property.owner != playerIndex)
        continue;

    if(game[0].board[i].property.mortgaged)
        continue;

    if(game[0].board[i].property.loanLocked)
        continue;

    if(game[0].board[i].type == PROPERTY)
    {
        availableCollateral +=
            (groupBasePrice(game, game[0].board[i].property.group) *
             modifierMultiplier(game, MOD_MARKET_MORTGAGE,
                                 game[0].board[i].property.group, -1)) / 100;
    }
    else
    {
        availableCollateral += game[0].board[i].property.mortgageValue;
    }
}

additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) / 100;

if(additional <= 0)
    return;

game[0].players[playerIndex].loan.amount += additional;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type != PROPERTY &&
       game[0].board[i].type != RAILWAY &&
       game[0].board[i].type != UTILITY)
    {
        continue;
    }

    if(game[0].board[i].property.owner == playerIndex &&
       !game[0].board[i].property.mortgaged &&
       !game[0].board[i].property.loanLocked)
    {
        game[0].board[i].property.loanLocked = 1;
    }
}

receiveMoney(game, playerIndex, additional);

printf("\n%s increased their loan by LKR %d.\n",
       game[0].players[playerIndex].name, additional);
printf("New Outstanding Balance : LKR %d\n",
       game[0].players[playerIndex].loan.amount);
}

/* Rule-LK 5: extend the loan term by 10 rounds. */
void extendLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;

    game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;

    printf("\n%s extended their loan by %d rounds.\n",
           game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);
    printf("New Duration : %d Rounds remaining\n",
           game[0].players[playerIndex].loan.remainingRounds);
}

/* Rule-LK 5: refinance the loan at the current economy interest rate. */
void refinanceLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;
}

```

```

game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;

printf("\n%s refinanced their loan at the current rate of %d%%.\n",
      game[0].players[playerIndex].name,
      game[0].economy.loanInterestRate);
}

void demolishBuildingsOn(GameState game[], int index)
{
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
}

/* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
are seized and auctioned. If nothing is left, the player is
declared bankrupt. */
void foreclose(GameState game[], int playerIndex)
{
    int i;
    int ownsAnythingLeft;

    printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);

    ownsAnythingLeft = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.loanLocked)
        {
            ownsAnythingLeft = 1;
            continue;
        }

        printf("%s (pledged collateral) transferred to the Bank.\n",
              game[0].board[i].name);

        demolishBuildingsOn(game, i);

        stripOwnership(game, i);

        adjustOwnedCount(game, playerIndex, i, -1);

        runAuction(game, i, -1);
    }

    resetLoan(game, playerIndex);

    printf("Outstanding debt cleared.\n");

    if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)
    {
        game[0].players[playerIndex].bankrupt = 1;
        printf("%s has no remaining assets and is declared BANKRUPT.\n",
              game[0].players[playerIndex].name);
    }
}

void processLoans(GameState game[])
{
    int i;
    int interest;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        if(!game[0].players[i].loan.active)
            continue;

        interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;
        game[0].players[i].loan.amount += interest;

        game[0].players[i].loan.remainingRounds--;
    }
}

```

```

        printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",
            game[0].players[i].name, interest, game[0].players[i].loan.amount);

        if(game[0].players[i].loan.remainingRounds <= 0)
        {
            foreclose(game, i);
        }
    }
}

```

A.9 auction.c - the auction engine (179 lines)

```

#include <stdio.h>
#include "types.h"
#include "functions.h"

/* Rule-LK 19: opening bid is 50% of market value. A market decline
lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
Railways and utilities use currentMarketValue too, so their value
modifiers (Port Expansion, Water Shortage, ...) apply the same way
they do for colour-group properties. */
int getAskingValue(GameState game[], int propIndex)
{
    int value;
    int group;

    value = currentMarketValue(game, propIndex);

    group = groupOf(game, propIndex);

    value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;

    return value;
}

/* Rule-LK 19-23: runs an auction among all solvent players. A
player who declines to bid is out permanently, so the pool of
bidders can only shrink - guaranteeing the loop finishes.
sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
proceeds to that player; sellerIndex == -1 pays the Bank. */
void runAuction(GameState game[], int propIndex, int sellerIndex)
{
    int active[MAX_PLAYERS];
    int activeCount;
    int i;
    int currentBid;
    int highBidder;
    int candidateBid;
    int safetyRounds;
    char moneyBuf[32];

    printf("\n*** AUCTION ***\n");
    printf("Property : %s\n", game[0].board[propIndex].name);

    currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;

    formatLKR(currentBid, moneyBuf);
    printf("Opening Bid : LKR %s\n", moneyBuf);

    highBidder = -1;
    activeCount = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        active[i] = !game[0].players[i].bankrupt;

        /* Anti-Speculation Act (Rule-LK 8): the forced seller must
        actually part with the property, so they never bid. If the
        seller were allowed to bid, they could win the auction back
        and merely pay the bid to themselves (net-zero cost), which
        defeats the forced sale. */
        if(i == sellerIndex)
            active[i] = 0;

        if(active[i])
            activeCount++;
    }
}

```



```

safetyRounds = 0;

while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)
{
    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!active[i])
            continue;

        if(activeCount <= 1)
            break;

        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, i, propIndex, candidateBid))
        {
            currentBid = candidateBid;
            highBidder = i;

            formatLKR(currentBid, moneyBuf);
            printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);
        }
        else
        {
            active[i] = 0;
            activeCount--;

            printf("%s withdraws.\n", game[0].players[i].name);
        }
    }

    safetyRounds++;
}

/* If exactly one bidder is still active, give them one final
   chance to bid at the next increment before awarding. */
if(activeCount == 1 && highBidder == -1)
{
    int remaining;

    remaining = -1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            remaining = i;
            break;
        }
    }

    if(remaining != -1)
    {
        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, remaining, propIndex, candidateBid))
        {
            currentBid = candidateBid;
            highBidder = remaining;

            formatLKR(currentBid, moneyBuf);
            printf("%s bids LKR %s.\n",
                game[0].players[remaining].name, moneyBuf);
        }
    }
}

if(highBidder == -1)
{
    int lastActive;

    /* One player never got a chance to bid (e.g. the first active
       player declined while only two were in) - they may take the
       property at the opening bid. */
    lastActive = -1;
}

```

```

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            lastActive = i;
            break;
        }
    }

    /* No one left to bid, or they decline the opening price */
    if(lastActive == -1 || !willingToBid(game, lastActive, propIndex, currentBid))
    {
        printf("No bids received. %s remains with the Bank.\n",
            game[0].board[propIndex].name);
        return;
    }

    highBidder = lastActive;
}

/* Always format the final winning price so the display is correct
   regardless of which code path set highBidder. */
formatLKR(currentBid, moneyBuf);

payMoney(game, highBidder, currentBid);

if(sellerIndex >= 0)
    receiveMoney(game, sellerIndex, currentBid);

game[0].board[propIndex].property.owner = highBidder;

adjustOwnedCount(game, highBidder, propIndex, 1);

printf("%s wins the auction for LKR %s.\n",
    game[0].players[highBidder].name, moneyBuf);
}

```

A.10 events.c - cards and regulations (403 lines)

```

#include <stdio.h>
#include <stdlib.h>
#include "types.h"
#include "functions.h"

/* Keeps the loan interest rate inside its allowed range. */
void clampLoanInterest(GameState game[])
{
    if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;

    if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;
}

/* Moves the loan interest rate up or down by deltaPercent. */
void adjustLoanInterest(GameState game[], int deltaPercent)
{
    game[0].economy.loanInterestRate += deltaPercent;
    clampLoanInterest(game);
}

/* Appendix A: 20 National Event Cards, drawn when a player lands on
   an EVENT square. currentCardIndex cycles 0-19 and wraps around,
   which behaves the same as "draw the top card, put it at the
   bottom" without needing a real deck data structure. */
void executeEvent(GameState game[], int playerIndex)
{
    (void)playerIndex; /* not needed - these effects are global, not per-player */

    printf("\n*** NATIONAL EVENT CARD ***\n");

    switch(game[0].economy.currentCardIndex)
    {
        case 0:
            printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");
            addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);
            break;
    }
}

```

```

case 1:
    printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");
    addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);
    break;

case 2:
    printf("Heavy Floods : a random coastal property is damaged.\n");
    triggerDisaster(game);
    break;

case 3:
{
    int candidates[BOARD_SIZE];
    int count;
    int i;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)
        {
            candidates[count] = i;
            count++;
        }
    }

    if(count > 0)
    {
        game[0].economy.closedPropertyIndex = candidates[rand() % count];
        game[0].economy.closedPropertyRoundsLeft = 2;

        printf("Political Rally : %s is closed for 2 rounds.\n",
            game[0].board[game[0].economy.closedPropertyIndex].name);
    }
    else
    {
        printf("Political Rally : no owned property to close.\n");
    }
    break;
}

case 4:
    printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
    break;

case 5:
    printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
    break;

case 6:
    printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);
    break;

case 7:
    adjustLoanInterest(game, -LOAN_INTEREST_STEP);

    printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 8:
    adjustLoanInterest(game, LOAN_INTEREST_STEP);

    printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 9:
{
    int i;

    printf("Tax Amnesty : Each player receives LKR 2,000.\n");
}

```

```

        for(i = 0; i < MAX_PLAYERS; i++)
            receiveMoney(game, i, 2000);

        break;
    }

    case 10:
        printf("Power Failure : Utility income halved for 3 rounds.\n");
        addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);
        break;

    case 11:
        printf("Foreign Funding : Commercial property values increase by 15%% for 15
rounds.\n");
        addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);
        break;

    case 12:
        printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");
        addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);
        break;

    case 13:
        printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");
        addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);
        break;

    case 14:
        printf("Labour Strike : Construction suspended for 2 rounds.\n");
        game[0].economy.constructionSuspendedRoundsLeft = 2;
        break;

    case 15:
        printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");
        addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);
        break;

    case 16:
    {
        PropertyGroup group;
        char groupBuffer[32];

        group = (PropertyGroup)(rand() % NO_GROUP);
        groupName(group, groupBuffer);

        printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",
            groupBuffer);
        addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);
        break;
    }

    case 17:
        printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");
        addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);
        break;

    case 18:
    {
        int lucky;

        /* Section 1.3: bankrupt players take no further part in the
           game, so a grant must go to a solvent player. Pick a
           random starting point and scan forward to the first
           solvent player - at least one must exist, because a live
           player drew this card. */
        lucky = rand() % MAX_PLAYERS;

        while(game[0].players[lucky].bankrupt)
            lucky = (lucky + 1) % MAX_PLAYERS;

        printf("Government Grant : %s receives LKR 5,000.\n",
            game[0].players[lucky].name);

        receiveMoney(game, lucky, 5000);
        break;
    }
}

```

```

    case 19:
        printf("National Disaster : a random developed property is damaged.\n");
        triggerDisaster(game);
        break;

    default:
        break;
}

game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;
}

/* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
void triggerEconomicEvent(GameState game[])
{
    int choice;

    choice = rand() % ECONOMIC_EVENT_COUNT;

    printf("\n=== Economic Event ===\n");

    switch(choice)
    {
        case 0:
            printf("Tourism Boom\n");
            printf("Hotels receive double rent for 15 rounds.\n");
            printf("Southern Province properties increase in value by 15%.\n");

            addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);
            addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);
            break;

        case 1:
            printf("Fuel Crisis\n");
            printf("Railway rent doubles for 15 rounds.\n");
            printf("Property development costs increase 20% for 15 rounds.\n");

            addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);
            addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);
            break;

        case 2:
            printf("Heavy Monsoon\n");
            printf("Insurance premiums increase for 15 rounds.\n");
            printf("Coastal properties lose 10% value.\n");

            addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);
            addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);
            addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);
            break;

        case 3:
            printf("Economic Recession\n");
            printf("Property values decrease 15%. Rent decreases 10%.\n");
            printf("Loan interest increases by 15%.\n");

            addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
            addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);
            addModifier(game, MOD_RECESSION, -1, -1, 100, 15);

            game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);
            clampLoanInterest(game);
            break;

        case 4:
            printf("Stock Market Boom\n");
            printf("Property values increase 10%. Loan interest decreases 10%.\n");

            addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);

            game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);
            clampLoanInterest(game);
            break;

        case 5:
            printf("Government Housing Programme\n");
            printf("House construction costs reduce 25%.\n");

```

```

        addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);
        break;

    case 6:
        printf("Foreign Investment\n");
        printf("Commercial properties increase 20%%.\n");

        addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);
        break;

    case 7:
        printf("Political Unrest\n");
        printf("Hotel rent drops by 50%% for 15 rounds.\n");

        addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);
        addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);
        addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);
        break;

    default:
        break;
}
}

/* Section 2.7: one of 8 regulations, every 20 rounds. */
void triggerGovernmentRegulation(GameState game[])
{
    int choice;

    choice = rand() % REGULATION_COUNT;

    printf("\n=== Government Regulation ===\n");

    switch(choice)
    {
        case 0:
            game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);

            if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)
                game[0].economy.incomeTaxRate = TAX_RATE_MAX;

            printf("Increase Property Tax\n");
            printf("Income Tax rate increased by 50%%. Now %d%%.\n", game[0].economy.incomeTaxRate);
            break;

        case 1:
            adjustLoanInterest(game, -LOAN_INTEREST_STEP);

            printf("Reduce Loan Interest\n");
            printf("Interest decreased by %d%%. Now %d%%.\n",
                LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
            break;

        case 2:
            printf("Housing Subsidy\n");
            printf("House construction costs reduced by 30%% for 20 rounds.\n");

            addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);
            break;

        case 3:
        {
            int i;
            int tax;

            printf("Luxury Property Tax\n");
            printf("Hotels pay a maintenance tax of 25%% of their value.\n");

            for(i = 0; i < BOARD_SIZE; i++)
            {
                if(game[0].board[i].type == PROPERTY && game[0].board[i].property.hotel &&
                    game[0].board[i].property.owner != -1)
                {
                    tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;

                    printf("%s pays LKR %d luxury tax on %s.\n",
                        game[0].players[game[0].board[i].property.owner].name,

```

```

        tax, game[0].board[i].name);

    payMoney(game, game[0].board[i].property.owner, tax);
}
}
break;
}

case 4:
    printf("Railway Modernization\n");
    printf("Railway rents increase 25%% for 20 rounds.\n");

    addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);
    break;

case 5:
    printf("Electricity Tariff Revision\n");
    printf("Utility rents increase 20%% for 20 rounds.\n");

    addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);
    break;

case 6:
    printf("Insurance Regulation\n");
    printf("Insurance premiums decrease 15%% for 20 rounds. Coverage unchanged.\n");

    addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);
    break;

case 7:
    printf("Anti-Speculation Act\n");
    printf("Players may now own at most three undeveloped properties.\n");

    game[0].economy.antiSpeculationActive = 1;
    break;

default:
    break;
}
}

/* Decrements the non-modifier event timers - the modifier list itself
   is ticked down separately by decrementModifiers(). */
void decrementEventTimers(GameState game[])
{
    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        game[0].economy.constructionSuspendedRoundsLeft--;

    if(game[0].economy.closedPropertyRoundsLeft > 0)
    {
        game[0].economy.closedPropertyRoundsLeft--;
        if(game[0].economy.closedPropertyRoundsLeft == 0)
            game[0].economy.closedPropertyIndex = -1;
    }
}

```

A.11 economy.c - inflation and modifiers (504 lines)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initEconomy(GameState game[])
{
    game[0].economy.inflationRate = 0;
    game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;

    game[0].economy.constructionSuspendedRoundsLeft = 0;

    game[0].economy.closedPropertyIndex = -1;
    game[0].economy.closedPropertyRoundsLeft = 0;

    game[0].economy.incomeTaxRate = INCOME_TAX_RATE;

    game[0].economy.antiSpeculationActive = 0;
}

```

```

    game[0].economy.currentCardIndex = 0;

    game[0].economy.modifierCount = 0;
}

/* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */
int applyRate(int oldValue, int ratePercent)
{
    int newValue;

    newValue = oldValue + (oldValue * ratePercent) / 100;

    if(newValue < 1)
        newValue = 1;

    return newValue;
}

/* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not
   relevant. Effects accumulate: two active +25% rents give
   roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
void addSourcedModifier(GameState game[], ModifierType type, int group,
                        int index, int percent, int roundsLeft,
                        ModifierSource source)
{
    if(game[0].economy.modifierCount >= MAX_MODIFIERS)
        return;

    game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
    game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
    game[0].economy.modifiers[game[0].economy.modifierCount].index = index;
    game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent;
    game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft;
    game[0].economy.modifiers[game[0].economy.modifierCount].source = source;

    game[0].economy.modifierCount++;
}

/* Events, national cards and regulations use this - they are all
   generic effects, not Market Boom/Decline or Regional cards. */
void addModifier(GameState game[], ModifierType type, int group, int index,
                 int percent, int roundsLeft)
{
    addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);
}

/* Effective percentage multiplier from every active matching modifier,
   multiplied together (Rule-LK 34). Returns 100 when none match.
   A modifier with group == -1 is global and applies to every group,
   so it matches regardless of the requested group (and likewise for
   index == -1). */
int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
{
    int mult;
    int i;

    mult = 100;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != -1 &&
           game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != -1 &&
           game[0].economy.modifiers[i].index != index)
            continue;

        mult = (mult * game[0].economy.modifiers[i].percent) / 100;
    }

    return mult;
}

/* Whether at least one matching modifier is currently active - used
   for flag-style modifiers such as flood risk or recession. */

```



```

int isModifierActive(GameState game[], ModifierType type, int group, int index)
{
    int i;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != index)
            continue;

        return 1;
    }

    return 0;
}

/* The effective tax rate used by the Income Tax and Community
   Development Fund squares. Starts at the given base rate (15% /
   10%) and is scaled by the currently active global value
   modifiers, so market fluctuations move the rate up or down. */
int currentTaxRatePercent(GameState game[], int baseRate)
{
    return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;
}

/* Ticks every active modifier down and removes the expired ones. */
void decrementModifiers(GameState game[])
{
    int i;

    i = 0;

    while(i < game[0].economy.modifierCount)
    {
        game[0].economy.modifiers[i].roundsLeft--;

        if(game[0].economy.modifiers[i].roundsLeft <= 0)
        {
            int j;

            for(j = i; j < game[0].economy.modifierCount - 1; j++)
                game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];

            game[0].economy.modifierCount--;
        }
        else
        {
            i++;
        }
    }
}

/* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */
void formatLKR(int amount, char out[])
{
    char digits[32];
    int len;
    int i;
    int w;
    int neg;

    neg = 0;

    if(amount < 0)
    {
        neg = 1;
        amount = -amount;
    }

    sprintf(digits, "%d", amount);
    len = strlen(digits);

    w = 0;

```

```

    if(neg)
        out[w++] = '-';

    for(i = 0; i < len; i++)
    {
        if(i > 0 && (len - i) % 3 == 0)
            out[w++] = ',';

        out[w++] = digits[i];
    }

    out[w] = '\0';
}

/* Every 10 rounds, a random inflation rate is applied to every
property's price, rent, and construction costs. */
void applyInflation(GameState game[])
{
    int possibleRates[6] = {-3, 0, 2, 5, 8, 12};
    int rate;
    int i;

    rate = possibleRates[rand() % 6];
    game[0].economy.inflationRate = rate;

    printf("\n=== Inflation Update ===\n");
    printf("New Inflation Rate : %d%%\n", rate);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
            game[0].board[i].type != RAILWAY &&
            game[0].board[i].type != UTILITY)
        {
            continue;
        }

        game[0].board[i].property.purchasePrice =
            applyRate(game[0].board[i].property.purchasePrice, rate);

        game[0].board[i].property.mortgageValue =
            applyRate(game[0].board[i].property.mortgageValue, rate);

        if(game[0].board[i].type == PROPERTY)
        {
            game[0].board[i].property.baseRent =
                applyRate(game[0].board[i].property.baseRent, rate);

            game[0].board[i].property.houseCost =
                applyRate(game[0].board[i].property.houseCost, rate);

            game[0].board[i].property.hotelCost =
                applyRate(game[0].board[i].property.hotelCost, rate);
        }
    }

    game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, rate);

    printf("New Loan Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
}

/* A property's real current worth: purchase price, minus age
depreciation, adjusted by any active market conditions. Reused
everywhere a "real price" is needed - buying, insurance, repairs,
renovation, net worth - so all of them stay consistent. */
/* Rule-LK 11-13, 24: the current market value of a property, after
age depreciation and every value modifier currently active
(Rule-LK 34 - all matching modifiers multiply together). */
int currentMarketValue(GameState game[], int propIndex)
{
    int price;
    int depreciation;
    int mult;

    price = game[0].board[propIndex].property.purchasePrice;
    depreciation = game[0].board[propIndex].property.depreciation;

```

```

price = price - (price * depreciation) / 100;

mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

if(game[0].board[propIndex].type == PROPERTY)
    mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,
                                      game[0].board[propIndex].property.group, -1)) / 100;

if(game[0].board[propIndex].type == RAILWAY)
    mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100;

mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100;

price = (price * mult) / 100;

return price;
}

/* Rule-LK 15, 16: properties get older every round, and lose value
past age 50. Prints the required "Property Depreciation" message
(Section 5) whenever a property's depreciation actually changes. */
void ageProperties(GameState game[])
{
    int i;
    int roundsOver50;
    int newDepreciation;
    int oldDepreciation;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        game[0].board[i].property.age++;

        if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)
        {
            oldDepreciation = game[0].board[i].property.depreciation;

            roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
            newDepreciation = roundsOver50 / DEPRECIATION_STEP;

            if(newDepreciation > MAX_DEPRECIATION)
                newDepreciation = MAX_DEPRECIATION;

            game[0].board[i].property.depreciation = newDepreciation;

            if(newDepreciation != oldDepreciation)
            {
                printf("\nProperty Depreciation\n");
                printf("Property\n%s\n", game[0].board[i].name);
                printf("has depreciated by %d%%\n", newDepreciation);
                printf("Current Value\nLKR %d\n", currentMarketValue(game, i));
            }
        }
    }
}

/* Rule-LK 17: landing on your own aged property offers a renovation. */
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
{
    int cost;

    if(game[0].board[propIndex].property.depreciation <= 0)
        return;

    if(!shouldRenovateAgeDepreciation(game, playerIndex,
                                      game[0].board[propIndex].property.depreciation))
    {
        return;
    }

    cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;

    if(game[0].players[playerIndex].cash < cost)
        return;
}

```

```

payMoney(game, playerIndex, cost);

game[0].board[propIndex].property.age = 0;
game[0].board[propIndex].property.depreciation = 0;

game[0].board[propIndex].property.baseRent =
    applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST);

printf("\n%s renovated %s.\n",
    game[0].players[playerIndex].name, game[0].board[propIndex].name);

printf("Renovation Cost : LKR %d\n", cost);
}

/* Table 3: rent collected as a percentage of building condition. */
int rentConditionPercent(int condition)
{
    if(condition >= 90)
        return 100;
    if(condition >= 75)
        return 90;
    if(condition >= 50)
        return 75;
    if(condition >= 25)
        return 50;

    return 0; /* below 25% - building closed */
}

/* Rule-LK 25, 28: building condition wears down every round, and
20+ rounds of neglect causes lasting structural damage. */
void ageBuildings(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        /* Business Interruption's 5-round lost-income countdown runs
even if the property isn't currently developed with houses
(it still has a hotel that just isn't earning anything). */
        if(game[0].board[i].property.lostIncomeRoundsLeft > 0)
            game[0].board[i].property.lostIncomeRoundsLeft--;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
            continue;

        game[0].board[i].property.condition -= 2;

        if(game[0].board[i].property.condition < 0)
            game[0].board[i].property.condition = 0;

        game[0].board[i].property.roundsSinceMaintenance++;

        if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&
!game[0].board[i].property.structurallyDamaged)
        {
            game[0].board[i].property.structurallyDamaged = 1;

            game[0].board[i].property.preDamagePurchasePrice =
                game[0].board[i].property.purchasePrice;
            game[0].board[i].property.preDamageBaseRent =
                game[0].board[i].property.baseRent;

            game[0].board[i].property.purchasePrice =
                game[0].board[i].property.purchasePrice -
                (game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;

            game[0].board[i].property.baseRent =
                game[0].board[i].property.baseRent -
                (game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;

            game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;
            game[0].board[i].property.condition = 0;

```

```

        printf("\n%s has suffered structural damage from neglect!\n",
               game[0].board[i].name);
    }
}

/* Rule-LK 27: restores a building's condition to 100% for a fee. */
void performMaintenance(GameState game[], int playerIndex)
{
    int i;
    int baseCost;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.structurallyDamaged)
            continue; /* needs a full renovation instead */

        if(game[0].board[i].property.condition >= 100)
            continue;

        if(game[0].board[i].property.hotel)
            baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
        else
            baseCost = (game[0].board[i].property.houseCost * 5) / 100;

        cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;

        if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))
            continue;

        if(game[0].players[playerIndex].cash < cost)
            continue;

        payMoney(game, playerIndex, cost);

        game[0].board[i].property.condition = 100;
        game[0].board[i].property.roundsSinceMaintenance = 0;

        printf("\n%s performed maintenance on %s.\n",
               game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Maintenance Cost : LKR %d\n", cost);
    }
}

/* Rule-LK 29: fully restores a structurally damaged building. */
void renovateStructuralDamage(GameState game[], int playerIndex)
{
    int i;
    int replacementValue;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.structurallyDamaged)
            continue;

        if(game[0].board[i].property.hotel)
            replacementValue = game[0].board[i].property.hotelCost;
        else
            replacementValue = game[0].board[i].property.houseCost;
    }
}

```

```

    cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;

    if(game[0].players[playerIndex].cash < cost)
        continue;

    payMoney(game, playerIndex, cost);

    game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;
    game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;
    game[0].board[i].property.condition = 100;
    game[0].board[i].property.structurallyDamaged = 0;
    game[0].board[i].property.maintenanceCostMultiplierPercent = 100;
    game[0].board[i].property.roundsSinceMaintenance = 0;

    printf("\n%s fully renovated %s after structural damage.\n",
        game[0].players[playerIndex].name, game[0].board[i].name);

    printf("Renovation Cost : LKR %d\n", cost);
}
}

```

A.12 insurance.c - policies and disasters (393 lines)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

int propertyValue(GameState game[], int propIndex)
{
    return currentMarketValue(game, propIndex);
}

/* Assumption: repair costs 30% of the property's value. The
assignment doesn't give an exact figure, so this is used
consistently everywhere repair cost is needed. */
int repairCost(GameState game[], int propIndex)
{
    return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;
}

int isCovered(InsuranceType policy, DisasterType disaster)
{
    if(policy == NO_INSURANCE)
        return 0;

    if(policy == BASIC_INSURANCE)
        return (disaster == FIRE || disaster == FLOOD);

    return 1; /* Comprehensive and Business Interruption cover everything */
}

int compensationPercent(InsuranceType policy)
{
    if(policy == BASIC_INSURANCE)
        return 80;

    if(policy == COMPREHENSIVE_INSURANCE || policy == BUSINESS_INTERRUPTION)
        return 100;

    return 0;
}

/* Picks a disaster. Every type is equally likely at base, but Heavy
Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
DisasterType pickDisaster(GameState game[])
{
    int weights[5];
    int total;
    int i;
    int roll;
    DisasterType result;

    weights[FIRE] = DISASTER_WEIGHT;
    weights[FLOOD] = DISASTER_WEIGHT;
    weights[RIOT] = DISASTER_WEIGHT;
    weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;

```

```

weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;

if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))
    weights[FLOOD] += DISASTER_BOOST;

if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))
    weights[RIOT] += DISASTER_BOOST;

total = 0;

for(i = 0; i < 5; i++)
    total += weights[i];

roll = rand() % total;

result = FIRE;

for(i = 0; i < 5; i++)
{
    if(roll < weights[i])
    {
        result = (DisasterType)i;
        break;
    }

    roll -= weights[i];
}

return result;
}

void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
{
    int value;
    int premium;
    int premiumPercent;
    char premiumBuf[32];

    if(type == BASIC_INSURANCE)
        premiumPercent = PREMIUM_BASIC_PERCENT;
    else if(type == COMPREHENSIVE_INSURANCE)
        premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
    else if(type == BUSINESS_INTERRUPTION)
        premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
    else
        return;

    value = propertyValue(game, propIndex);
    premium = (value * premiumPercent) / 100;

    /* Rule-LK 34: premiums respond to insurance modifiers. */
    premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;

    if(game[0].players[playerIndex].cash < premium)
        return;

    payMoney(game, playerIndex, premium);

    game[0].board[propIndex].property.insurance = type;
    game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;

    printf("\n%s purchased insurance for %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name);

    formatLKR(premium, premiumBuf);
    printf("Premium : LKR %s.\n", premiumBuf);
}

/* Any owned property can be insured (Section 2.8) - no longer just
developed ones. */
int findPropertyToInsure(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {

```

```

        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance != NO_INSURANCE)
            continue;

        if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)
            return i;
    }

    return -1;
}

/* Finds a policy close to expiring so it can be renewed. */
int findPropertyToRenew(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)
            continue;

        if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)
            return i;
    }

    return -1;
}

void handleInsuranceVisit(GameState game[], int playerIndex)
{
    int propIndex;
    InsuranceType type;

    printf("\n%s landed on an Insurance square.\n",
        game[0].players[playerIndex].name);

    propIndex = findPropertyToRenew(game, playerIndex);

    if(propIndex != -1)
    {
        type = game[0].board[propIndex].property.insurance;

        purchaseInsurance(game, playerIndex, propIndex, type);

        printf("The policy on %s was renewed for another %d rounds.\n",
            game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
        return;
    }

    propIndex = findPropertyToInsure(game, playerIndex);

    if(propIndex == -1)
    {
        printf("No suitable property to insure right now.\n");
        return;
    }

    type = desiredInsurance(game, playerIndex, propIndex);

    purchaseInsurance(game, playerIndex, propIndex, type);
}

/* Rule-LK 11: fixes a disaster-damaged building automatically once
the owner can afford it. Called at the start of every turn. */
void tryAutoRepair(GameState game[], int playerIndex)
{

```



```

int i;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type != PROPERTY)
        continue;

    if(game[0].board[i].property.owner != playerIndex)
        continue;

    if(!game[0].board[i].property.damaged)
        continue;

    if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)
    {
        int cost;
        char costBuf[32];

        cost = game[0].board[i].property.repairCostOwed;

        formatLKR(cost, costBuf);

        payMoney(game, playerIndex, cost);

        printf("\n%s repaired %s. It can collect rent again.\n",
            game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Repair Cost : LKR %s.\n", costBuf);

        game[0].board[i].property.damaged = 0;
        game[0].board[i].property.repairCostOwed = 0;
    }
}

/* Writes the name of a disaster into out. */
void disasterName(DisasterType disaster, char out[])
{
    if(disaster == FIRE)
        strcpy(out, "Fire");
    else if(disaster == FLOOD)
        strcpy(out, "Flood");
    else if(disaster == RIOT)
        strcpy(out, "Riot");
    else if(disaster == BUILDING_COLLAPSE)
        strcpy(out, "Building Collapse");
    else
        strcpy(out, "Electrical Failure");
}

/* Pays an insured player's compensation for a disaster and returns
how much they received. */
int payCompensation(GameState game[], int chosen, int owner, int cost)
{
    int compensation;

    compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;

    /* Political Unrest inflates Business Interruption claims (Rule-LK 34). */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&
        isModifierActive(game, MOD_BI_CLAIMS, -1, -1))
    {
        compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;
    }

    receiveMoney(game, owner, compensation);

    printf("Insurance Claim Approved.\n");
    printf("Compensation Paid : LKR %d\n", compensation);

    /* Business Interruption also covers 5 rounds of lost rental
income - the repair cost is fully paid above, but the
property still earns nothing for 5 rounds. */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)
    {
        game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;
    }
}

```

```

        printf("%s will earn no rent for the next %d rounds "
               "(Business Interruption).\n", game[0].board[chosen].name,
               BI_LOST_INCOME_ROUNDS);
    }

    return compensation;
}

/* Rule-LK 10: every 10 rounds, one random developed property may
be hit by a disaster. */
void triggerDisaster(GameState game[])
{
    int developed[BOARD_SIZE];
    int developedCount;
    int i;
    int chosen;
    DisasterType disaster;
    int owner;
    int cost;
    int compensation;
    char name[32];

    developedCount = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
           (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
        {
            developed[developedCount] = i;
            developedCount++;
        }
    }

    if(developedCount == 0)
        return;

    chosen = developed[rand() % developedCount];

    disaster = pickDisaster(game);

    owner = game[0].board[chosen].property.owner;

    printf("\n*** DISASTER ***\n");

    disasterName(disaster, name);
    printf("%s occurred.\n", name);

    printf("Affected Property : %s (owner : %s)\n",
           game[0].board[chosen].name, game[0].players[owner].name);

    cost = repairCost(game, chosen);

    if(isCovered(game[0].board[chosen].property.insurance, disaster))
    {
        compensation = payCompensation(game, chosen, owner, cost);
    }
    else
    {
        compensation = 0;
        game[0].players[owner].sufferedLoss = 1;

        printf("Property was NOT insured against this disaster.\n");
    }

    game[0].board[chosen].property.repairCostOwed = cost - compensation;

    if(game[0].board[chosen].property.repairCostOwed < 0)
        game[0].board[chosen].property.repairCostOwed = 0;

    if(game[0].board[chosen].property.repairCostOwed > 0)
    {
        game[0].board[chosen].property.damaged = 1;
        printf("%s is damaged and stops earning rent until repaired.\n",
               game[0].board[chosen].name);
    }
}

```

```

void processInsuranceExpiry(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)
            continue;

        game[0].board[i].property.insuranceRoundsLeft--;

        if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)
        {
            printf("\nInsurance policy on %s expires in %d rounds.\n",
                game[0].board[i].name, INSURANCE_EXPIRY_WARNING);
        }

        if(game[0].board[i].property.insuranceRoundsLeft <= 0)
        {
            printf("\nInsurance policy on %s has expired.\n",
                game[0].board[i].name);

            game[0].board[i].property.insurance = NO_INSURANCE;
        }
    }
}

```

A.13 market.c - market review and regional cards (431 lines)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initMarket(GameState game[])
{
    int i;

    for(i = 0; i < NO_GROUP; i++)
        game[0].economy.groupCooldownUntilRound[i] = 0;

    game[0].economy.lastBoomGroup = -1;
    game[0].economy.lastDeclineGroup = -1;
}

void groupName(PropertyGroup group, char buffer[])
{
    if(group == BROWN)
        strcpy(buffer, "Brown");
    else if(group == LIGHT_BLUE)
        strcpy(buffer, "Light Blue");
    else if(group == PINK)
        strcpy(buffer, "Pink");
    else if(group == ORANGE)
        strcpy(buffer, "Orange");
    else if(group == RED)
        strcpy(buffer, "Red");
    else if(group == YELLOW)
        strcpy(buffer, "Yellow");
    else if(group == GREEN)
        strcpy(buffer, "Green");
    else if(group == DARK_BLUE)
        strcpy(buffer, "Dark Blue");
    else
        strcpy(buffer, "Unknown");
}

/* Picks a random group that isn't on cooldown and isn't one of the
   two groups to avoid (Rule-LK 30, 33). */
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
{
    int candidates[NO_GROUP];
    int count;
    int g;

```

```

count = 0;

for(g = 0; g < NO_GROUP; g++)
{
    if(g == avoid1 || g == avoid2)
        continue;

    if(game[0].economy.groupCooldownUntilRound[g] > currentRound)
        continue;

    candidates[count] = g;
    count++;
}

if(count == 0)
    return (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */

return (PropertyGroup)candidates[rand() % count];
}

/* Rule-LK 30-34: every 10 rounds, one group booms and a different
one declines, each for 10 rounds. All effects are temporary
modifiers (Rule-LK 34) so no base value or cost is ever changed. */
void reviewPropertyMarket(GameState game[], int currentRound)
{
    PropertyGroup boomGroup;
    PropertyGroup declineGroup;
    char boomName[12];
    char declineName[12];

    boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);
    declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup,
boomGroup);

    groupName(boomGroup, boomName);
    groupName(declineGroup, declineName);

    printf("\n=== Property Market Review ===\n");
    printf("Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n", boomName);
    printf("Market Decline : %s (values -15%%, rent -20%%) for 10 rounds.\n", declineName);

    addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);

    /* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,
direct purchase prices rise by 15% and construction costs rise
by 10%; the decline group's mortgage values fall by 10% and
auction starting prices fall by 25%. All are scoped to the
affected group. */
    addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);

    game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
    game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;

    game[0].economy.lastBoomGroup = boomGroup;
    game[0].economy.lastDeclineGroup = declineGroup;
}

/* Table 4: one card every 15 rounds, active for 15 rounds. */
void drawRegionalCard(GameState game[])
{
    int choice;

    choice = rand() % REGIONAL_CARD_COUNT;

    printf("\n=== Regional Development Card ===\n");

    switch(choice)

```

```

{
    case 0:
        printf("Southern Tourism Boom : Galle Fort, Unawatuna and "
            "Hikkaduwa rental income +40%%.\n");
        addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);
        break;

    case 1:
        printf("Port City Expansion : Pettah and Maradana values +25%%.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL);
/* Colombo
Fort Station */
        break;

    case 2:
        printf("IT Industry Growth : Maharagama, Nugegoda and "
            "Kottawa values +20%%.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);
        break;

    case 3:
        printf("Northern Development Programme : Jaffna Town, Nallur "
            "and Trincomalee values +30%%.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);
        break;

    case 4:
        printf("Tea Export Boom : Nuwara Eliya value +35%%.\n");
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);
        break;

    case 5:
        printf("Airport Expansion : Negombo, Katunayake and "
            "Ja-Ela rents +30%%.\n");
        addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);
        break;

    case 6:
        printf("University City Growth : Peradeniya and "
            "Kandy City values +20%%.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);
        break;

    case 7:
        printf("Beach Pollution : Southern coastal rents -30%%.\n");
        addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);
        break;

    case 8:
        printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);
        break;

    case 9:
        printf("Transport Strike : Railway revenue reduced by 40%%.\n");
        addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);
        break;

    case 10:
        printf("Electricity Tariff Increase : Utility rent +25%%.\n");
        addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);
        break;

    case 11:
        printf("Water Shortage : Utility revenue +20%%.\n");
        addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);
        break;

    default:
        break;
}
}

/* Prints a section title followed by a separator line. */

```

```

void printSectionHeader(char title[])
{
    printf("%s\n", title);
    printf("-----\n");
}

/* Prints a Market Boom or Market Decline block. */
void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
{
    printSectionHeader(title);
    printf("%s (%+d%)\n", groupNameBuffer, percent - 100);
    printf("Rounds Remaining : %d\n", roundsLeft);
}

/* Writes one Regional Development card line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[48];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        if(mod.group >= 0 && mod.group < NO_GROUP)
            groupName((PropertyGroup)mod.group, name);
        else
            strcpy(name, "Unknown");

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s (%+d%)", name, mod.percent - 100);
        else
            sprintf(out, "%s rents (%+d%)", name, mod.percent - 100);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s (%+d%)",
            game[0].board[mod.index].name, mod.percent - 100);
    }
    else if(mod.type == MOD_RAIL_RENT)
    {
        sprintf(out, "Railway rents (%+d%)", mod.percent - 100);
    }
    else if(mod.type == MOD_UTIL_RENT)
    {
        sprintf(out, "Utility rents (%+d%)", mod.percent - 100);
    }
    else
    {
        return 0;
    }

    return 1;
}

/* Prints the "Regional Development" section. Returns 1 if at least
one card effect was shown. */
int printRegionalSection(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_REGIONAL)
            continue;

        if(!describeRegionalCondition(game, mod, line))
            continue;

        if(!shown)
        {

```

```

        printSectionHeader("Regional Development");
        shown = 1;
    }

    printf("%s\n", line);
    printf("Rounds Remaining : %d\n", mod.roundsLeft);
}

return shown;
}

/* Writes one "Other Active Conditions" line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[32];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        groupName((PropertyGroup)mod.group, name);

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s : values x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
        else
            sprintf(out, "%s : rents x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s : value x%d%% (%d rounds remaining)",
                game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_RAIL_VALUE)
    {
        sprintf(out, "Railway values : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_RAIL_RENT)
    {
        sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_UTIL_RENT)
    {
        sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else
    {
        return 0;
    }

    return 1;
}

/* Prints the "Other Active Conditions" section (events and
government regulations still running). */
void printOtherConditions(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_GENERAL)
            continue;

        if(!describeOtherCondition(game, mod, line))
            continue;
    }
}

```

```

    if(!shown)
    {
        printSectionHeader("Other Active Conditions");
        shown = 1;
    }

    printf("%s\n", line);
}

}

/* Rule-LK 36: shows the currently active market conditions at the
end of every round, in the Section 5 format. */
void displayMarketConditions(GameState game[])
{
    int i;
    int shown;

    printf("\n=====n");
    printf("Current Market Conditions\n");
    printf("=====n");

    shown = 0;

    /* Market Boom (Rule-LK 31) */
    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        char name[32];

        if(game[0].economy.modifiers[i].source != SRC_BOOM ||
            game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
            continue;

        groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
        printBoomDecline("Market Boom", name,
                        game[0].economy.modifiers[i].percent,
                        game[0].economy.modifiers[i].roundsLeft);

        shown = 1;
        break;
    }

    /* Market Decline (Rule-LK 32) */
    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        char name[32];

        if(game[0].economy.modifiers[i].source != SRC_DECLINE ||
            game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
            continue;

        groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
        printBoomDecline("Market Decline", name,
                        game[0].economy.modifiers[i].percent,
                        game[0].economy.modifiers[i].roundsLeft);

        shown = 1;
        break;
    }

    /* Regional Development Card effects (Table 4) */
    if(printRegionalSection(game))
        shown = 1;

    if(!shown)
        printf("No active market booms, declines or regional conditions right now.\n");

    /* Other active event/regulation effects */
    printOtherConditions(game);

    printSectionHeader("Inflation");
    printf("%+d%%\n", game[0].economy.inflationRate);

    printSectionHeader("Current Loan Interest");
    printf("%d%%\n", game[0].economy.loanInterestRate);

    printSectionHeader("Current Income Tax Rate");
    printf("%d%%\n", game[0].economy.incomeTaxRate);

    printf("=====n");
}

```



```
}
```

A.14 verify_analysis.py - the verification analyzer (1459 lines)

```
#!/usr/bin/env python3
```

```
"""MONOPOLY-LK verification analyzer.
```

Programmatically parses a redirected stdout log (run.txt / test.txt) and checks **for** bugs and anomalies following MonopolyLK_Verification_Technique.pdf:

1. State-space consistency (square index <-> name)
2. Ownership / assignment integrity
3. Lifecycle / elimination handling
4. Ledger / running-total spot-checks
5. Final / summary arithmetic
6. Range / bounds sanity
7. Structural consistency

Usage: python verify_analysis.py <logfile> [<logfile> ...]

```
"""
```

```
import re
import sys
```

```
# -----
# Board constants, mirroring board.c / types.h (base values, pre-inflation)
# -----
```

```
BOARD = {
    0: "GO", 1: "Pettah", 2: "Community Development Fund", 3: "Maradana",
    4: "Income Tax", 5: "Colombo Fort Railway Station", 6: "Bambalapitiya",
    7: "National Event Card", 8: "Wellawatte", 9: "Mount Lavinia",
    10: "Jail / Just Visiting", 11: "Nugegoda", 12: "Ceylon Electricity Board",
    13: "Maharagama", 14: "Kottawa", 15: "Kandy Railway Station",
    16: "Negombo", 17: "Sri Lanka Insurance", 18: "Katunayake", 19: "Ja-Ela",
    20: "Free Parking", 21: "Kandy City", 22: "National Event Card",
    23: "Peradeniya", 24: "Katugastota", 25: "Galle Railway Station",
    26: "Galle Fort", 27: "Unawatuna",
    28: "National Water Supply and Drainage Board", 29: "Hikkaduwa",
    30: "Go To Jail", 31: "Jaffna Town", 32: "Nallur",
    33: "Ceylinco Insurance", 34: "Trincomalee", 35: "Jaffna Railway Station",
    36: "National Event Card", 37: "Nuwara Eliya", 38: "Bank of Ceylon",
    39: "Galle Face",
}
```

```
NAME2IDX = {v: k for k, v in BOARD.items()}
```

```
PROP_TYPE = {1, 3, 6, 8, 9, 11, 13, 14, 16, 18, 19, 21, 23, 24, 26, 27,
              29, 31, 32, 34, 37, 39}
```

```
RAIL_TYPE = {5, 15, 25, 35}
```

```
UTIL_TYPE = {12, 28}
```

```
GROUP = {1: "BROWN", 3: "BROWN", 6: "LB", 8: "LB", 9: "LB", 11: "PINK",
          13: "PINK", 14: "PINK", 16: "ORANGE", 18: "ORANGE", 19: "ORANGE",
          21: "RED", 23: "RED", 24: "RED", 26: "YELLOW", 27: "YELLOW",
          29: "YELLOW", 31: "GREEN", 32: "GREEN", 34: "GREEN", 37: "DB",
          39: "DB"}
```

```
BASE_PRICE = {1: 1500, 3: 1800, 6: 2500, 8: 2700, 9: 3000, 11: 3500,
              13: 3800, 14: 4000, 16: 4500, 18: 4700, 19: 5000, 21: 5500,
              23: 5800, 24: 6000, 26: 6500, 27: 6800, 29: 7000, 31: 8000,
              32: 8300, 34: 8500, 37: 10000, 39: 12000}
```

```
BASE_HCOST = {1: 500, 3: 500, 6: 750, 8: 750, 9: 750, 11: 1000, 13: 1000,
              14: 1000, 16: 1250, 18: 1250, 19: 1250, 21: 1500, 23: 1500,
              24: 1500, 26: 2000, 27: 2000, 29: 2000, 31: 2500, 32: 2500,
              34: 2500, 37: 3000, 39: 3000}
```

```
BASE_TCost = {1: 2000, 3: 2000, 6: 3000, 8: 3000, 9: 3000, 11: 4000,
              13: 4000, 14: 4000, 16: 5000, 18: 5000, 19: 5000, 21: 6000,
              23: 6000, 24: 6000, 26: 8000, 27: 8000, 29: 8000, 31: 10000,
              32: 10000, 34: 10000, 37: 12000, 39: 12000}
```

```
BASE_MORTGAGE = {1: 750, 3: 900, 6: 1250, 8: 1350, 9: 1500, 11: 1750,
                  13: 1900, 14: 2000, 16: 2250, 18: 2350, 19: 2500,
                  21: 2750, 23: 2900, 24: 3000, 26: 3250, 27: 3400,
                  29: 3500, 31: 4000, 32: 4150, 34: 4250, 37: 5000, 39: 6000}
```

```
BASE_RENT = {1: 100, 3: 120, 6: 180, 8: 200, 9: 220, 11: 260, 13: 280,
              14: 300, 16: 350, 18: 370, 19: 400, 21: 450, 23: 480,
              24: 500, 26: 600, 27: 620, 29: 650, 31: 750, 32: 780,
              34: 800, 37: 1000, 39: 1200}
```

```

RAIL_PRICE = {5: 1500, 15: 1500, 25: 1500, 35: 1500}
UTIL_PRICE = {12: 1500, 28: 1500}

PLAYERS = ["Aggressive Investor", "Conservative Banker", "Risk Taker",
           "Opportunistic Trader"]
P2I = {p: i for i, p in enumerate(PLAYERS)}

INFLATION_SET = {-3, 0, 2, 5, 8, 12}

def parse_money(s):
    """'LKR 12,345.' / 'LKR 12345' / '12,345' -> 12345"""
    s = s.replace("LKR", "").replace(",", "").replace(" ", "").replace(".", "")
    return int(s)

# -----
# Log reader (handles UTF-8 and UTF-16 LE, which Windows redirects produce)
# -----
def read_lines(path):
    with open(path, "rb") as f:
        raw = f.read()
        head = raw[:4096]
        if b"\x00" in head and (b"MONOPOLY" not in head and b"Pettah" not in head):
            text = raw.decode("utf-16-le", errors="replace")
        else:
            text = raw.decode("utf-8", errors="replace")
        if text.startswith("\uffff"):
            text = text[1:]
        return text.splitlines()

# -----
# Modifier type tags tracked from the log
# -----
VAL_GLOBAL = "VAL_GLOBAL"
VAL_GROUP = "VAL_GROUP"
VAL_INDEX = "VAL_INDEX"
VAL_RAIL = "VAL_RAIL"
RENT_GLOBAL = "RENT_GLOBAL"
RENT_GROUP = "RENT_GROUP"
RENT_HOTEL = "RENT_HOTEL"
RENT_RAIL = "RENT_RAIL"
RENT_UTIL = "RENT_UTIL"
CONSTRUCTION = "CONSTRUCTION"
INSURANCE = "INSURANCE"
MORTGAGE = "MORTGAGE"
AUCTION = "AUCTION"
RECESSION = "RECESSION"

class Analyzer:
    def __init__(self, path):
        self.path = path
        self.lines = read_lines(path)
        self.n = len(self.lines)
        self.findings = []

        # per-square live state
        self.owner = [-1] * 40
        self.mortgaged = [0] * 40
        self.houses = [0] * 40
        self.hotel = [0] * 40
        self.insurance = [0] * 40 # 0 none, 1 basic, 2 comp, 3 BI
        self.ins_rounds = [0] * 40
        self.price = [BASE_PRICE.get(i, RAIL_PRICE.get(i, UTIL_PRICE.get(i, 0)))]
            for i in range(40)]
        self.mortgage_val = [BASE_MORTGAGE.get(i, 0) for i in range(40)]
        self.hcost = [BASE_HCOST.get(i, 0) for i in range(40)]
        self.tcost = [BASE_T COST.get(i, 0) for i in range(40)]
        self.baserent = [BASE_RENT.get(i, 0) for i in range(40)]
        self.age = [0] * 40
        self.ren_round = [0] * 40 # last round renovated (0 = never)
        self.struct = [0] * 40
        self.pre_price = [0] * 40
        self.pre_rent = [0] * 40
        self.damaged = [0] * 40 # disaster-damaged (no rent)

```

```

self.repair_owed = [0] * 40
self.bi_lost = [0] * 40          # business interruption countdown

# per-player live state
self.pos = [0] * 4
self.cash = [30000] * 4
self.bankrupt = [0] * 4
self.loan_active = [0] * 4
self.loan_amount = [0] * 4
self.loan_remaining = [0] * 4
self.loan_rate = [0] * 4
self.last_dice = [0] * 4
self.bankrupt_round = [0] * 4

# economy
self.round = 0
self.modifiers = []              # [type, group|-1, index|-1, pct, left]
self.collected_mods = []       # modifiers added this round
self.inflation_rate = 0
self.loan_interest_rate = 8
self.income_tax_rate = 15
self.closed_prop = -1
self.closed_rounds = 0

# contextual
self.auction_prop = -1
self.auction_seller = -1
self.in_summary = False
self.summary_round = 0
self.lux_tax_fired = False
self.anti_spec_active = False
self.foreclose_pending = False

# -----
# helpers
# -----
def report(self, level, claim, line, quote=""):
    self.findings.append({
        "level": level,
        "claim": claim,
        "line": line,
        "quote": quote,
    })

def prop_name(self, idx):
    return BOARD.get(idx, "?")

def count_owned(self, p, types):
    c = 0
    for i in range(40):
        if self.owner[i] == p and (i in types or not types):
            c += 1
    return c

def count_houses(self, p):
    return sum(self.houses[i] for i in range(40)
               if self.owner[i] == p and self.hotel[i] == 0)

def count_hotels(self, p):
    return sum(1 for i in range(40)
               if self.owner[i] == p and self.hotel[i] == 1)

# -- value model -----
def apply_inflation(self, rate):
    for i in range(40):
        if i not in PROP_TYPE and i not in RAIL_TYPE and i not in UTIL_TYPE:
            continue
        self.price[i] = self.price[i] + self.price[i] * rate // 100
        if self.price[i] < 1:
            self.price[i] = 1
        self.mortgage_val[i] = self.mortgage_val[i] + \
            self.mortgage_val[i] * rate // 100
        if self.mortgage_val[i] < 1:
            self.mortgage_val[i] = 1
        if i in PROP_TYPE:
            self.baserent[i] = self.baserent[i] + \
                self.baserent[i] * rate // 100

```

```

        self.hcost[i] = self.hcost[i] + self.hcost[i] * rate // 100
        self.tcost[i] = self.tcost[i] + self.tcost[i] * rate // 100

def add_mod(self, mtype, group, index, pct, left):
    self.collected_mods.append([mtype, group, index, pct, left])

def value_mult(self, idx):
    mult = 100
    for m in self.modifiers:
        if m[0] == VAL_GLOBAL and (m[1] == -1 or True):
            mult = mult * m[3] // 100
    if idx in PROP_TYPE:
        g = GROUP[idx]
        for m in self.modifiers:
            if m[0] == VAL_GROUP and (m[1] == g or m[1] == -1):
                mult = mult * m[3] // 100
    if idx in RAIL_TYPE:
        for m in self.modifiers:
            if m[0] == VAL_RAIL:
                mult = mult * m[3] // 100
    for m in self.modifiers:
        if m[0] == VAL_INDEX and (m[2] == idx or m[2] == -1):
            mult = mult * m[3] // 100
    return mult

def market_value(self, idx):
    price = self.price[idx]
    age = self.age[idx]
    depr = 0
    if age > 50:
        depr = (age - 50) // 5
        if depr > 30:
            depr = 30
    price = price - price * depr // 100
    price = price * self.value_mult(idx) // 100
    return price

def mult_for(self, mtype, group=-1, index=-1):
    mult = 100
    for m in self.modifiers:
        if m[0] != mtype:
            continue
        if group != -1 and m[1] != -1 and m[1] != group:
            continue
        if index != -1 and m[2] != -1 and m[2] != index:
            continue
        mult = mult * m[3] // 100
    return mult

def building_value(self, p):
    total = 0
    for i in range(40):
        if self.owner[i] != p:
            continue
        if self.hotel[i]:
            total += self.tcost[i]
        else:
            total += self.houses[i] * self.hcost[i]
    return total

def prop_value(self, p):
    total = 0
    for i in range(40):
        if self.owner[i] == p:
            total += self.market_value(i)
    return total

def net_worth(self, p):
    return (self.cash[p] + self.prop_value(p) + self.building_value(p)
            - self.loan_amount[p])

def dec_mods(self):
    kept = []
    for m in self.modifiers:
        m[4] -= 1
        if m[4] > 0:
            kept.append(m)

```

```

self.modifiers = kept
for m in self.collected_mods:
    self.modifiers.append(m)
self.collected_mods = []

def land_index_from_name(self, name, line):
    idx = NAME2IDX.get(name)
    if idx is None:
        self.report("REVIEW", "Unknown square name in log: %r" % name,
                    line, name)
        return -1
    return idx

# -----
# main loop
# -----
def run(self):
    i = 0
    while i < self.n:
        line = self.lines[i].strip()
        ln = i + 1
        self.handle(line, ln, i)
        i += 1
    return self.findings

# -----
def handle(self, line, ln, i):
    # --- round headers -----
    m = re.match(r"^ROUND (\d+)$", line)
    if m:
        self.round = int(m.group(1))
        return
    m = re.match(r"^Round (\d+) Summary$", line)
    if m:
        self.summary_round = int(m.group(1))
        self.end_of_round(i + 1)
        return

    # --- dice & movement -----
    m = re.match(r"^(.+?) rolled (\d+)\.\\.$", line)
    if m and m.group(1) in P2I:
        p = P2I[m.group(1)]
        d = int(m.group(2))
        if self.bankrupt[p]:
            self.report("BUG", "Bankrupt player %s rolled dice." %
                        PLAYERS[p], ln, line)
        if not (2 <= d <= 12):
            self.report("BUG", "Dice roll %d out of range 2..12 for %s." %
                        (d, PLAYERS[p]), ln, line)
        self.last_dice[p] = d
        return
    m = re.match(r"^(.+?) rolls (\d+)\.\\.$", line)
    if m and m.group(1) in P2I:
        d = int(m.group(2))
        if not (2 <= d <= 12):
            self.report("BUG", "First-player roll %d out of range 2..12." %
                        d, ln, line)
        return
    m = re.match(r"^(.+?) is in Jail\\. Rolled (\d+) and (\d+)\.\\.$", line)
    if m and m.group(1) in P2I:
        p = P2I[m.group(1)]
        a = int(m.group(2))
        b = int(m.group(3))
        if not (1 <= a <= 6 and 1 <= b <= 6):
            self.report("BUG", "Jail dice %d/%d out of range 1..6." %
                        (a, b), ln, line)
        return
    m = re.match(r"^(.+?) moves from Square (\d+) to Square (\d+)\.\\.$", line)
    if m and m.group(1) in P2I:
        p = P2I[m.group(1)]
        frm = int(m.group(2))
        to = int(m.group(3))
        if not (0 <= frm < 40 and 0 <= to < 40):
            self.report("BUG", "Square %d/%d outside 0..39." %
                        (frm, to), ln, line)
        if self.last_dice[p] and (frm + self.last_dice[p]) % 40 != to:
            self.report("BUG",

```

```

                "%s moved %d->%d but dice was %d." %
                (PLAYERS[p], frm, to, self.last_dice[p]),
                ln, line)
        self.pos[p] = to
        return
    m = re.match(r"^(.+) is sent to Jail!$", line)
    if m and m.group(1) in P2I:
        self.pos[P2I[m.group(1)]] = 10
        return

# ---- GO -----
m = re.match(r"^(.+) passed GO\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    go = 2000
    if i + 1 < self.n:
        m2 = re.match(r"^Collected LKR ([0-9,]+)$",
                        self.lines[i + 1].strip())
        if m2:
            go = parse_money(m2.group(1))
    if self.bankrupt[p]:
        self.report("BUG", "Bankrupt player %s passed GO." %
                    PLAYERS[p], ln, line)
    prev = self.cash[p]
    self.cash[p] += go
    if i + 2 < self.n:
        m3 = re.match(r"^Current Balance : LKR ([0-9,]+)$",
                        self.lines[i + 2].strip())
        if m3:
            bal = parse_money(m3.group(1))
            if bal != prev + go:
                self.report("BUG",
                            "%s GO balance %d != %d + %d." %
                            (PLAYERS[p], bal, prev, go), ln,
                            self.lines[i + 2].strip())

    return

# ---- purchases -----
m = re.match(r"^(.+) purchased (.+) for LKR ([0-9,]+\.$)", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    price = parse_money(m.group(3))
    idx = self.land_index_from_name(name, ln)
    if idx >= 0 and idx != self.pos[p]:
        self.report("BUG",
                    "%s bought %s (square %d) while on square %d." %
                    (PLAYERS[p], name, idx, self.pos[p]), ln, line)
    if idx >= 0 and self.owner[idx] != -1:
        self.report("BUG",
                    "%s bought already-owned %s (owner %d)." %
                    (PLAYERS[p], name, self.owner[idx]), ln, line)
    if self.bankrupt[p]:
        self.report("BUG", "Bankrupt player %s purchased %s." %
                    (PLAYERS[p], name), ln, line)
    prev = self.cash[p]
    self.cash[p] -= price
    if idx >= 0:
        self.owner[idx] = p
        self.mortgaged[idx] = 0
    # verify declared remaining balance
    if i + 1 < self.n:
        m2 = re.match(r"^Remaining Balance : LKR ([0-9,]+\.$)",
                        self.lines[i + 1].strip())
        if m2:
            bal = parse_money(m2.group(1))
            if bal != prev - price:
                self.report("BUG",
                            "%s purchase balance %d != %d - %d." %
                            (PLAYERS[p], bal, prev, price), ln,
                            self.lines[i + 1].strip())

    return

# ---- rent -----
m = re.match(r"^(.+) landed on (.+)\.$", line)
if m and m.group(1) in P2I and \
    m.group(2) not in ("Income Tax", "Community Development Fund"):
```

```

p = P2I[m.group(1)]
name = m.group(2)
if name in NAME2IDX and self.pos[p] != NAME2IDX[name]:
    self.report("BUG",
                "%s landed on %s (square %d) but is on square %d." %
                (PLAYERS[p], name, NAME2IDX[name], self.pos[p]),
                ln, line)
# is this a rent-paying landing? (next line says Rent Paid)
nxt = self.lines[i + 1].strip() if i + 1 < self.n else ""
if re.match(r"^Rent Paid : LKR", nxt):
    self.handle_rent(p, name, ln, i)
    return
return

# ---- construction -----
m = re.match(r"^(.+) constructed one house on (.+)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    idx = self.land_index_from_name(name, ln)
    if idx >= 0:
        if self.owner[idx] != p:
            self.report("BUG",
                        "%s built a house on %s (owner %d)." %
                        (PLAYERS[p], name, self.owner[idx]), ln, line)
        if self.mortgaged[idx]:
            self.report("BUG",
                        "%s built a house on mortgaged %s." %
                        (PLAYERS[p], name), ln, line)
        self.houses[idx] += 1
    cost = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Construction Cost : LKR ([0-9,]+\.)\.$",
                      self.lines[i + 1].strip())
        if m2:
            cost = parse_money(m2.group(1))
    self.cash[p] -= cost
    return
m = re.match(r"^(.+) upgraded (.+) to a Hotel\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    idx = self.land_index_from_name(name, ln)
    if idx >= 0:
        if self.owner[idx] != p:
            self.report("BUG",
                        "%s built a hotel on %s (owner %d)." %
                        (PLAYERS[p], name, self.owner[idx]), ln, line)
        self.hotel[idx] = 1
        self.houses[idx] = 0
    cost = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Construction Cost : LKR ([0-9,]+\.)\.$",
                      self.lines[i + 1].strip())
        if m2:
            cost = parse_money(m2.group(1))
    self.cash[p] -= cost
    return

# ---- mortgage / redeem -----
m = re.match(r"^(.+) mortgaged (.+) for LKR ([0-9,]+\.)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    payout = parse_money(m.group(3))
    idx = self.land_index_from_name(name, ln)
    if idx >= 0 and self.owner[idx] != p:
        self.report("BUG",
                    "%s mortgaged %s (owner %d)." %
                    (PLAYERS[p], name, self.owner[idx]), ln, line)
        self.mortgaged[idx] = 1
        self.cash[p] += payout
    return
m = re.match(r"^(.+) redeemed the mortgage on (.+) for LKR ([0-9,]+\.)\.$",
              line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]

```

```

name = m.group(2)
cost = parse_money(m.group(3))
idx = self.land_index_from_name(name, ln)
if idx >= 0 and self.owner[idx] != p:
    self.report("BUG",
                "%s redeemed mortgage on %s (owner %d)." %
                (PLAYERS[p], name, self.owner[idx]), ln, line)
self.mortgaged[idx] = 0
self.cash[p] -= cost
return

# ---- sales to bank -----
m = re.match(r"^(.+?) sold (.+?) to the Bank for LKR ([0-9,]+)", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    value = parse_money(m.group(3))
    idx = self.land_index_from_name(name, ln)
    if idx >= 0 and self.owner[idx] != p:
        self.report("BUG",
                    "%s sold %s (owner %d) to the Bank." %
                    (PLAYERS[p], name, self.owner[idx]), ln, line)
    self.cash[p] += value
    if idx >= 0:
        self.owner[idx] = -1
        self.mortgaged[idx] = 0
        self.houses[idx] = 0
        self.hotel[idx] = 0
    return

# ---- loans -----
m = re.match(r"^(.+?) obtained a secured loan\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    amt = 0
    rate = 0
    j = i + 1
    while j < self.n:
        l2 = self.lines[j].strip()
        m2 = re.match(r"^Loan Amount : LKR ([0-9,]+)\.$", l2)
        if m2:
            amt = parse_money(m2.group(1))
        m3 = re.match(r"^Interest Rate : (\d+)\%$", l2)
        if m3:
            rate = int(m3.group(1))
        if l2 == "" and m2 is None:
            break
        if re.match(r"^Collateral :$", l2):
            break
        j += 1
    if self.loan_active[p]:
        self.report("BUG",
                    "%s obtained a second loan while one was active." %
                    PLAYERS[p], ln, line)
    self.loan_active[p] = 1
    self.loan_amount[p] = amt
    self.loan_rate[p] = rate
    self.loan_remaining[p] = 20
    self.cash[p] += amt
    return
m = re.match(r"^(.+?) repaid LKR (\d+) towards their loan\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    amt = int(m.group(2))
    self.cash[p] -= amt
    self.loan_amount[p] -= amt
    if self.loan_amount[p] <= 0:
        self.loan_active[p] = 0
        self.loan_amount[p] = 0
    return
m = re.match(r"^(.+?) increased their loan by LKR (\d+)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    amt = int(m.group(2))
    self.cash[p] += amt
    self.loan_amount[p] += amt
    return

```



```

m = re.match(r"^(.+)'s loan accrued LKR (\d+) interest\."
              r"New balance : LKR (\d+)$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    interest = int(m.group(2))
    newbal = int(m.group(3))
    if self.loan_amount[p] + interest != newbal:
        self.report("BUG",
                    "%s loan interest %d + %d != new balance %d." %
                    (PLAYERS[p], self.loan_amount[p], interest, newbal),
                    ln, line)
    self.loan_amount[p] = newbal
    self.loan_remaining[p] -= 1
    return
m = re.match(r"^(.+?) extended their loan by (\d+) rounds\.$", line)
if m and m.group(1) in P2I:
    self.loan_remaining[P2I[m.group(1)]] += int(m.group(2))
    return
m = re.match(r"^(.+?) refinanced their loan at the current rate of "
              r"(\d+)\%\.\\$", line)
if m and m.group(1) in P2I:
    self.loan_rate[P2I[m.group(1)]] = int(m.group(2))
    return
m = re.match(r"^(.+?) has defaulted on their loan!$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    self.foreclose(p, ln, i)
    return

# ---- bankruptcy -----
if line == "*** BANKRUPTCY ***":
    nxt = self.lines[i + 1].strip()
    if i + 1 < self.n else ""
    m2 = re.match(r"^(.+?) has been declared bankrupt\.$", nxt)
    if m2 and m2.group(1) in P2I:
        p = P2I[m2.group(1)]
        self.handle_bankruptcy(p, ln, i + 1)
    return

# ---- auctions -----
m = re.match(r"^\*\*\* AUCTION \*\*\*$", line)
if m:
    self.auction_prop = -1
    self.auction_seller = -1
    if i + 1 < self.n:
        m2 = re.match(r"^Property : (.+)$", self.lines[i + 1].strip())
        if m2:
            nm = m2.group(1)
            self.auction_prop = self.land_index_from_name(nm, ln + 1)
    return
m = re.match(r"^(.+?) wins the auction for LKR ([0-9,]+)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    bid = parse_money(m.group(2))
    idx = self.auction_prop
    if idx >= 0 and self.owner[idx] != -1 and self.auction_seller == -1 \
        and not getattr(self, "foreclose_pending", False):
        self.report("BUG",
                    "Auction %s was already owned by player %d." %
                    (self.prop_name(idx), self.owner[idx]), ln, line)
    if self.bankrupt[p]:
        self.report("BUG", "Bankrupt player %s won an auction." %
                    PLAYERS[p], ln, line)
    self.cash[p] -= bid
    if self.auction_seller >= 0:
        self.cash[self.auction_seller] += bid
        if self.auction_seller == p:
            self.report("BUG",
                        "Anti-Speculation forced seller %s bought "
                        "back %s (net-zero cost)." %
                        (PLAYERS[p], self.prop_name(idx)), ln, line)
    if idx >= 0:
        self.owner[idx] = p
        self.mortgaged[idx] = 0
        self.houses[idx] = 0
        self.hotel[idx] = 0
    self.auction_prop = -1
    self.auction_seller = -1

```

```

    return
m = re.match(r"^No bids received\. (.+?) remains with the Bank\.$",
              line)
if m:
    idx = self.auction_prop
    if idx >= 0:
        self.owner[idx] = -1
        self.mortgaged[idx] = 0
        self.houses[idx] = 0
        self.hotel[idx] = 0
    self.auction_prop = -1
    self.auction_seller = -1
    return
m = re.match(r"^Anti-Speculation Act : (.+?) must sell (.+)\.$", line)
if m and m.group(1) in P2I:
    self.auction_seller = P2I[m.group(1)]
    return

# ---- insurance -----
m = re.match(r"^(.+?) purchased insurance for (.+)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    name = m.group(2)
    idx = self.land_index_from_name(name, ln)
    if idx >= 0 and self.owner[idx] != p:
        self.report("BUG",
                    "%s insured %s (owner %d)." %
                    (PLAYERS[p], name, self.owner[idx]), ln, line)

    premium = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Premium : LKR ([0-9,]+\.)\.$",
                      self.lines[i + 1].strip())
        if m2:
            premium = parse_money(m2.group(1))
        self.cash[p] -= premium
        if idx >= 0:
            self.ins_rounds[idx] = 20
    return
m = re.match(r"^Insurance policy on (.+) expires in (\d+) rounds\.$",
              line)
if m:
    return
m = re.match(r"^Insurance policy on (.+) has expired\.$", line)
if m:
    idx = self.land_index_from_name(m.group(1), ln)
    if idx >= 0:
        self.insurance[idx] = 0
    return

# ---- taxes -----
m = re.match(r"^(.+?) landed on Income Tax\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    amt = 0
    j = i + 1
    while j < self.n:
        l2 = self.lines[j].strip()
        m2 = re.match(r"^(.+?) paid tax : LKR (\d+)\.$", l2)
        if m2:
            amt = int(m2.group(2))
            break
        j += 1
    self.cash[p] -= amt
    return
m = re.match(r"^(.+?) landed on Community Development Fund\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    amt = 0
    j = i + 1
    while j < self.n:
        l2 = self.lines[j].strip()
        m2 = re.match(r"^(.+?) paid community development tax : LKR (\d+)\.$",
                      l2)
        if m2:
            amt = int(m2.group(2))
            break
        j += 1

```

```

        self.cash[p] -= amt
        return

# ---- bail -----
m = re.match(r"^(.+?) pays bail of LKR (\d+) and is released from "
            r"Jail\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    self.cash[p] -= int(m.group(2))
    return
m = re.match(r"^(.+?) has been in Jail for \d+ turns - must pay bail "
            r"of LKR (\d+) and is released\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    self.cash[p] -= int(m.group(2))
    return

# ---- disaster / repair -----
m = re.match(r"^Affected Property : (.+?) \(owner : (.+?)\)$", line)
if m and m.group(2) in P2I:
    self.disaster_prop = NAME2IDX.get(m.group(1), -1)
    self.disaster_owner = P2I[m.group(2)]
    return
m = re.match(r"^Compensation Paid : LKR (\d+)\$", line)
if m:
    comp = int(m.group(1))
    if self.disaster_owner >= 0:
        self.cash[self.disaster_owner] += comp
    if self.disaster_prop >= 0:
        self.repair_owed[self.disaster_prop] -= comp
    return
m = re.match(r"^(.+?) is damaged and stops earning rent until "
            r"repaired\.$", line)
if m:
    idx = NAME2IDX.get(m.group(1), -1)
    if idx >= 0:
        self.damaged[idx] = 1
    return
m = re.match(r"^(.+?) repaired (.+?)\.. It can collect rent again\.$",
            line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    idx = NAME2IDX.get(m.group(2), -1)
    cost = self.repair_owed[idx] if idx >= 0 else 0
    if i + 1 < self.n:
        m2 = re.match(r"^Repair Cost : LKR ([0-9,]+)\.?\$",
            self.lines[i + 1].strip())
        if m2:
            cost = parse_money(m2.group(1))
    self.cash[p] -= cost
    if idx >= 0:
        self.damaged[idx] = 0
        self.repair_owed[idx] = 0
    return

# ---- maintenance / renovation -----
m = re.match(r"^(.+?) performed maintenance on (.+?)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    idx = NAME2IDX.get(m.group(2), -1)
    cost = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Maintenance Cost : LKR (\d+)\$",
            self.lines[i + 1].strip())
        if m2:
            cost = int(m2.group(1))
    self.cash[p] -= cost
    return
m = re.match(r"^(.+?) renovated (.+?)\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    idx = NAME2IDX.get(m.group(2), -1)
    cost = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Renovation Cost : LKR (\d+)\$",
            self.lines[i + 1].strip())
        if m2:

```

```

        cost = int(m2.group(1))
        self.cash[p] -= cost
        if idx >= 0:
            self.age[idx] = 0
            self.ren_round[idx] = self.round
        return
m = re.match(r"^(.+?) fully renovated (.+?) after structural "
            r"damage\.$", line)
if m and m.group(1) in P2I:
    p = P2I[m.group(1)]
    idx = NAME2IDX.get(m.group(2), -1)
    cost = 0
    if i + 1 < self.n:
        m2 = re.match(r"^Renovation Cost : LKR (\d+)$",
            self.lines[i + 1].strip())
        if m2:
            cost = int(m2.group(1))
        self.cash[p] -= cost
    if idx >= 0:
        self.price[idx] = self.pre_price[idx]
        self.basement[idx] = self.pre_rent[idx]
        self.struct[idx] = 0
    return
m = re.match(r"^(.+?) has suffered structural damage from neglect!$",
    line)
if m:
    idx = NAME2IDX.get(m.group(1), -1)
    if idx >= 0:
        self.pre_price[idx] = self.price[idx]
        self.pre_rent[idx] = self.basement[idx]
        self.price[idx] = self.price[idx] * 85 // 100
        self.basement[idx] = self.basement[idx] * 75 // 100
        self.struct[idx] = 1
    return

# ---- economy triggers -----
m = re.match(r"^New Inflation Rate : ([+-]?\d+)%$", line)
if m:
    rate = int(m.group(1))
    if rate not in INFLATION_SET:
        self.report("BUG", "Unexpected inflation rate %d." % rate,
            ln, line)
    self.inflation_rate = rate
    self.apply_inflation(rate)
    return
m = re.match(r"^New Loan Interest Rate : (\d+)%$", line)
if m:
    self.loan_interest_rate = int(m.group(1))
    if not (1 <= self.loan_interest_rate <= 25):
        self.report("BUG",
            "Loan interest %d outside 1..25." %
            self.loan_interest_rate, ln, line)
    return

# national event cards
if line == "*** NATIONAL EVENT CARD ***":
    self.parse_event_card(ln, i)
    return
if line == "=== Economic Event ===":
    self.parse_econ_event(ln, i)
    return
if line == "=== Government Regulation ===":
    self.parse_regulation(ln, i)
    return
if line == "=== Regional Development Card ===":
    self.parse_regional(ln, i)
    return
if line == "=== Property Market Review ===":
    self.parse_market_review(ln, i)
    return
if line == "*** DISASTER ***":
    return

# game over
if line == "GAME OVER":
    self.parse_game_over(ln, i)
    return

```

```

# -----
def handle_rent(self, payer, name, ln, i):
    nxt = self.lines[i + 1].strip()
    m = re.match(r"^Rent Paid : LKR ([0-9,]+)\.\"", nxt)
    if not m:
        return
    rent = parse_money(m.group(1))
    owner_name = ""
    if i + 2 < self.n:
        m2 = re.match(r"^Owner : (.+?)\.", self.lines[i + 2].strip())
        if m2:
            owner_name = m2.group(1)
    owner = P2I.get(owner_name, -1)
    idx = NAME2IDX.get(name, -1)

    if owner == payer:
        self.report("BUG",
            "%s paid rent to themselves on %s." %
            (PLAYERS[payer], name), ln, nxt)
    if owner >= 0 and self.bankrupt[owner]:
        self.report("BUG",
            "Rent on %s paid to bankrupt owner %s." %
            (name, PLAYERS[owner]), ln, nxt)
    if idx >= 0 and self.owner[idx] != owner:
        self.report("BUG",
            "Rent on %s paid to %s but tracked owner is %s." %
            (name, PLAYERS[owner] if owner >= 0 else "?",
            PLAYERS[self.owner[idx] if self.owner[idx] >= 0
            else "Bank"), ln, nxt)
    if idx >= 0 and self.mortgaged[idx]:
        self.report("BUG", "Rent collected on mortgaged %s." % name,
            ln, nxt)
    if not self.bankrupt[payer]:
        self.cash[payer] -= rent
    if owner >= 0:
        self.cash[owner] += rent
    self.last_rent_owner = owner

# -----
def handle_bankruptcy(self, p, ln, li):
    self.bankrupt[p] = 1
    self.bankrupt_round[p] = self.summary_round or self.round
    self.cash[p] = 0
    self.loan_active[p] = 0
    self.loan_amount[p] = 0

    # liquidateBankruptAssets() strips ownership of every square the
    # player owns before auctioning it - mirror that here so the
    # follow-up auctions see owner -1
    for i2 in range(40):
        if self.owner[i2] == p:
            self.owner[i2] = -1
            self.mortgaged[i2] = 0
            self.houses[i2] = 0
            self.hotel[i2] = 0

    # evidence: what preceded the bankruptcy message?
    found = ""
    for k in range(max(0, li - 8), li):
        t = self.lines[k].strip()
        if re.search(r"LKR [0-9,]+", t):
            found = t
    if not found:
        self.report("REVIEW",
            "No monetary evidence immediately before %s's "
            "bankruptcy." % PLAYERS[p], ln, "")

    # assets must be auctioned next
    owned = [i for i in range(40) if self.owner[i] == p]
    if owned:
        msg = "Bankrupt %s owned %d square(s) that must be auctioned." % (
            PLAYERS[p], len(owned))
        self.pending_liquidation = owned
        self.pending_liquidation_player = p
        self.liquidation_checked_at = ln
        self.pending_auctions = list(owned)
    else:

```

```

        self.pending_liquidation = []

def foreclose(self, p, ln, i):
    # verify evidence (accrued interest line precedes)
    found = ""
    for k in range(max(0, i - 20), i):
        t = self.lines[k].strip()
        if "accrued" in t or "New balance" in t:
            found = t
    if not found:
        self.report("REVIEW",
                    "No loan-interest evidence before %s's foreclosure." %
                    PLAYERS[p], ln, "")

    # the defaulted loan is cleared (bank.c foreclose) - the pledged
    # collateral squares keep their tracked owner until each auction
    # reassigns them, so suppress the "already owned" auction check
    # until the next round summary
    self.loan_active[p] = 0
    self.loan_amount[p] = 0
    self.foreclose_pending = True

# -----
def parse_event_card(self, ln, i):
    j = i + 1
    while j < self.n and self.lines[j].strip() == "":
        j += 1
    if j >= self.n:
        return
    txt = self.lines[j].strip()
    g = 0
    if "Tourism Hype" in txt:
        self.add_mod(RENT_HOTEL, -1, -1, 200, 5)
    elif "Fuel Shortage" in txt:
        self.add_mod(RENT_RAIL, -1, -1, 200, 5)
    elif "Heavy Floods" in txt:
        pass
    elif "Political Rally" in txt:
        pass
    elif "Stock Market Rise" in txt:
        self.add_mod(VAL_GLOBAL, -1, -1, 110, 15)
    elif "Economic Downturn" in txt:
        self.add_mod(VAL_GLOBAL, -1, -1, 85, 15)
    elif "Housing Subsidy" in txt:
        self.add_mod(CONSTRUCTION, -1, -1, 70, 15)
    elif "Interest Rate Cut" in txt:
        self.loan_interest_rate -= 2
    elif "Interest Rate Increase" in txt:
        self.loan_interest_rate += 2
    elif "Tax Amnesty" in txt:
        for p in range(4):
            if not self.bankrupt[p]:
                self.cash[p] += 2000
    elif "Power Failure" in txt:
        self.add_mod(RENT_UTIL, -1, -1, 50, 3)
    elif "Foreign Funding" in txt:
        self.add_mod(VAL_GROUP, GROUP[16], -1, 115, 15)
    elif "Port Expansion" in txt:
        self.add_mod(VAL_RAIL, -1, -1, 120, 15)
    elif "Festival Season" in txt:
        self.add_mod(RENT_HOTEL, -1, -1, 150, 5)
    elif "Labour Strike" in txt:
        pass
    elif "Insurance Discount" in txt:
        self.add_mod(INSURANCE, -1, -1, 80, 10)
    elif "Property Revaluation" in txt:
        for nm, gi in (("Brown", 1), ("Light Blue", 6), ("Pink", 11),
                      ("Orange", 16), ("Red", 21), ("Yellow", 26),
                      ("Green", 31), ("Dark Blue", 37)):
            if txt.startswith("Property Revaluation : %s " % nm):
                self.add_mod(VAL_GROUP, gi, -1, 115, 15)
                break
    elif "Currency Depreciation" in txt:
        self.add_mod(CONSTRUCTION, -1, -1, 110, 15)
    elif "Government Grant" in txt:
        m = re.match(r"^Government Grant : (.+?) receives LKR 5,000\.$",
                    txt)

```

```

        if m and m.group(1) in P2I:
            lucky = P2I[m.group(1)]
            if self.bankrupt[lucky]:
                self.report("BUG",
                            "Government Grant awarded to bankrupt %s." %
                            PLAYERS[lucky], ln, txt)
            else:
                self.cash[lucky] += 5000
    elif "National Disaster" in txt:
        pass

def parse_econ_event(self, ln, i):
    j = i + 1
    while j < self.n and self.lines[j].strip() == "":
        j += 1
    if j >= self.n:
        return
    txt = self.lines[j].strip()
    if "Tourism Boom" in txt:
        self.add_mod(RENT_HOTEL, -1, -1, 200, 15)
        self.add_mod(VAL_GROUP, GROUP[26], -1, 115, 15)
    elif "Fuel Crisis" in txt:
        self.add_mod(RENT_RAIL, -1, -1, 200, 15)
        self.add_mod(CONSTRUCTION, -1, -1, 120, 15)
    elif "Heavy Monsoon" in txt:
        self.add_mod(INSURANCE, -1, -1, 115, 15)
        self.add_mod(VAL_GROUP, GROUP[26], -1, 90, 15)
    elif "Economic Recession" in txt:
        self.add_mod(VAL_GLOBAL, -1, -1, 85, 15)
        self.add_mod(RENT_GLOBAL, -1, -1, 90, 15)
        self.add_mod(RECESSION, -1, -1, 100, 15)
        self.loan_interest_rate = self.loan_interest_rate + \
            self.loan_interest_rate * 15 // 100
    elif "Stock Market Boom" in txt:
        self.add_mod(VAL_GLOBAL, -1, -1, 110, 15)
        self.loan_interest_rate = self.loan_interest_rate - \
            self.loan_interest_rate * 10 // 100
    elif "Government Housing Programme" in txt:
        self.add_mod(CONSTRUCTION, -1, -1, 75, 15)
    elif "Foreign Investment" in txt:
        self.add_mod(VAL_GROUP, GROUP[16], -1, 120, 15)
    elif "Political Unrest" in txt:
        self.add_mod(RENT_HOTEL, -1, -1, 50, 15)

def parse_regulation(self, ln, i):
    j = i + 1
    while j < self.n and self.lines[j].strip() == "":
        j += 1
    if j >= self.n:
        return
    txt = self.lines[j].strip()
    if "Increase Property Tax" in txt:
        self.income_tax_rate = self.income_tax_rate + \
            self.income_tax_rate * 50 // 100
        if self.income_tax_rate > 25:
            self.income_tax_rate = 25
    elif "Reduce Loan Interest" in txt:
        self.loan_interest_rate -= 2
        if self.loan_interest_rate < 1:
            self.loan_interest_rate = 1
    elif "Housing Subsidy" in txt:
        self.add_mod(CONSTRUCTION, -1, -1, 70, 20)
    elif "Luxury Property Tax" in txt:
        self.lux_tax_fired = True
        # amount per hotel is not printed - charged silently
        for i2 in range(40):
            if self.hotel[i2] and self.owner[i2] >= 0:
                tax = self.market_value(i2) * 25 // 100
                self.cash[self.owner[i2]] -= tax
    elif "Railway Modernization" in txt:
        self.add_mod(RENT_RAIL, -1, -1, 125, 20)
    elif "Electricity Tariff Revision" in txt:
        self.add_mod(RENT_UTIL, -1, -1, 120, 20)
    elif "Insurance Regulation" in txt:
        self.add_mod(INSURANCE, -1, -1, 85, 20)
    elif "Anti-Speculation Act" in txt:
        self.anti_spec_active = True

```

```

def parse_regional(self, ln, i):
    j = i + 1
    while j < self.n and self.lines[j].strip() == "":
        j += 1
    if j >= self.n:
        return
    txt = self.lines[j].strip()
    if "Southern Tourism Boom" in txt:
        self.add_mod(RENT_GROUP, GROUP[26], -1, 140, 15)
    elif "Port City Expansion" in txt:
        self.add_mod(VAL_GROUP, GROUP[1], -1, 125, 15)
        self.add_mod(VAL_INDEX, -1, 5, 125, 15)
    elif "IT Industry Growth" in txt:
        self.add_mod(VAL_GROUP, GROUP[11], -1, 120, 15)
    elif "Northern Development" in txt:
        self.add_mod(VAL_GROUP, GROUP[31], -1, 130, 15)
    elif "Tea Export Boom" in txt:
        self.add_mod(VAL_INDEX, -1, 37, 135, 15)
    elif "Airport Expansion" in txt:
        self.add_mod(RENT_GROUP, GROUP[16], -1, 130, 15)
    elif "University City Growth" in txt:
        self.add_mod(VAL_GROUP, GROUP[21], -1, 120, 15)
    elif "Beach Pollution" in txt:
        self.add_mod(RENT_GROUP, GROUP[26], -1, 70, 15)
    elif "Flood Damage" in txt:
        self.add_mod(VAL_GROUP, GROUP[26], -1, 80, 15)
    elif "Transport Strike" in txt:
        self.add_mod(RENT_RAIL, -1, -1, 60, 15)
    elif "Electricity Tariff Increase" in txt:
        self.add_mod(RENT_UTIL, -1, -1, 125, 15)
    elif "Water Shortage" in txt:
        self.add_mod(RENT_UTIL, -1, -1, 120, 15)
        self.add_mod(VAL_INDEX, -1, 13, 90, 15)
        self.add_mod(VAL_INDEX, -1, 29, 90, 15)

def parse_market_review(self, ln, i):
    boom = ""
    decline = ""
    for k in range(i + 1, min(i + 6, self.n)):
        t = self.lines[k].strip()
        m = re.match(r"^Market Boom : (.+?) \\\(values", t)
        if m:
            boom = m.group(1)
            m = re.match(r"^Market Decline : (.+?) \\\(values", t)
            if m:
                decline = m.group(1)
    GROUP_N2I = {"Brown": 1, "Light Blue": 6, "Pink": 11, "Orange": 16,
                 "Red": 21, "Yellow": 26, "Green": 31, "Dark Blue": 37}
    if boom in GROUP_N2I:
        g = GROUP_N2I[boom]
        self.add_mod(VAL_GROUP, g, -1, 120, 10)
        self.add_mod(RENT_GROUP, g, -1, 125, 10)
        self.add_mod(MORTGAGE, g, -1, 115, 10)
        self.add_mod(CONSTRUCTION, g, -1, 110, 10)
    if decline in GROUP_N2I:
        g = GROUP_N2I[decline]
        self.add_mod(VAL_GROUP, g, -1, 85, 10)
        self.add_mod(RENT_GROUP, g, -1, 80, 10)
        self.add_mod(MORTGAGE, g, -1, 90, 10)
        self.add_mod(AUCTION, g, -1, 75, 10)

# -----
def end_of_round(self, start_line):
    """A 'Round N Summary' header was just seen. Apply the round-end
    tick (age, modifier decay) and then verify the summary block."""
    # replicate: ageProperties (every property age++), decrementModifiers
    for i2 in range(40):
        self.age[i2] += 1
    self.dec_mods()

    # parse the summary block that follows
    nw = [None] * 4
    cash = [None] * 4
    props = [None] * 4
    rails = [None] * 4
    utils = [None] * 4
    houses = [None] * 4

```



```

hotels = [None] * 4
loan = [None] * 4
status = [None] * 4
cur = None
k = start_line
while k < self.n:
    t = self.lines[k].strip()
    if t == "Current Market Conditions":
        break
    if t in P2I:
        cur = P2I[t]
    elif cur is not None:
        m = re.match(r"^Cash : LKR ([0-9,]+)$", t)
        if m:
            cash[cur] = parse_money(m.group(1))
        m = re.match(r"^Net Worth : LKR ([0-9,]+)$", t)
        if m:
            nw[cur] = parse_money(m.group(1))
        m = re.match(r"^Properties : (\d+)$", t)
        if m:
            props[cur] = int(m.group(1))
        m = re.match(r"^Railways\s+: (\d+)$", t)
        if m:
            rails[cur] = int(m.group(1))
        m = re.match(r"^Utilities\s+: (\d+)$", t)
        if m:
            utils[cur] = int(m.group(1))
        m = re.match(r"^Houses\s+: (\d+)$", t)
        if m:
            houses[cur] = int(m.group(1))
        m = re.match(r"^Hotels\s+: (\d+)$", t)
        if m:
            hotels[cur] = int(m.group(1))
        m = re.match(r"^Outstanding Loan : LKR ([0-9,]+)$", t)
        if m:
            loan[cur] = parse_money(m.group(1))
        m = re.match(r"^Status : (.+)$", t)
        if m:
            status[cur] = m.group(1)
    k += 1

for p in range(4):
    if self.bankrupt[p]:
        # a bankrupt player must show 0 everywhere
        if cash[p] not in (0, None):
            self.report("BUG",
                "Round %d: bankrupt %s has non-zero cash %d." %
                (self.summary_round, PLAYERS[p], cash[p]),
                start_line, "")
        if nw[p] not in (0, None):
            self.report("BUG",
                "Round %d: bankrupt %s has non-zero net "
                "worth %d." % (self.summary_round, PLAYERS[p],
                nw[p]), start_line, "")
        continue
    if cash[p] is not None:
        diff = cash[p] - self.cash[p]
        if diff != 0:
            self.report("REVIEW",
                "Round %d: %s reported cash %d vs tracked %d "
                "(diff %d)." % (self.summary_round, PLAYERS[p],
                cash[p], self.cash[p], diff),
                start_line, "")
            self.cash[p] = cash[p] # resync
    if props[p] is not None:
        if props[p] != self.count_owned(p, PROP_TYPE):
            self.report("BUG",
                "Round %d: %s Properties %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], props[p],
                self.count_owned(p, PROP_TYPE)),
                start_line, "")
    if rails[p] is not None:
        if rails[p] != self.count_owned(p, RAIL_TYPE):
            self.report("BUG",
                "Round %d: %s Railways %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], rails[p],
                self.count_owned(p, RAIL_TYPE)),

```

```

        start_line, "")
    if utils[p] is not None:
        if utils[p] != self.count_owned(p, UTIL_TYPE):
            self.report("BUG",
                "Round %d: %s Utilities %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], utils[p],
                 self.count_owned(p, UTIL_TYPE)),
                start_line, "")
    if houses[p] is not None:
        if houses[p] != self.count_houses(p):
            self.report("BUG",
                "Round %d: %s Houses %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], houses[p],
                 self.count_houses(p)), start_line, "")
    if hotels[p] is not None:
        if hotels[p] != self.count_hotels(p):
            self.report("BUG",
                "Round %d: %s Hotels %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], hotels[p],
                 self.count_hotels(p)), start_line, "")
    if loan[p] is not None:
        exp = self.loan_amount[p] if self.loan_active[p] else 0
        if loan[p] != exp:
            self.report("BUG",
                "Round %d: %s loan %d vs tracked %d." %
                (self.summary_round, PLAYERS[p], loan[p],
                 exp), start_line, "")
    if nw[p] is not None:
        exp = self.net_worth(p)
        if nw[p] != exp:
            self.report("REVIEW",
                "Round %d: %s net worth %d vs recomputed %d "
                "(diff %d)." % (self.summary_round, PLAYERS[p],
                               nw[p], exp, nw[p] - exp),
                start_line, "")

# liquidation completeness: bankrupt players' assets were auctioned
if hasattr(self, "pending_liquidation") and self.pending_liquidation:
    still = [i for i in self.pending_liquidation
              if self.owner[i] == self.pending_liquidation_player]
    if still:
        self.report("BUG",
            "Ghost owners after bankruptcy of %s: %s not "
            "reassigned." %
            (PLAYERS[self.pending_liquidation_player],
             [self.prop_name(x) for x in still]),
            start_line, "")
    self.pending_liquidation = []

self.lux_tax_fired = False
self.foreclose_pending = False

# -----
def parse_game_over(self, ln, i):
    # winner block: Winner / name / Total Cash / LKR x / Total Property
    # Value / LKR x / Outstanding Loans / LKR x|None / Net Worth / LKR x
    j = i + 1
    winner = None
    cash = None
    prop = None
    loans = None
    nw = None
    while j < self.n:
        t = self.lines[j].strip()
        if t == "Final Standings (all players)":
            break
        if t in P2I:
            winner = P2I[t]
        elif t == "Total Cash":
            if j + 1 < self.n:
                m = re.match(r"^LKR ([0-9,]+)$",
                             self.lines[j + 1].strip())
                if m:
                    cash = parse_money(m.group(1))
        elif t == "Total Property Value":
            if j + 1 < self.n:
                m = re.match(r"^LKR ([0-9,]+)$",

```

```

        self.lines[j + 1].strip())
        if m:
            prop = parse_money(m.group(1))
    elif t == "Outstanding Loans":
        if j + 1 < self.n:
            m = re.match(r"^LKR ([0-9,]+)$",
                self.lines[j + 1].strip())
            if m:
                loans = parse_money(m.group(1))
            else:
                loans = 0
    elif t == "Net Worth":
        if j + 1 < self.n:
            m = re.match(r"^LKR ([0-9,]+)$",
                self.lines[j + 1].strip())
            if m:
                nw = parse_money(m.group(1))
        j += 1

    if winner is not None and cash is not None and prop is not None \
        and loans is not None and nw is not None:
        if cash + prop - loans != nw:
            self.report("BUG",
                "Winner %s final arithmetic: %d + %d - %d = %d, "
                "printed net worth %d." %
                (PLAYERS[winner], cash, prop, loans,
                 cash + prop - loans, nw), ln, "")

# final standings: bankrupt players must be 0
cur = None
while j < self.n:
    t = self.lines[j].strip()
    if t in P2I:
        cur = P2I[t]
    elif cur is not None:
        m = re.match(r"^Cash : LKR ([0-9,]+)$", t)
        if m:
            c = parse_money(m.group(1))
            if self.bankrupt[cur] and c != 0:
                self.report("BUG",
                    "Final: bankrupt %s has cash %d." %
                    (PLAYERS[cur], c), ln, t)
        m = re.match(r"^Net Worth : LKR ([0-9,]+)$", t)
        if m:
            v = parse_money(m.group(1))
            if self.bankrupt[cur] and v != 0:
                self.report("BUG",
                    "Final: bankrupt %s has net worth %d." %
                    (PLAYERS[cur], v), ln, t)
    j += 1

def analyze(path):
    a = Analyzer(path)
    a.run()
    print("=" * 70)
    print("LOG: %s" % path)
    print("=" * 70)
    bugs = [f for f in a.findings if f["level"] == "BUG"]
    reviews = [f for f in a.findings if f["level"] == "REVIEW"]
    print("confirmed bugs: %d    review items: %d" % (len(bugs), len(reviews)))
    print("-" * 70)
    for f in bugs + reviews:
        print("[%s] line %d: %s" % (f["level"], f["line"], f["claim"]))
        if f["quote"]:
            print("    evidence: \"%s\"" % f["quote"])
    print()
    return a.findings

if __name__ == "__main__":
    for path in sys.argv[1:]:
        analyze(path)

```

A.15 Rent.csv - property price/rent data (24 lines)

Property Group,Property,Purchase Price (LKR),Base Rent (LKR)
 Brown,Pettah,"1,500",100

Brown, Maradana, "1,800", 120
Light Blue, Bambalapitiya, "2,500", 180
Light Blue, Wellawatte, "2,700", 200
Light Blue, Mount Lavinia, "3,000", 220
Pink, Nugegoda, "3,500", 260
Pink, Maharagama, "3,800", 280
Pink, Kottawa, "4,000", 300
Orange, Negombo, "4,500", 350
Orange, Katunayake, "4,700", 370
Orange, Ja-Ela, "5,000", 400
Red, Kandy City, "5,500", 450
Red, Peradeniya, "5,800", 480
Red, Katugastota, "6,000", 500
Yellow, Galle Fort, "6,500", 600
Yellow, Unawatuna, "6,800", 620
Yellow, Hikkaduwa, "7,000", 650
Green, Jaffna Town, "8,000", 750
Green, Nallur, "8,300", 780
Green, Trincomalee, "8,500", 800
Dark Blue, Nuwara Eliya, "10,000", "1,000"
Dark Blue, Galle Face, "12,000", "1,200"